

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2025

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2025

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2025

IOI China candidate team papers / National training team 2025 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2025. Tập được dàn trang bằng Adobe InDesign và có lớp văn bản đọc được cho mục lục cũng như các trang thân bài đã kiểm tra. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, bài đầu tiên của Liu Hengxi về đoạn liên tiếp cùng các bài toán liên quan, và bài thứ hai của Gou Jingyu về các phương pháp chứng minh tính lỗi trong đề OI, cùng bài thứ ba của Liu Haifeng về xử lý lân cận trên cây bằng phân rã chuỗi dài, và bài thứ tư của Xu Xiaoyang về nguyên lý chuyển vị trong một lớp bài toán quy hoạch động, cùng bài thứ năm của Chen Xinyang về cơ sở tuyển tính và các biến thể có hỗ trợ xóa, cùng bài thứ sáu của Fan Sizhe tổng kết một số ý tưởng cho các bài toán liên quan tới dãy, bài thứ bảy của Zheng Jun về dãy Thue–Morse, và bài thứ tám của Chen Nuo về nhân ma trận và phép quy giảm, cùng bài thứ chín của Wu Lin về một số bài toán kinh điển modulo lũy thừa của số nguyên tố nhỏ, và bài thứ mười của Xiao Daien về các phương pháp để cài đặt để duy trì thông tin trên cây động, cùng bài thứ mười một của Jiao Siyuan về các bài toán lân cận trên đồ thị, và bài thứ mười hai của Huang Jiaxu về duy trì thông tin bằng chia để trị trên cây đoạn.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2025*. Tập PDF gồm 168 trang; mục lục dùng số trang in của tập sách, chạy đến hơn 300 vì bản PDF được dàn trang dạng nhiều trang nội dung trên một trang PDF.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Bàn về đoạn liên tiếp và các bài toán liên quan	Liu Hengxi
22	Các phương pháp chứng minh tính lời trong đề OI	Gou Jingyu
49	Một lời giải dựa trên phân rã chuỗi dài cho bài toán lân cận trên cây	Liu Haifeng
61	Ứng dụng nguyên lý chuyển vị trong một lớp bài toán quy hoạch động	Xu Xiaoyang
73	Bàn về ứng dụng cơ sở tuyến tính trong OI	Chen Xinyang
101	Tổng kết một số ý tưởng cho các bài toán liên quan tới dãy	Fan Sizhe
115	Bàn về tính chất và ứng dụng của dãy Thue–Morse	Zheng Jun
139	Bàn về nhân ma trận và phép quy giảm	Chen Nuo
150	Cách làm một số bài toán cổ điển modulo lũy thừa của số nguyên tố nhỏ	Wu Lin
161	Bàn về một lớp phương pháp duy trì thông tin trên cấu trúc cây động dễ cài đặt	Xiao Daien
169	Bàn về một lớp bài toán lân cận trên đồ thị	Jiao Siyuan
179	Bàn về một lớp thuật toán duy trì thông tin dựa trên chia để trị bằng cây đoạn	Huang Jiaxu
187	Bàn về các bài toán liên quan tới biến ngẫu nhiên liên tục	Zhou Huanyi
207	Báo cáo ra đề <i>Địa ngục vô hạn</i>	Chen Xulei
213	Báo cáo lời giải <i>Cấu trúc dữ liệu</i>	Zhu Yifan
219	Bài toán xâu dưới quan hệ tương đương đặc biệt	Zhang Mixuan
228	Bàn về hình học tính toán không sai số chính xác	Yao Xi
243	Bàn về bài toán ba lô dung lượng lớn trong OI	Fang Xintong
251	Bàn về một lớp bài toán đổi gốc trên cây	Zhao Haikun
272	Một lớp bài toán xâu trên cây được giải bằng cấu trúc dữ liệu	Li Jingrong
282	Bàn về một lớp phương pháp trích hệ số của phân thức hữu tỉ	Huang Jianheng
293	Bàn về tensor và ứng dụng trong OI	Wei Zhonghao
314	Bàn về một lớp bài toán bất đẳng thức đồng dư tuyến tính	Yang Xinhe

Bài 1. Bàn về đoạn liên tiếp và các bài toán liên quan

Liu Hengxi, Trường Trung học Trần Hải, Ninh Ba

Tóm tắt

Đoạn liên tiếp là một đối tượng thường gặp trong các kỳ thi tin học. Bài viết này chuyển một loạt bài toán liên quan đến đoạn liên tiếp về một dạng thống nhất, khảo sát các tính chất của tập khoảng, đưa ra thuật toán cho một lớp tập khoảng đặc biệt, rồi dùng các thuật toán trên tập khoảng để giải một phần các bài toán. Ngoài ra, bài viết chứng minh độ khó của một số bài toán bằng quy giảm.

1. Mở đầu

Trong các bài toán thi tin học, ta thường gặp những mô tả như “giá trị của các phần tử tạo thành một khoảng” hoặc “ $r - l = \max - \min$ ”. Những bài toán kiểu này thường liên quan đến đoạn liên tiếp. Khi làm bài [CCPC Final 2021] *Tree Partition*, tác giả nhận thấy còn có cách làm tốt hơn, từ đó suy nghĩ về trường hợp tổng quát hơn và thu được một số kết quả trong bài viết này.

Tác giả mong rằng thông qua bài viết này, người đọc có thể hiểu bản chất của bài toán đoạn liên tiếp, nắm được một số tính chất của đoạn liên tiếp, và biết cách dùng thuật toán trên tập khoảng để giải một số bài toán.

2. Quy ước

Nếu không nói rõ, các số xuất hiện đều là số nguyên.

Với $l, r \in \mathbb{Z}$, $l \leq r$, khoảng $[l, r]$ biểu thị tập các số nguyên giữa l và r , tức là

$$[l, r] = \{i \mid i \in \mathbb{Z}, l \leq i \leq r\}.$$

Khi $l < r$, gọi $[l, r]$ là một khoảng không tầm thường. Độ dài của khoảng $[l, r]$ là số phần tử trong nó, tức là $r - l + 1$.

Với một hoán vị p , ký hiệu p^{-1} là hoán vị nghịch đảo của p , tức là $p_{p_i}^{-1} = i$.

Với một hoán vị p , ký hiệu $p_{[l,r]}$ là tập tạo bởi $p_l, \dots, p_r$, tức là

$$p_{[l,r]} = \{p_i \mid i \in [l, r]\}.$$

Nếu không nói rõ, kích thước của hoán vị p là n , chỉ số của p nằm trong $[1, n]$, và khoảng $[l, r]$ thỏa $1 \leq l \leq r \leq n$.

Ký hiệu U là tập tất cả các khoảng được chứa trong $[1, n]$, tức là

$$U = \{[l, r] \mid 1 \leq l \leq r \leq n\}.$$

Ký hiệu 2^A là tập lũy thừa của tập A , tức là $2^A = \{B \mid B \subseteq A\}$.

3. Đoạn liên tiếp

3.1. Dẫn nhập

Định nghĩa 3.1 (đoạn liên tiếp của hoán vị). Với một hoán vị p , nếu khoảng $[l, r]$ thỏa $p_{[l,r]} \in U$, thì gọi $[l, r]$ là một đoạn liên tiếp của p .

Trong các kỳ thi tin học, ta thường gặp các bài toán về đoạn liên tiếp và các mở rộng liên quan. Mở rộng có thể là thay khoảng của hoán vị bằng cây con liên thông của cây, hoặc thay một hoán vị bằng nhiều hoán vị. Những mở rộng này thường đều có thể biểu diễn dưới một dạng thống nhất.

3.2. Dạng thống nhất

Giả sử có m cấu trúc và n phần tử. Gọi tập tạo bởi n phần tử là A ; để tiện, có thể xem $A = [1, n]$. Mỗi cấu trúc chứa n phần tử, và trên mỗi cấu trúc ta định nghĩa một tập con của A có “liên thông” hay không. Ta cần tìm thông tin về các tập con của A đồng thời “liên thông” trên cả m cấu trúc.

Xem các tập con của A đồng thời “liên thông” trên m cấu trúc là các đoạn liên tiếp suy rộng.

Việc định nghĩa mỗi tập con của A có “liên thông” trên một cấu trúc hay không tương đương với việc cấu trúc đó có một tập liên thông $C \subseteq 2^A$ chứa tất cả các tập con “liên thông”.

Do đó bài toán cũng có thể phát biểu là: có n phần tử, gọi tập của chúng là A , và có m tập con $C_1, \dots, C_m$ của 2^A ; ta cần tìm thông tin về

$$\bigcap_i C_i.$$

Trong các bài toán thi tin học, những cấu trúc thường gặp gồm:

- Hoán vị, tức dãy chuyển: tập liên thông của hoán vị p thường được định nghĩa là $\{p_{[l,r]} \mid [l,r] \in U\}$, tức là các khoảng của p . Tập liên thông này cũng chính là tập liên thông của dãy chuyển $T = (V, E)$, $V = [1, n]$, $E = \{(p_i, p_{i+1}) \mid i \in [1, n-1]\}$.
- Cây: với cây $T = (V, E)$, $V = A$, tập liên thông thường được định nghĩa là

$$\{S \mid S \subseteq V, T[S] \text{ là đồ thị liên thông}\}.$$

Định nghĩa này tương đương với

$$\{S \mid S \subseteq V, |\{(u, v) \in E \mid u, v \in S\}| = |S| - 1\}.$$

Ở đây $G[S]$ là đồ thị con cảm sinh bởi tập đỉnh S .

- Đồ thị: với đồ thị $G = (V, E)$, $V = A$, tập liên thông thường được định nghĩa là $\{S \mid S \subseteq V, G[S] \text{ là đồ thị liên thông}\}$.

Những tổ hợp cấu trúc có thể xuất hiện gồm:

- hai dãy chuyển, tức đoạn liên tiếp của hoán vị;
- nhiều dãy chuyển hơn;
- dãy chuyển và cây;
- dãy chuyển và nhiều cây;
- dãy chuyển và đồ thị;
- hai cây.

Với tổ hợp không chứa dãy chuyển, chẳng hạn hai cây, việc đếm số đoạn liên tiếp đã khá khó.

Với tổ hợp có chứa dãy chuyển, số đoạn liên tiếp không vượt quá $O(n^2)$, và bài toán tương ứng cũng tương đối dễ hơn. Ta có thể đánh số lại các phần tử thành $1, 2, \dots, n$ theo thứ tự trên một dãy chuyển. Sau khi đánh số lại, mọi đoạn liên tiếp đều là khoảng, vì vậy ta cần khảo sát tính chất của các tập khoảng.

4. Tập khoảng

Định nghĩa 4.1 (chính quy). Nếu họ tập S thỏa

$$S \subseteq U, \quad [1, n] \in S, \quad \forall i \in [1, n], \{i\} \in S,$$

thì gọi S là chính quy. Ở đây họ tập là một tập có phần tử là các tập.

Định nghĩa 4.2 (đóng loại I). Nếu họ tập chính quy S thỏa

$$\forall A, B \in S, \quad A \cap B \neq \emptyset \Rightarrow A \cup B \in S,$$

thì gọi S là đóng loại I.

Định nghĩa 4.3 (đóng loại II). Nếu họ tập chính quy S thỏa

$$\forall A, B \in S, \quad A \cap B \neq \emptyset \Rightarrow A \cap B \in S,$$

thì gọi S là đóng loại II.

Định nghĩa 4.4 (đóng loại III). Nếu họ tập chính quy S thỏa

$$\forall A, B \in S, \quad A \cap B \neq \emptyset \wedge A \not\subseteq B \wedge B \not\subseteq A \Rightarrow A \setminus B \in S,$$

thì gọi S là đóng loại III.

Ta có ngay các kết luận sau từ định nghĩa:

1. Giao của một số họ tập chính quy là họ tập chính quy.
2. Giao của một số họ tập đóng loại I là họ tập đóng loại I.
3. Giao của một số họ tập đóng loại II là họ tập đóng loại II.
4. Giao của một số họ tập đóng loại III là họ tập đóng loại III.

4.1. Tập khoảng đóng loại I và II

Định nghĩa 4.5 (khoảng cực tiểu). Giả sử tập khoảng S là tập khoảng đóng loại I và II. Với $i \in [1, n-1]$, nếu khoảng $[l, r]$ chứa $[i, i+1]$, thuộc S , và mọi khoảng như vậy đều chứa $[l, r]$, tức là

$$[l, r] \supseteq [i, i+1], \quad [l, r] \in S, \quad \forall [l', r'] \in S, \quad [l', r'] \supseteq [i, i+1] \Rightarrow [l', r'] \supseteq [l, r],$$

thì gọi $[l, r]$ là khoảng cực tiểu của S chứa $[i, i+1]$, ký hiệu

$$M_S(i) = [ml_S(i), mr_S(i)].$$

Dãy gồm $n-1$ khoảng cực tiểu được gọi là dãy khoảng cực tiểu của S , ký hiệu M_S .

Định lý 4.1. Giả sử tập khoảng S đóng loại I và II. Với mọi $i \in [1, n-1]$, tồn tại khoảng cực tiểu của S chứa $[i, i+1]$.

Chứng minh. Với S đóng loại I và II và $i \in [1, n-1]$, lấy

$$[l, r] = \bigcap_{[x, y] \supseteq [i, i+1] \wedge [x, y] \in S} [x, y].$$

Vì $[1, n] \in S$, về phải có ít nhất một khoảng được lấy giao. Rõ ràng $[l, r] \supseteq [i, i+1]$, và nếu $[l', r'] \supseteq [i, i+1]$ thì $[l', r'] \supseteq [l, r]$. Chỉ cần chứng minh $[l, r] \in S$. Lần lượt áp dụng tính đóng loại II của S cho các khoảng ở về phải, suy ra $[l, r] \in S$. $\dashv$

Định lý 4.2. Giả sử tập khoảng S đóng loại I và II. Với khoảng không tầm thường $[l, r]$ ($l < r$), điều kiện cần và đủ để $[l, r] \in S$ là

$$\forall i \in [l, r-1], \quad M_S(i) \subseteq [l, r].$$

Chứng minh. Xét tập khoảng S đóng loại I và II và khoảng không tầm thường $[l, r]$.

Tính đủ: nếu $\forall i \in [l, r-1], M_S(i) \subseteq [l, r]$, thì

$$[l, r] = \bigcup_{i=l}^{r-1} M_S(i).$$

Với $i \in [l+1, r-1]$, khi lấy hợp $M_S(i)$ với $\bigcup_{j=l}^{i-1} M_S(j)$, ta có $i \in M_S(i)$ và $i \in \bigcup_{j=l}^{i-1} M_S(j)$. Vì vậy có thể lần lượt áp dụng tính đóng loại I của S , suy ra $[l, r] \in S$.

Tính cần: nếu $[l, r] \in S$, theo định nghĩa khoảng cực tiểu, với mọi $i \in [l, r-1]$, rõ ràng $[l, r] \supseteq [i, i+1]$, nên $M_S(i) \subseteq [l, r]$. $\dashv$

Từ định lý trên, một tập khoảng đóng loại I và II có thể được xác định bởi $n-1$ khoảng cực tiểu.

4.2. Tập khoảng đóng loại I, II và III

Bổ đề 4.1. Giả sử tập khoảng S đóng loại I, II và III. Nếu i, j ($1 \leq i < j \leq n - 1$) thỏa

$$i + 1 \leq ml_S(j) \leq mr_S(i) \leq j,$$

thì $ml_S(j) = i + 1$ và $mr_S(i) = j$.

Chứng minh. Giả sử $i + 1 < ml_S(j) \leq mr_S(i) \leq j$. Theo định nghĩa khoảng cực tiểu, $[ml_S(i), mr_S(i)]$ và $[ml_S(j), mr_S(j)]$ đều thuộc S . Vì S đóng loại III,

$$[ml_S(i), mr_S(i)] \setminus [ml_S(j), mr_S(j)] = [ml_S(i), ml_S(j) - 1] \in S.$$

Khoảng này chứa $[i, i + 1]$, mâu thuẫn với việc $[ml_S(i), mr_S(i)]$ là khoảng cực tiểu chứa $[i, i + 1]$. Do đó $ml_S(j) = i + 1$. Tương tự, $mr_S(i) = j$. $\dashv$

Lấy T là tập các chỉ số i sao cho khoảng cực tiểu chứa $[i, i + 1]$ không bị một khoảng cực tiểu khác chứa thật:

$$T = \{i \in [1, n - 1] \mid \neg(\exists j \in [1, n - 1], M_S(i) \subsetneq M_S(j))\}.$$

Bổ đề 4.2. Với $i \in [1, n - 1]$, nếu tồn tại $t \in T$ sao cho $M_S(i) \supseteq [t, t + 1]$, thì $i \in T$.

Chứng minh. Giả sử $i \notin T$. Khi đó tồn tại $j \in [1, n - 1]$ sao cho $M_S(i) \subsetneq M_S(j)$. Vì $M_S(t)$ là khoảng cực tiểu chứa $[t, t + 1]$ và $M_S(i) \supseteq [t, t + 1]$, ta có $M_S(t) \subseteq M_S(i)$. Do $M_S(i) \subsetneq M_S(j)$, suy ra $M_S(t) \subseteq M_S(i) \subsetneq M_S(j)$, mâu thuẫn với $t \in T$. $\dashv$

Bổ đề 4.3. Với mọi $i \in [1, n - 1]$, tồn tại $t \in T$ sao cho $M_S(t)$ chứa $[i, i + 1]$.

Chứng minh. Với $i \in [1, n - 1]$, trong các $M_S(j)$ chứa $[i, i + 1]$, chọn khoảng có độ dài lớn nhất và gọi là $M_S(t)$. Nếu $M_S(t)$ bị một $M_S(u)$ chứa thật, thì $M_S(u)$ cũng chứa $[i, i + 1]$ và có độ dài lớn hơn, mâu thuẫn. Vì vậy $M_S(t)$ không bị khoảng cực tiểu khác chứa thật, tức là $t \in T$. $\dashv$

Bổ đề 4.4. Nếu tồn tại $i, j \in T$, $i \neq j$, sao cho $M_S(i) = M_S(j)$, thì $\forall t \in T$, $M_S(t) = [1, n]$.

Chứng minh. Giả sử $i, j \in T$, $i \neq j$, và $M_S(i) = M_S(j)$. Nếu $ml_S(i) \neq 1$, lấy $k = ml_S(i) - 1$.

Nếu $mr_S(k) \geq i + 1$, thì $M_S(k) \supseteq [i, i + 1]$. Vì $[ml_S(i), mr_S(i)]$ là khoảng cực tiểu chứa $[i, i + 1]$, $M_S(k) \supseteq M_S(i)$. Lại có $k \in M_S(k)$ và $k \notin M_S(i)$, nên $M_S(k) \supsetneq M_S(i)$, mâu thuẫn với $i \in T$.

Nếu $mr_S(k) \leq i$, thì $k \leq ml_S(i) \leq mr_S(k) \leq i$. Theo bổ đề 4.1, $mr_S(k) = i$. Mặt khác, $k \leq ml_S(j) \leq mr_S(k) \leq j$, nên theo bổ đề 4.1, $mr_S(k) = j$, mâu thuẫn.

Do đó $ml_S(i) = 1$. Tương tự, $mr_S(i) = n$. Theo tính chất của T , $\forall t \in T$, không thể có $M_S(t) \subsetneq M_S(i) = [1, n]$, do đó $\forall t \in T$, $M_S(t) = [1, n]$. $\dashv$

Bổ đề 4.5. Khi các $M_S(t)$, $t \in T$, đôi một khác nhau, không tồn tại $i, j \in T$, $i \neq j$, sao cho $M_S(i) \supseteq [j, j + 1]$.

Chứng minh. Nếu tồn tại $i, j \in T$, $i \neq j$, sao cho $M_S(i) \supseteq [j, j + 1]$, thì theo định nghĩa khoảng cực tiểu, $M_S(i) \supseteq M_S(j)$. Do $M_S(i) \neq M_S(j)$, ta có $M_S(i) \supsetneq M_S(j)$, mâu thuẫn với $j \in T$. $\dashv$

Bổ đề 4.6. Khi các $M_S(t)$, $t \in T$, đôi một khác nhau, giả sử $T = \{t_1, t_2, \dots, t_k\}$, $t_0 = 0 < t_1 < t_2 < \dots < t_k < t_{k+1} = n$. Khi đó

$$M_S(t_i) = [t_{i-1} + 1, t_{i+1}].$$

Chứng minh. Theo bổ đề 4.5, $M_S(t_i) \subseteq [t_{i-1} + 1, t_{i+1}]$. Chỉ cần chứng minh

$$ml_S(t_i) = \begin{cases} 1, & i = 1, \\ t_{i-1} + 1, & i \geq 2, \end{cases}$$

phần về $mr_S(t_i)$ tương tự.

Nếu $ml_S(t_1) > 1$, thì $[1, 2]$ không bị bất kỳ $M_S(t_i)$ nào chứa, mâu thuẫn với bổ đề 4.3. Vì vậy $ml_S(t_1) = 1$.

Với $i \in [2, k]$, nếu $mr_S(t_{i-1}) < ml_S(t_i)$, thì $[ml_S(t_i) - 1, ml_S(t_i)]$ không bị bất kỳ $M_S(t_i)$ nào chứa, mâu thuẫn với bổ đề 4.3. Do đó $t_{i-1} + 1 \leq ml_S(t_i) \leq mr_S(t_{i-1}) \leq t_i$. Theo bổ đề 4.1, $ml_S(t_i) = t_{i-1} + 1$. $\dashv$

Định lý 4.3. Giả sử tập khoảng S đóng loại I, II và III. Khi $n \geq 2$, luôn có thể chia khoảng $[1, n]$ thành k khoảng con $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$, $k \geq 2$, $l_1 = 1$, $r_k = n$, $l_i \leq r_i$, $r_i + 1 = l_{i+1}$, sao cho:

- Với mọi $i \in [1, k]$, $[l_i, r_i] \in S$.
- Mọi phần tử của S hoặc bị chứa trong một khoảng con $[l_i, r_i]$, hoặc có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn liên tiếp, tức là

$$S \subseteq \bigcup_{i=1}^k \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\} \cup \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

- Tập các khoảng liên tiếp có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn thỏa một trong hai điều kiện:

$$S \supseteq \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

Khi đó gọi phép chia này là phép chia hợp.

- Hoặc S chỉ chứa các khoảng con nguyên vẹn và $[1, n]$, đồng thời $k \geq 3$:

$$S \cap \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\} = \{[l_i, r_i] \mid i \in [1, k]\} \cup \{[1, n]\}.$$

Khi đó gọi phép chia này là phép chia tách.

Chứng minh. Lấy

$$T = \{t_1, t_2, \dots, t_k\} = \{i \in [1, n-1] \mid \neg(\exists j \in [1, n-1], M_S(i) \subsetneq M_S(j))\},$$

với $t_0 = 0 < t_1 < \dots < t_k < t_{k+1} = n$. Chia $[1, n]$ thành

$$[t_0 + 1, t_1], [t_1 + 1, t_2], \dots, [t_k + 1, t_{k+1}].$$

Với khoảng con $[t_i + 1, t_{i+1}]$, theo bổ đề 4.2, $\forall j \in [t_i + 1, t_{i+1} - 1]$, $M_S(j) \subseteq [t_i + 1, t_{i+1}]$. Theo định lý 4.2, $[t_i + 1, t_{i+1}] \in S$.

Nếu các $M_S(t)$, $t \in T$, đôi một khác nhau, theo bổ đề 4.6, $M_S(t_i) = [t_{i-1} + 1, t_{i+1}]$. Theo định lý 4.2, phép chia này thỏa $S \supseteq \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}$, nên là phép chia hợp.

Ngược lại, $k \geq 2$ và theo bổ đề 4.4, $\forall t \in T, M_S(t) = [1, n]$. Theo định lý 4.2, phép chia này thỏa

$$S \cap \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\} = \{[l_i, r_i] \mid i \in [1, k]\} \cup \{[1, n]\},$$

nên là phép chia tách.

Trong cả hai trường hợp đều có $M_S(t_i) \supseteq [t_{i-1} + 1, t_{i+1}]$. Với

$$[l', r'] \in S \setminus \bigcup_{i=1}^k \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\},$$

giả sử $t_i + 1 \leq l' \leq t_{i+1} \leq t_j + 1 \leq r' \leq t_{j+1}$. Theo định lý 4.2, $[l', r'] \supseteq M_S(t_{i+1})$ và $[l', r'] \supseteq M_S(t_j)$, do đó $[l', r'] = [t_i + 1, t_{j+1}]$. Suy ra phép chia thỏa điều kiện đã nêu. $\dashv$

Định lý 4.4. Phép chia trong định lý 4.3 là duy nhất.

Chứng minh. Giả sử trên S , khoảng $[1, n]$ có hai phép chia:

$$[l_{i,1}, r_{i,1}], [l_{i,2}, r_{i,2}], \dots, [l_{i,k_i}, r_{i,k_i}] \quad (i \in [1, 2], k_i \geq 2).$$

Tìm i nhỏ nhất sao cho $[l_{1,i}, r_{1,i}] \neq [l_{2,i}, r_{2,i}]$; không mất tổng quát, giả sử $r_{1,i} < r_{2,i}$.

Nếu phép chia 1 là phép chia hợp, thì khi $i = 1$, đối với phép chia 2, khoảng $[r_{1,i} + 1, n]$ vừa không bị một khoảng con $[l_{2,j}, r_{2,j}]$ chứa, vừa không thể biểu diễn thành hợp của một số khoảng con nguyên vẹn, mâu thuẫn. Khi $i \geq 2$, đối với phép chia 2, khoảng $[1, r_{1,i}]$ cũng gây mâu thuẫn tương tự.

Nếu phép chia 1 là phép chia tách, thì khoảng $[l_{2,i}, r_{2,i}]$ vừa không bị một khoảng con $[l_{1,i}, r_{1,i}]$ chứa, vừa không phải $[1, n]$, mâu thuẫn. Vì vậy giả thiết ban đầu không đúng. $\dashv$

4.3. Cây tách–hợp

Cây tách–hợp do Liu Cheng’ao nêu ra trong trao đổi học viên WC2019 và cho thuật toán xây dựng tuyến tính; có thể xem tài liệu [3]. Định nghĩa ở đây hơi khác định nghĩa trong slide trao đổi ban đầu, nhưng bản chất là như nhau.

Định nghĩa 4.6 (cây tách–hợp). Cây tách–hợp của một tập khoảng là cây có gốc gồm ba loại nút:

- Nút lá (L, i) , biểu diễn khoảng $[i, i]$.
- Nút hợp (J, l, r) , biểu diễn khoảng $[l, r]$.
- Nút tách (C, l, r) , biểu diễn khoảng $[l, r]$.

Cây tách–hợp của tập khoảng S phải thỏa các ràng buộc sau:

- Nút gốc biểu diễn khoảng $[1, n]$.
- Mọi khoảng do nút biểu diễn đều thuộc S .
- Mỗi nút không phải lá có ít nhất hai con, và các khoảng do các con này biểu diễn tạo thành một phép chia của khoảng do nút đó biểu diễn. Riêng nút tách có ít nhất ba con.
- Với nút không phải lá (c, u, v) , $c \in \{J, C\}$, giả sử các khoảng do con của nó biểu diễn là $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$, $k \geq 2$, $l_1 = u$, $r_k = v$, $l_i \leq r_i$, $r_i + 1 = l_{i+1}$. Mọi khoảng trong S nằm trong $[u, v]$ hoặc bị chứa trong một khoảng con $[l_i, r_i]$, hoặc có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn:

$$S \cap \{[l, r] \mid [l, r] \subseteq [u, v]\} \subseteq \bigcup_{i=1}^k \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\} \cup \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

- Với nút hợp (J, u, v) , S chứa mọi khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn:

$$S \supseteq \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

- Với nút tách (C, u, v) , $k \geq 3$ và trong S , những khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn chỉ gồm các khoảng con và khoảng toàn bộ:

$$S \cap \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\} = \{[l_i, r_i] \mid i \in [1, k]\} \cup \{[u, v]\}.$$

Ở đây L viết tắt của leaf, J của join, và C của cut.

Định lý 4.5. Với tập khoảng S đóng loại I, II và III, cây tách–hợp của S tồn tại và duy nhất.

Chứng minh. Khi $n = 1$, cây chỉ gồm một nút lá $(L, 1)$ thỏa điều kiện, và hiển nhiên không có cây nào khác.

Giả sử kết luận đúng với $n \leq N$ ($N \geq 1$). Khi $n = N + 1 \geq 2$, chia $[1, n]$ theo định lý 4.3 thành k khoảng con $[l_1, r_1], \dots, [l_k, r_k]$. Để kiểm tra rằng với mọi $i \in [1, k]$,

$$S \cap \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\},$$

sau khi tịnh tiến các phần tử, vẫn đóng loại I, II và III. Theo giả thiết quy nạp, mỗi tập con này có cây tách–hợp duy nhất.

Nếu S chứa mọi khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn, tạo nút hợp $(J, 1, n)$. Ngược lại, trong S , các khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn chỉ là $[l_1, r_1], \dots, [l_k, r_k]$ và $[1, n]$; tạo nút tách $(C, 1, n)$. Gắn gốc của k cây con sau khi tịnh tiến về vị trí ban đầu làm k con của nút mới. Để kiểm tra cây thu được thỏa điều kiện, nên cây tách–hợp của S tồn tại.

Khi $n = N + 1 \geq 2$, các khoảng do con của gốc biểu diễn tạo thành phép chia trong định lý 4.3. Theo định lý 4.4, phép chia này duy nhất, nên với mọi cây tách–hợp của S , kiểu của gốc và các khoảng do con của gốc biểu diễn đều giống nhau. Theo giả thiết quy nạp, mỗi cây con cũng duy nhất, do đó cây tách–hợp của S duy nhất. $\dashv$

Định lý 4.6. Với tập khoảng S , cây tách–hợp của S tồn tại khi và chỉ khi S đóng loại I, II và III.

Chứng minh. Chiều đủ đã được định lý 4.5 cho.

Chiều cần: giả sử cây tách–hợp của S tồn tại. Vì trong cây chắc chắn có n nút lá (L, i) , $i \in [1, n]$, và nút gốc biểu diễn $[1, n]$, ta có $[1, n] \in S$ và $\forall i \in [1, n]$, $\{i\} \in S$, nên S chính quy.

Với hai khoảng $[l_1, r_1]$, $[l_2, r_2]$ có giao không rỗng, gọi u là nút sâu nhất có khoảng biểu diễn chứa $[l_1, r_1]$, và v là nút sâu nhất có khoảng biểu diễn chứa $[l_2, r_2]$. Lấy $i \in [l_1, r_1] \cap [l_2, r_2]$. Vì u và v đều là tổ tiên của (L, i) , chúng có quan hệ tổ tiên–hậu duệ; không mất tổng quát, nếu $u \neq v$ thì u là tổ tiên của v .

Nếu $u = v$, xét theo loại của u . Nếu u là lá hoặc nút tách, thì $[l_1, r_1]$ và $[l_2, r_2]$ đều chính là khoảng do u biểu diễn, nên hợp và giao của chúng đều thuộc S . Nếu u là nút hợp, thì $[l_1, r_1] \cap [l_2, r_2]$ và $[l_1, r_1] \cup [l_2, r_2]$ đều là hợp của một số khoảng con trong phép chia của u , nên thuộc S .

Nếu u là tổ tiên của v , thì $[l_1, r_1] \cup [l_2, r_2] = [l_1, r_1] \in S$. Vì vậy S đóng loại I và II.

Nếu $[l_1, r_1]$ và $[l_2, r_2]$ không chứa nhau, thì $u = v$ và u là nút hợp. Khi đó $[l_1, r_1] \setminus [l_2, r_2]$ là hợp của một số khoảng con trong phép chia của khoảng do u biểu diễn, nên theo tính chất của nút hợp, nó thuộc S . Do đó S đóng loại III. $\dashv$

Ví dụ, với

$$S = \{[1, 9], [1, 2], [3, 6], [5, 6], [7, 9], [7, 8], [8, 9]\} \cup \{[i, i] \mid i \in [1, 9]\},$$

cây tách–hợp có gốc là nút tách $C, 1, 9$; các nút trong cây gồm các nút hợp $J, 1, 2, J, 5, 6, J, 7, 9$, nút tách $C, 3, 6$, và các lá $L, 1, \dots, L, 9$.

4.4. Xây dựng cây tách–hợp

Trong bài toán thực tế, dãy khoảng cực tiểu của tập khoảng thường không dễ tìm nhanh, vì vậy ta đưa vào dãy khoảng cực tiểu giả.

Định nghĩa 4.7 (dãy khoảng cực tiểu giả). Dãy khoảng $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của tập khoảng đóng loại I và II S khi và chỉ khi

$$\forall i \in [1, n-1], \quad [l_i, r_i] \supseteq [i, i+1],$$

và

$$\forall 1 \leq u \leq v \leq n, \quad [u, v] \in S \iff (\forall i \in [u, v-1], [l_i, r_i] \subseteq [u, v]).$$

Nói cách khác, dãy khoảng cực tiểu giả có tính chất của khoảng cực tiểu được mô tả trong định lý 4.2.

4.4.1. Thuật toán thô. Cho dãy khoảng cực tiểu giả của tập khoảng đóng loại I, II và III S , có thể dùng phương pháp tham lam gộp dần để xây dựng cây tách–hợp của S .

Thuật toán 1. Xây dựng thô cây tách–hợp từ dãy khoảng cực tiểu giả.

Yêu cầu: $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của S . Kết quả: ngăn xếp s cuối cùng chứa nút gốc của cây tách–hợp.

1. $s \leftarrow$ ngăn xếp rỗng.
2. Với $i = 1$ đến n :
3. $u \leftarrow \text{Node}(L, i, i, \emptyset)$, lần lượt biểu thị kiểu, trái, phải và danh sách con.
4. $\text{sufmin} \leftarrow i, \text{sufmax} \leftarrow i, m \leftarrow |s|$.
5. Với $j = m$ giảm đến 1:
6. $\text{sufmin} \leftarrow \min\{\text{sufmin}, l_{s_j.\text{right}}\}$.
7. $\text{sufmax} \leftarrow \max\{\text{sufmax}, r_{s_j.\text{right}}\}$.
8. Nếu $\text{sufmin} \geq s_j.\text{left}$ và $\text{sufmax} \leq i$:
9. Nếu $j = |s|$:
10. Nếu $s_j.\text{type} = J$ và $\text{sufmin} \geq s_j.\text{child}_{|s_j.\text{child}|}.\text{left}$, thêm u vào con của s_j , đặt $s_j.\text{right} \leftarrow i$, $u \leftarrow s_j$.
11. Ngược lại, đặt $u \leftarrow \text{Node}(J, s_j.\text{left}, i, (s_j, u))$.
12. Ngược lại, đặt $u \leftarrow \text{Node}(C, s_j.\text{left}, i, (s_j, s_{j+1}, \dots, s_{|s|}, u))$.
13. Trong khi $|s| \geq j$, lấy phần tử khỏi s .
14. Đưa u vào s .

Có thể thấy các nút mới mà thuật toán tạo ra luôn biểu diễn các khoảng thuộc S . Với một nút không phải lá trong cây tách–hợp của S , thuật toán không gộp các nút thuộc các cây con khác nhau, đồng thời thiết lập đúng nút hợp và nút tách. Vì vậy thuật toán luôn cho kết quả đúng.

4.4.2. Thuật toán tuyến tính. Do tổng số nút của cây tách–hợp là $O(n)$, nút thất của thuật toán thô là tìm nút mới cần tạo.

Trong thuật toán thô, nếu $sufmax > i$, thì với i hiện tại không thể tạo nút mới nữa, nên có thể thoát ngay khỏi vòng lặp trong. Nếu $l_{s_j.right} < s_j.left$, thì s_j về sau không thể làm đầu trái của một nút mới nào nữa, nên không cần xét lại. Dùng hai ý tưởng tối ưu này, ta thu được thuật toán tuyến tính.

Thuật toán 2. Xây dựng tuyến tính cây tách–hợp từ dãy khoảng cực tiểu giả.

Yêu cầu: $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của S . Kết quả: ngăn xếp s cuối cùng chứa nút gốc của cây tách–hợp.

1. Đặt $l_n \leftarrow 1$, $s \leftarrow$ ngăn xếp rỗng, $t \leftarrow$ ngăn xếp rỗng.
2. Với $i = 1$ đến n :
3. $u \leftarrow \text{Node}(L, i, i, \emptyset)$.
4. Trong khi t không rỗng và $r_{t.top()} \leq i$:
5. $j \leftarrow |s|$.
6. Trong khi $s_j.left > t.top()$, đặt $j \leftarrow j - 1$.
7. Nếu $j = |s|$:
8. Nếu $s_j.type = J$ và $l_{s_j.right} \geq s_j.child_{|s_j.child|-1}.left$, thêm u vào con của s_j , đặt $s_j.right \leftarrow i$, $u \leftarrow s_j$.
9. Ngược lại, đặt $u \leftarrow \text{Node}(J, s_j.left, i, (s_j, u))$.
10. Nếu không, đặt $u \leftarrow \text{Node}(C, s_j.left, i, (s_j, s_{j+1}, \dots, s_{|s|}, u))$.
11. Trong khi $|s| \geq j$, lấy phần tử khỏi s .
12. Lấy phần tử khỏi t .
13. Đưa u vào s .
14. Đưa $u.left$ vào t .
15. Trong khi $t.top() > l_i$, lấy phần tử khỏi t .
16. Đặt $r_{t.top()} \leftarrow \max\{r_{t.top()}, r_i\}$.

Có thể kiểm tra thuật toán tuyến tính trên tương đương với thuật toán thô, nên nó luôn cho kết quả đúng.

4.5. Tổng thông tin tại đầu trái ứng với đầu phải

Cho tập khoảng S và n thông tin $a_1, a_2, \dots, a_n$. Phép gộp thông tin thỏa tính chất nửa nhóm, tức là đóng và kết hợp; ký hiệu phép gộp là $\times$. Cần với mỗi $r \in [1, n]$ tính

$$\prod_{l \in [1, r], [l, r] \in S} a_l.$$

Với tập khoảng đóng loại I, II và III, có thể dựng cây tách–hợp rồi giải bằng một lần DFS.

Với tập khoảng đóng loại I và II, nếu phép gộp thông tin thỏa tính chất nhóm, tức ngoài tính chất nửa nhóm còn có đơn vị và mỗi phần tử có nghịch đảo, thì có thuật toán tuyến tính. Ký hiệu đơn vị là 1, nghịch đảo của a là a^{-1} .

Thuật toán 3. Tuyến tính để tìm tích thông tin tại đầu trái ứng với đầu phải.

Yêu cầu: $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của S , và phép nhân của các a_i thỏa tính chất nhóm. Kết quả:

$$b_i = \prod_{j \in [1, i], [i, j] \in S} a_j.$$

1. Với $i = 1$ đến $n - 1$, đặt $c_i \leftarrow 0$.
2. $t \leftarrow$ ngăn xếp rỗng.
3. Với $i = n - 1$ giảm đến 1:
4. Đưa i vào t .
5. Trong khi $t.top() < r_i - 1$:
6. $c_{t.top()} \leftarrow i$.
7. Lấy phần tử khỏi t .
8. $s \leftarrow$ ngăn xếp rỗng, đưa 0 vào s .
9. $p_0 \leftarrow 1, b_1 \leftarrow a_1$.
10. Với $i = 1$ đến $n - 1$:
11. $p_i \leftarrow p_{s.top()} \times a_i$.
12. Đưa i vào s .
13. Trong khi $s.top() > l_i$:
14. $\text{Erase}(s.top())$.
15. Lấy phần tử khỏi s .
16. $v \leftarrow \text{Query}(c_i)$.
17. $b_{i+1} \leftarrow p_v^{-1} \times p_{s.top()} \times a_{i+1}$.

Giải thích. Trong thuật toán, c_i là k lớn nhất trong $[1, i]$ sao cho $r_k > i + 1$; nếu không tồn tại thì $c_i = 0$. Ràng buộc trên r trong dãy khoảng cực tiểu giả được chuyển thành $j > c_i$, còn ràng buộc trên l được chuyển thành thao tác lấy khỏi đỉnh ngăn xếp s . Thuật toán cần một cấu trúc dữ liệu:

- Duy trì một xâu 01 độ dài $n + 1$, ban đầu toàn 1.
- $\text{Erase}(x)$: đổi s_x thành 0.
- $\text{Query}(x)$: truy vấn y lớn nhất sao cho $y \leq x$ và $s_y = 1$.

Cài đặt cấu trúc dữ liệu. Trong mô hình Word RAM, có thể dùng DSU nén bit để cài đặt cấu trúc dữ liệu này. Đại khái, chia xâu s thành các khối w bit, xử lý trong khối bằng phép toán bit trong $O(1)$, còn giữa các khối dùng DSU có nén đường đi và gộp theo kích thước. Theo [2], độ phức tạp là $O(n\alpha(n, n/w)) = O(n)$.

Thuật toán trên, cũng như cách giải bằng cây tách–hợp, đều là trực tuyến: có thể lần lượt tính đáp án cho $r = 1, 2, \dots, n$. Khi tính đáp án cho $r = r_0$, không cần biết $a_{r_0+1}, \dots, a_n$.

5. Phân tích thêm về đoạn liên tiếp

Tập liên thông của dây chuyền, tức hoán vị, là chính quy và đóng loại I, II, III. Tập liên thông của cây là chính quy và đóng loại I, II. Tập liên thông của đồ thị là chính quy và đóng loại I.

Vì giao của một số họ tập cùng thỏa một trong các tính chất chính quy, đóng loại I, đóng loại II, đóng loại III vẫn thỏa tính chất đó, ta có thể trực tiếp suy ra tính chất của một số tổ hợp cấu trúc.

5.1. Hai dây chuyền

5.1.1. Tính chất. Đánh số lại các phần tử trên một dây chuyền theo thứ tự $1, 2, \dots, n$, ta thu được rằng đoạn liên tiếp của hai dây chuyền tương đương với đoạn liên tiếp của hoán vị. Tương tự, mọi tổ hợp chứa dây chuyền đều có thể chuyển đổi theo cách này.

Tập đoạn liên tiếp của hai dây chuyền là chính quy và đóng loại I, II, III, nghĩa là có thể dùng cây tách–hợp để mô tả đoạn liên tiếp của hoán vị.

Một dãy khoảng cực tiểu giả của tập đoạn liên tiếp của hoán vị p là

$$\left(\left[\min_{j \in A_i} p_j^{-1}, \max_{j \in A_i} p_j^{-1} \right] \right)_{i \in [1, n-1]},$$

trong đó

$$A_i = [\min\{p_i, p_{i+1}\}, \max\{p_i, p_{i+1}\}].$$

5.1.2. Độ phức tạp. Định lý 5.1. Với một cây tách–hợp, nếu mọi nút tách của nó đều có ít nhất bốn con, thì tồn tại một hoán vị p sao cho cây tách–hợp của tập đoạn liên tiếp của p chính là cây đã cho.

Định nghĩa $\text{combine}(p, q_1, q_2, \dots, q_{|p|})$ là hoán vị duy nhất thu được bằng cách cộng cùng một số vào toàn bộ mỗi q_i rồi ghép theo thứ tự, sao cho thứ tự tương đối giữa các q_i khớp với p . Ví dụ:

$$\text{combine}((1, 3, 2), (1, 2), (2, 1), (1)) = (1, 2, 5, 4, 3).$$

Định nghĩa

$$\text{flip}(p) = (|p| + 1 - p_i)_{i \in [1, |p|]}.$$

Ví dụ $\text{flip}((1, 4, 2, 3)) = (4, 1, 3, 2)$.

Với một cây tách–hợp thỏa điều kiện, có thể xây dựng p như sau:

- Nếu gốc là lá, đặt $p = (1)$.
- Ngược lại, giả sử các cây con được xây dựng thành $p_1, \dots, p_k$.
- Nếu gốc là nút hợp, đặt

$$p = \text{combine}((1, \dots, k), \text{flip}(p_1), \dots, \text{flip}(p_k)).$$

- Nếu gốc là nút tách, đặt

$$p = \text{combine}(q, \text{flip}(p_1), \dots, \text{flip}(p_k)),$$

trong đó

$$q = \begin{cases} (2, 4, \dots, k, 1, 3, \dots, k-1), & 2 \mid k, \\ (2, 4, \dots, k-1, 1, k, 3, \dots, k-2), & 2 \nmid k. \end{cases}$$

5.2. Nhiều dây chuyền hơn

5.2.1. Tính chất. Định nghĩa 5.1 (đoạn liên tiếp của nhiều hoán vị). Với dãy gồm m hoán vị $p = (p_1, \dots, p_m)$, nếu khoảng $[l, r]$ là đoạn liên tiếp của mọi p_i , thì gọi $[l, r]$ là đoạn liên tiếp của p .

Đoạn liên tiếp của k dây chuyền ($k \geq 3$) tương đương với đoạn liên tiếp của $k-1$ hoán vị.

Tập đoạn liên tiếp của k dây chuyền là chính quy và đóng loại I, II, III, nghĩa là vẫn có thể dùng cây tách-hợp để mô tả đoạn liên tiếp của nhiều hoán vị.

Một dãy khoảng cực tiểu giả của tập đoạn liên tiếp của dãy hoán vị p là dãy thu được bằng cách lấy hợp theo từng vị trí của các dãy khoảng cực tiểu giả của từng hoán vị p_i .

5.2.2. Độ phức tạp. Định lý 5.2. Với mọi cây tách-hợp, tồn tại hai hoán vị p_1, p_2 sao cho cây tách-hợp của tập đoạn liên tiếp của (p_1, p_2) chính là cây đó.

Vì mọi tập khoảng đóng loại I, II, III đều tương ứng với một cây tách-hợp duy nhất, định lý này thực chất chỉ ra rằng tập đoạn liên tiếp của hai hoán vị có thể là mọi tập khoảng đóng loại I, II, III.

Với một cây tách-hợp thỏa điều kiện, có thể xây dựng cặp hoán vị (p, q) như sau:

- Nếu gốc là lá, đặt $(p, q) = ((1), (1))$.
- Ngược lại, giả sử các cây con được xây dựng thành các cặp hoán vị $(p_1, q_1), \dots, (p_k, q_k)$.
- Nếu gốc là nút hợp, đặt

$$p = \text{combine}((1, \dots, k), \text{flip}(p_1), \dots, \text{flip}(p_k)),$$

$$q = \text{combine}((1, \dots, k), \text{flip}(q_1), \dots, \text{flip}(q_k)).$$

- Nếu gốc là nút tách, đặt

$$p = \text{combine}(p', \text{flip}(p_1), \dots, \text{flip}(p_k)),$$

$$q = \text{combine}(q', \text{flip}(q_1), \dots, \text{flip}(q_k)),$$

trong đó

$$p' = \begin{cases} (2, 3, 1), & k = 3, \\ (2, 4, \dots, k, 1, 3, \dots, k-1), & k \geq 4, 2 \mid k, \\ (2, 4, \dots, k-1, 1, k, 3, \dots, k-2), & k \geq 4, 2 \nmid k, \end{cases}$$

và

$$q' = \begin{cases} (3, 1, 2), & k = 3, \\ (2, 4, \dots, k, 1, 3, \dots, k-1), & k \geq 4, 2 \mid k, \\ (2, 4, \dots, k-1, 1, k, 3, \dots, k-2), & k \geq 4, 2 \nmid k. \end{cases}$$

Chẳng hạn, với cây tách-hợp ở ví dụ trước, có thể xây dựng hai hoán vị

$$(5, 4, 9, 8, 6, 7, 2, 1, 3), \quad (9, 8, 4, 3, 2, 1, 5, 7, 6).$$

5.3. Dây chuyền và cây

5.3.1. Tính chất. **Định nghĩa 5.2** (đoạn liên tiếp của cây). Với cây $T = (V, E)$, $V = [1, n]$, nếu khoảng $[l, r]$ thỏa đồ thị con cảm sinh của T trên tập đỉnh $[l, r]$ là đồ thị liên thông, thì gọi $[l, r]$ là đoạn liên tiếp của T .

Tổ hợp “dây chuyền, cây” tương đương với đoạn liên tiếp của cây.

Tập đoạn liên tiếp của tổ hợp “dây chuyền, cây” là chính quy và đóng loại I, II, nghĩa là có thể dùng $n - 1$ khoảng cực tiểu để mô tả đoạn liên tiếp của cây.

Một dây khoảng cực tiểu giả của tập đoạn liên tiếp của cây là

$$([\min \text{path}(i, i + 1), \max \text{path}(i, i + 1)])_{i \in [1, n-1]},$$

trong đó $\text{path}(i, i + 1)$ biểu thị tập đỉnh trên đường đi từ i đến $i + 1$ trong cây.

5.3.2. Độ phức tạp. Tập đoạn liên tiếp của cây không đóng loại III.

Xét cây $T = (V, E)$, $V = \{1, 2, 3, 4\}$, $E = \{(1, 3), (2, 3), (3, 4)\}$. Các khoảng $[1, 3]$ và $[3, 4]$ đều là đoạn liên tiếp của nó, nhưng

$$[1, 3] \setminus [3, 4] = [1, 2]$$

không phải đoạn liên tiếp của nó.

5.4. Dây chuyền và nhiều cây

5.4.1. Tính chất. **Định nghĩa 5.3** (đoạn liên tiếp của nhiều cây). Với dãy gồm m cây $T = (T_1, \dots, T_m)$, $V(T_i) = [1, n]$, nếu khoảng $[l, r]$ là đoạn liên tiếp của mọi T_i , thì gọi $[l, r]$ là đoạn liên tiếp của T .

Tổ hợp “dây chuyền, nhiều cây” tương đương với đoạn liên tiếp của nhiều cây.

Tập đoạn liên tiếp của tổ hợp này là chính quy và đóng loại I, II, nghĩa là có thể dùng $n - 1$ khoảng cực tiểu để mô tả đoạn liên tiếp của nhiều cây.

Một dây khoảng cực tiểu giả của tập đoạn liên tiếp của dãy cây T là dãy thu được bằng cách lấy hợp theo từng vị trí của các dây khoảng cực tiểu giả của từng cây T_i .

5.4.2. Độ phức tạp. Với dãy gồm $n - 1$ khoảng $A = (A_1, \dots, A_{n-1})$, $A_i = [u_i, v_i]$, nếu với mọi $i \in [1, n - 1]$ đều có $A_i \supseteq [i, i + 1]$ và

$$\forall j \in [u_i, v_i - 1], \quad A_j \subseteq A_i,$$

thì gọi A là hợp lệ.

Từ định nghĩa khoảng cực tiểu và bổ đề 4.2, dãy khoảng cực tiểu M_S của tập khoảng đóng loại I và II là hợp lệ.

Định lý 5.3. Với mọi dãy hợp lệ A gồm $n - 1$ khoảng, tồn tại hai cây T_1, T_2 sao cho dãy khoảng cực tiểu của tập đoạn liên tiếp của (T_1, T_2) chính là A .

Vì mọi tập khoảng đóng loại I và II đều tương ứng với một dãy khoảng cực tiểu, và dãy khoảng cực tiểu luôn hợp lệ, định lý này thực chất chỉ ra rằng tập đoạn liên tiếp của hai cây có thể là mọi tập khoảng đóng loại I và II.

Với một dãy khoảng cực tiểu $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$, đặt

$$V = [1, n], \quad E_1 = \{(l_i, i + 1) \mid i \in [1, n - 1]\},$$

$$E_2 = \{(r_i, i) \mid i \in [1, n - 1]\}, \quad T_1 = (V, E_1), \quad T_2 = (V, E_2).$$

Có thể kiểm tra dãy khoảng cực tiểu của tập đoạn liên tiếp của (T_1, T_2) chính là $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$.

5.5. Dây chuyền và đồ thị

5.5.1. Tính chất. **Định nghĩa 5.4** (đoạn liên tiếp của đồ thị). Với đồ thị $G = (V, E)$, $V = [1, n]$, nếu khoảng $[l, r]$ thỏa đồ thị con cảm sinh của G trên tập đỉnh $[l, r]$ là đồ thị liên thông, thì gọi $[l, r]$ là đoạn liên tiếp của G .

Tổ hợp “dây chuyền, đồ thị” tương đương với đoạn liên tiếp của đồ thị.

Tập đoạn liên tiếp của tổ hợp này là chính quy và đóng loại I.

5.5.2. Độ phức tạp. Tập đoạn liên tiếp của đồ thị không đóng loại II.

Xét đồ thị $G = (V, E)$, $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (2, 4), (4, 3), (3, 1)\}$. Các khoảng $[1, 3]$ và $[2, 4]$ đều là đoạn liên tiếp của nó, nhưng

$$[1, 3] \cap [2, 4] = [2, 3]$$

không phải đoạn liên tiếp của nó.

5.6. Hai cây

5.6.1. Tính chất. Tập đoạn liên tiếp của hai cây là chính quy và đóng loại I, II. Nhưng vì không có dây chuyền, ta không thể đánh số lại các phần tử sao cho đoạn liên tiếp trở thành khoảng.

5.6.2. Độ phức tạp. Đếm đoạn liên tiếp của hai cây là #P-complete.

Theo [1], đếm số phản dây chuyền trên một tập có thứ tự bộ phận cho trước là #P-complete. Do đó chỉ cần quy giảm trong thời gian đa thức bài toán đếm phản dây chuyền về bài toán đếm đoạn liên tiếp của hai cây, ta chứng minh được đếm đoạn liên tiếp của hai cây là #P-complete.

Giả sử tập có thứ tự bộ phận cho trước là $(S, \leq)$. Để tiện, đặt $S = \{1, 2, \dots, n\}$. Đặt

$$\{x_{i,1}, x_{i,2}, \dots, x_{i,k_i}\} = \{j \in S \mid i \leq j, i \neq j\}.$$

Xây dựng tập đỉnh

$$V = \{a\} \cup \{b_i \mid i \in S\} \cup \{c_i \mid i \in S\} \cup \{d_i \mid i \in S\} \cup \{e_{i,j} \mid i, j \in S, i \neq j, i \leq j\}.$$

Đặt

$$(h_{i,1}, h_{i,2}, \dots, h_{i,k_i+4}) = (a, b_i, d_i, e_{i,x_{i,1}}, e_{i,x_{i,2}}, \dots, e_{i,x_{i,k_i}}, c_i).$$

Xây dựng tập cạnh

$$E_1 = \bigcup_{i \in S} \{(h_{i,j}, h_{i,j+1}) \mid j \in [1, k_i + 3]\},$$
$$E_2 = \bigcup_{i \in S} \{(a, d_i), (a, c_i), (c_i, b_i)\} \cup \bigcup_{i, j \in S, i \neq j, i \leq j} \{(e_{i,j}, b_j)\}.$$

Số đoạn liên tiếp của hai cây $T_1 = (V, E_1)$, $T_2 = (V, E_2)$ trừ đi $|V|$ chính là số phản dây chuyền của $(S, \leq)$.

Nguyên nhân là mọi đoạn liên tiếp không tầm thường, tức đoạn có ít nhất hai phần tử, tương ứng một-một với phản dây chuyền.

Dạng của đoạn liên tiếp. Mọi đoạn liên tiếp không tầm thường đều có thể viết dưới dạng

$$\{a\} \cup \{b_i \mid i \in A\} \cup \{c_i \mid i \in A\} \cup \{d_i \mid i \in A\} \cup \{e_{i,j} \mid i \in A, j \in S, i \neq j, i \leq j\},$$

trong đó $\emptyset \neq A \subseteq S$.

Từ đoạn liên tiếp sang phản dây chuyền. Đoạn liên tiếp không tầm thường ở dạng trên tương ứng với phản dây chuyền

$$\{i \mid i \in A, \forall j \leq i, j \notin A\}.$$

Từ phản dây chuyền sang đoạn liên tiếp. Phản dây chuyền A tương ứng với đoạn liên tiếp không tầm thường

$$\{a\} \cup \{b_i \mid i \in B\} \cup \{c_i \mid i \in B\} \cup \{d_i \mid i \in B\} \cup \{e_{i,j} \mid i \in B, j \in S, i \neq j, i \leq j\},$$

trong đó

$$B = \{i \mid \exists j \in A, j \leq i\}.$$

Quá trình trên quy giảm bài toán đếm phản dây chuyền của tập có thứ tự bộ phận gồm n phần tử về bài toán đếm đoạn liên tiếp của hai cây có $O(n^2)$ đỉnh. Vì vậy đếm đoạn liên tiếp của hai cây là #P-complete; điều này có nghĩa là nếu tồn tại thuật toán đa thức cho bài toán này thì $P = NP$.

6. Tổng kết

Độ phức tạp hoặc độ khó của các bài toán và cấu trúc trong bài viết có thể biểu diễn xấp xỉ như sau:

hai dây chuyền đơn giản	nhiều dây chuyền	dây chuyền, cây	dây chuyền, đồ thị	hai cây khó phức tạp
đóng I, II, III	đóng I, II, III	đóng I, II	đã biết thuật toán đa thức	

Hai dây chuyền và nhiều dây chuyền được mô tả bằng cây tách-hợp; dây chuyền với cây hoặc nhiều cây được mô tả bằng dãy khoảng cực tiểu.

Bài viết chỉ ra bản chất của một lớp bài toán đoạn liên tiếp: giao của các tập con “liên thông” trên nhiều cấu trúc. Bài viết dùng dãy khoảng cực tiểu để mô tả tập khoảng S đóng loại I và II, đồng thời đưa ra thuật toán tuyến tính để, với một đầu phải r , tính tổng thông tin tại mọi đầu trái l thỏa $[l, r] \in S$. Tuy nhiên thuật toán này cần thông tin được duy trì có nghịch đảo, nên còn khá hạn chế. Do năng lực có hạn, tác giả chưa tìm được thuật toán cho thông tin nửa nhóm hoặc chứng minh rằng thuật toán như vậy không tồn tại.

Ngoài ra, bài viết cũng đưa ra điều kiện cần và đủ để tập khoảng S có cây tách-hợp, tức là S đóng loại I, II và III. Cuối cùng, bài viết dùng thuật toán trên tập khoảng để giải các bài toán đoạn liên tiếp, đồng thời chứng minh độ phức tạp hoặc độ khó của một số bài toán đoạn liên tiếp, chẳng hạn đếm số đoạn liên tiếp của hai cây là #P-complete.

Hy vọng bài viết giúp người đọc hiểu sâu hơn về đoạn liên tiếp và tiếp tục nghiên cứu thêm.

7. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yang Yaoliang, Peng Sijin và Zhang Yibin đã chỉ đạo. Xin cảm ơn cha mẹ đã nuôi dưỡng và giáo dục tác giả. Xin cảm ơn nhà trường đã bồi dưỡng, cảm ơn các thầy Fu Shuibao, Ying Ping'an, Yu Ting, Lin Naijie, Dong Yi đã giảng dạy, và cảm ơn các bạn học đã giúp đỡ. Xin cảm ơn các bạn học tại Trường Trung học Trần Hải đã đọc kiểm bản thảo. Xin cảm ơn Liu Cheng'ao đã trao đổi với tác giả và đem lại cảm hứng cho bài viết.

Tài liệu tham khảo

1. Scott Provan and Michael O. Ball. The complexity of counting cuts and of computing the probability that a graph is connected. *SIAM Journal on Computing*, 12, 1983.
2. Jan van Leeuwen and Robert E. Tarjan. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31:245–281, 1984.
3. Liu Cheng’ao. Cấu trúc dữ liệu đơn giản cho đoạn liên tiếp. Trao đổi học viên WC2019, 2019.

Bài 2. Các phương pháp chứng minh tính lỗi trong đề OI

Gou Jingyu, Trường Trung học Nam Khai, Trùng Khánh

Tóm tắt

Tính lỗi thường xuất hiện trong các bài toán Olympic Tin học. Khi một bài toán có tính lỗi, ta có thể dùng tìm kiếm tam phân, tối ưu hóa quy hoạch động bằng bất đẳng thức tứ giác, WQS, hay các kỹ thuật liên quan tới bao lỗi. Tuy nhiên, nhiều tài liệu chỉ tập trung vào cách dùng tính lỗi sau khi đã có nó, còn việc chứng minh tính lỗi lại bị bỏ qua hoặc chỉ được trình bày rất sơ lược. Bài viết này tổng kết một số cách chứng minh tính lỗi thường gặp trong đề OI: quy về mô hình thuật toán quen thuộc, chứng minh bằng đại số, và chứng minh dựa trên bản chất hình học của bài toán.

1. Lời nói đầu

Trong lời giải OI, tính lỗi thường đóng vai trò là chiếc cầu nối giữa nhận xét toán học và thuật toán hiệu quả. Một khi chứng minh được hàm mục tiêu hoặc dãy trạng thái có tính lỗi, ta có thể suy ra tính đơn điệu của điểm quyết định, tính đơn đỉnh, hoặc tính chất của sai phân, từ đó giảm độ phức tạp đáng kể.

Khó khăn là nhiều tính chất này không nằm ngay trên bề mặt đề bài. Người giải thường quan sát bằng bảng giá trị hoặc bằng trực giác thuật toán, nhưng một lời giải hoàn chỉnh vẫn cần chứng minh chặt chẽ. Vì vậy trọng tâm của bài viết là “chứng minh tính lỗi”, không phải chỉ là “sử dụng tính lỗi”.

2. Khái niệm cơ bản

Bài viết dùng cùng quy ước với một số tài liệu đội tuyển gần đây: phân biệt *lỗi trên* và *lỗi dưới*. Với hàm f trên miền thực, f là lỗi trên nếu

$$f\left(\frac{a+b}{2}\right) \geq \frac{f(a)+f(b)}{2},$$

và là lỗi dưới nếu dấu bất đẳng thức đảo chiều. Trên dãy số, a_i là lỗi trên nếu

$$2a_i \geq a_{i-1} + a_{i+1},$$

và là lỗi dưới nếu

$$2a_i \leq a_{i-1} + a_{i+1}.$$

Nói theo sai phân bậc nhất, dãy lỗi dưới có sai phân không giảm, còn dãy lỗi trên có sai phân không tăng. Nói theo sai phân bậc hai, dấu của sai phân bậc hai quyết định chiều lỗi.

Khi chứng minh trong bài toán rời rạc, cách kiểm tra bằng sai phân thường tiện hơn định nghĩa trung điểm. Với bài toán liên tục hoặc hình học, định nghĩa trung điểm lại thường tự nhiên hơn.

3. Quy về mô hình thuật toán quen thuộc

Cách chứng minh đầu tiên là nhận ra bài toán đang ẩn một mô hình đã biết có tính lời. Hai mô hình tiêu biểu là luồng chi phí và bất đẳng thức tứ giác.

3.1. Luồng chi phí

Trong mạng luồng với chi phí cạnh tuyến tính, nếu xét chi phí nhỏ nhất khi đẩy đúng k đơn vị luồng, hàm theo k là lồi dưới. Tương tự, chi phí lớn nhất theo lượng luồng là lồi trên. Có thể hiểu điều này qua đường đi tăng luồng trong đồ thị dư: các chi phí biên được chọn theo thứ tự không giảm.

Ví dụ 3.1 (House Deconstruction, dạng rút gọn). Trên một vòng tròn độ dài x có n người và m căn nhà, $n < m$. Cần gán mỗi người vào một căn nhà sao cho tổng khoảng cách trên vòng tròn là nhỏ nhất.

Khi $n = m$, có thể cắt vòng thành đường thẳng sau khi cố định số người đi qua cạnh cắt. Gọi c_i là tổng chênh lệch tiền tố giữa số người và số nhà, khi có k người đi qua cạnh cắt thì chi phí có dạng

$$h(k) = \sum_i |k + c_i|.$$

Đây là một hàm lồi dưới, nên có thể tìm cực tiểu bằng tam phân hoặc bằng sai phân.

Với $n < m$, bài toán có thể mô hình hóa bằng luồng chi phí nhỏ nhất. Nếu cố định lượng luồng k đi qua cạnh từ x về 1, chi phí tối ưu $g(k)$ là lồi dưới. Lập luận là đưa ràng buộc cận dưới/cận trên lên cạnh đó, rồi xét thay đổi một đơn vị luồng trong mạng dư. Chi phí biên của các lần tăng luồng không giảm, do đó g lồi dưới.

Bài toán còn có thể giải bằng quy hoạch động trên vị trí. Đặt $dp(i, j)$ là chi phí tối ưu sau khi quét tới vị trí i và còn chênh lệch j giữa số người và số nhà đã xử lý. Khi chuyển qua một người, một căn nhà hoặc vị trí trống, trạng thái theo j vẫn giữ tính lồi dưới. Vì vậy có thể duy trì mảng sai phân của hàm lồi thay vì tính từng giá trị độc lập, đạt độ phức tạp $O(m \log n)$ và dùng $O(n + m)$ bộ nhớ.

3.2. Bất đẳng thức tứ giác

Một hàm chi phí đoạn $w(l, r)$ thỏa bất đẳng thức tứ giác nếu với $a \leq b \leq c \leq d$ có

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c).$$

Khi w có tính chất này, nhiều quy hoạch động chia đoạn sẽ có điểm quyết định đơn điệu.

Ví dụ 3.2 (chia đoạn một tầng). Với

$$f(i) = \min_{1 \leq j \leq i} \{f(j-1) + w(j, i)\},$$

nếu w thỏa bất đẳng thức tứ giác thì điểm quyết định tối ưu của $f(i)$ không giảm theo i . Đây là cơ sở cho tối ưu hóa chia để trị hoặc tối ưu hóa bằng hàng đợi tùy dạng bài.

Ví dụ 3.3 (chia thành m đoạn). Với

$$f(i, k) = \min_{1 \leq j \leq i} \{f(j-1, k-1) + w(j, i)\},$$

nếu w thỏa bất đẳng thức tứ giác thì giá trị tối ưu theo số đoạn k là lồi dưới. Tính chất này là nền tảng cho WQS: thêm phạt tuyến tính cho mỗi đoạn, tìm số đoạn tối ưu bằng nhị phân trên hệ số phạt, rồi khôi phục đáp án mong muốn.

Ví dụ 3.4 (Bar Cover). Cho dãy độ dài n . Mỗi thẻ che một đoạn liên tiếp độ dài k , các thẻ không được chồng nhau. Cần tính tổng lớn nhất có thể che bởi đúng i thẻ, với mọi i .

Gọi đáp án theo số thẻ là $f(i)$. Có thể chứng minh $f(i)$ là lồi trên. Một cách chứng minh tổ hợp là lấy phương án tối ưu của $i - 1$ thẻ và của $i + 1$ thẻ, trộn các điểm bắt đầu đoạn rồi tách theo vị trí chắn lẻ. Hai tập thu được đều là phương án hợp lệ với i thẻ, nên tổng của chúng không vượt quá $2f(i)$. Do đó

$$f(i - 1) + f(i + 1) \leq 2f(i).$$

Cũng có thể chứng minh bằng bất đẳng thức tứ giác. Đặt

$$b_p = \sum_{t=p}^{p+k-1} a_t, \quad w(l, r) = \max_{l \leq p \leq r-k+1} b_p,$$

và coi $w(l, r) = -\infty$ nếu đoạn quá ngắn. Khi đó có thể kiểm tra

$$w(l, r - 1) + w(l + 1, r) - w(l + 1, r - 1) - w(l, r) \geq 0,$$

nên w thỏa bất đẳng thức tứ giác. Từ đó suy ra tính lồi trên của đáp án.

Về thuật toán, có thể dùng chia để trị với chập max-plus của các dãy lồi, hoặc dùng DP

$$f(p, i) = \max\{f(p - 1, i), f(p - k, i - 1) + b_{p-k+1}\}$$

kết hợp ngưỡng quyết định i_p và cây cân bằng bền vững để đạt $O(n \log^2 n)$ thời gian, $O(n)$ bộ nhớ.

4. Chứng minh bằng đại số

Khi không nhận ra mô hình thuật toán có sẵn, ta có thể quay lại định nghĩa lồi và phân tích công thức của hàm. Cách này thường dài hơn nhưng cho hiểu biết sâu hơn về cấu trúc bài toán.

4.1. Một ví dụ

Ví dụ 4.1 (High Performance Computer). Có n_A tác vụ loại A và n_B tác vụ loại B, cần phân cho p nút tính toán. Nút thứ i có tham số $t_i^A, t_i^B, k_i^A, k_i^B$: thực hiện liên tiếp x tác vụ loại A mất $t_i^A + k_i^A x^2$ nano giây, còn x tác vụ loại B mất $t_i^B + k_i^B x^2$ nano giây. Các nút chạy song song; cần tối thiểu hóa thời gian hoàn thành.

Thuật toán trực tiếp gồm hai tầng DP. Trước hết tính $f(i, a, b, c)$ là thời gian ngắn nhất để nút i xử lý a tác vụ A và b tác vụ B, với loại tác vụ cuối cùng là c . Chuyển trạng thái bằng cách liệt kê độ dài đoạn cuối:

$$\begin{aligned} f(i, a, b, 0) &= \min_{1 \leq x \leq a} \{f(i, a - x, b, 1) + t_i^A + k_i^A x^2\}, \\ f(i, a, b, 1) &= \min_{1 \leq x \leq b} \{f(i, a, b - x, 0) + t_i^B + k_i^B x^2\}. \end{aligned}$$

Sau đó gộp các nút:

$$g(i, a, b) = \min_{0 \leq x \leq a, 0 \leq y \leq b} \max\{g(i - 1, a - x, b - y), f(i, x, y)\}.$$

Phần gộp này quá đắt trên máy chấm thời điểm bài gốc, nên cần khai thác tính chất của f .

Nhận xét quan trọng là khi cố định a , hàm $f(i, a, b)$ theo b là đơn đỉnh. Khi nhị phân đáp án thời gian, tập các b thỏa $f(i, a, b) \leq ans$ là một đoạn liên tiếp. Do đó có thể tính với mỗi nút và mỗi a khoảng

$$h_{\min}(i, a) = \min\{b \mid f(i, a, b) \leq ans\}, \quad h_{\max}(i, a) = \max\{b \mid f(i, a, b) \leq ans\},$$

rồi gộp các khoảng bằng DP theo số tác vụ A. Phần này nhanh hơn, nhưng cần chứng minh tính đơn đỉnh.

Xét riêng một nút, viết gọn $f(a, b)$. Một phương án tối ưu chia các tác vụ A thành các đoạn liên tiếp độ dài $a_1, \dots, a_r$, các tác vụ B thành $b_1, \dots, b_s$, với $|r - s| \leq 1$. Nếu tồn tại hai đoạn A có độ dài chênh nhau ít nhất 2, chuyển một tác vụ từ đoạn dài sang đoạn ngắn làm thay đổi chi phí

$$k_i^A((a_{j_1} - 1)^2 + (a_{j_2} + 1)^2 - a_{j_1}^2 - a_{j_2}^2) = k_i^A(2 - 2(a_{j_1} - a_{j_2})) < 0,$$

mâu thuẫn với tối ưu. Vì vậy trong tối ưu, các đoạn cùng loại có độ dài chỉ chênh nhau nhiều nhất 1.

Nếu chia a tác vụ A thành r đoạn, viết $a = \alpha r + \beta$ với $0 \leq \beta < r$, chi phí nhỏ nhất của phần A là

$$f_A(a, r) = r t_i^A + k_i^A(r - \beta)\alpha^2 + k_i^A\beta(\alpha + 1)^2.$$

Hàm tương tự $f_B(b, s)$ cho tác vụ B. Bài toán một nút trở thành

$$f(a, b) = \min_{|r-s| \leq 1} \{f_A(a, r) + f_B(b, s)\}.$$

Các tính chất cần chứng minh là:

- (1) $f_A(a, r)$ theo a là lồi dưới và tăng khi cố định r ,
- (2) $f_A(a, r)$ theo r là lồi dưới khi cố định a ,
- (3) điểm cực tiểu theo r nằm gần cực tiểu của dạng liên tục.

Dạng liên tục thu được bằng cách cho độ dài đoạn không cần nguyên:

$$f_A^*(a, r) = r t_i^A + k_i^A r \left(\frac{a}{r}\right)^2 = r t_i^A + \frac{k_i^A a^2}{r}.$$

Dạng này gợi ý rõ ràng về lồi dưới và vị trí cực tiểu; phần còn lại là đưa nhận xét đó về trường hợp nguyên.

Khi cố định r , nếu tăng a thêm 1, một đoạn độ dài α trở thành $\alpha + 1$. Độ tăng là

$$\Delta(a) = k_i^A((\alpha + 1)^2 - \alpha^2) = k_i^A(2\alpha + 1),$$

không giảm theo a , nên $f_A(a, r)$ lồi dưới theo a .

Khi cố định a , xét các khối chia theo giá trị $\alpha = \lfloor a/r \rfloor$. Trong mỗi khối, $f_A(a, r)$ là hàm bậc nhất theo r và độ dốc bằng

$$t_i^A - k_i^A \alpha(\alpha + 1).$$

Khi r tăng thì α giảm, nên độ dốc không giảm. Cần kiểm tra thêm tại ranh giới các khối; có thể viết mỗi khối thành một đường thẳng $L_\alpha(r)$ và chứng minh

$$f_A(a, r) = \max_{1 \leq \alpha \leq a} L_\alpha(r).$$

Vì bao trên của các đường thẳng là lồi dưới, suy ra $f_A(a, r)$ lồi dưới theo r .

Từ đây, xét cặp số đoạn tối ưu (r, s) . Nếu $r < s$ thì cực tiểu riêng của A nằm bên trái r và cực tiểu riêng của B nằm bên phải s ; nếu $r > s$ thì ngược lại; nếu $r = s$ thì hai cực tiểu riêng cùng rơi vào vị trí đó. Khi cố định b và tăng a , cực tiểu phía A chỉ dịch sang phải, nên cặp tối ưu đi qua ba vùng theo thứ tự. Có thể chứng minh: nếu $f(a, b) \geq f(a + 1, b)$ thì trong phương án tối ưu của $f(a, b)$ phải có $r = a$. Trong vùng này

$$f(a, b) = f_A(a, a) + f_B(b, a + 1),$$

là tổng của một hàm tuyến tính và một hàm lồi dưới. Vì thế $f(a, b)$ theo a là ghép của một đoạn lồi dưới với một đoạn tăng, tức là đơn đỉnh. Tính đơn đỉnh dùng trong thuật toán nhị phân thời gian đã được chứng minh.

4.2. Một ví dụ khác

Ví dụ 4.2 (Dedenne). Một từ là xâu nhị phân không chứa hai số 0 liên tiếp. Từ điển là một tập từ không từ nào là tiền tố của từ khác. Nếu một xâu nhị phân là tiền tố của k từ trong từ điển thì chi phí của nó là

$$w(k) = \sum_{j=1}^k (1 + \lfloor \log_2 j \rfloor).$$

Chi phí của từ điển là tổng chi phí của mọi xâu nhị phân, kể cả xâu rỗng. Cần tối thiểu hóa chi phí của từ điển có n từ, với n rất lớn.

Mỗi xâu đóng góp chi phí tương ứng với một nút trong trie của từ điển. Gọi $f(n)$ là đáp án nhỏ nhất. Nếu cây con trái chứa k lá và cây con phải chứa $n - k$ lá, ta có

$$f(1) = w(1) = 1,$$

$$f(n) = w(n) + \min_{1 \leq k \leq n-1} \{w(k)[k > 1] + f(k) + f(n - k)\}.$$

DP trực tiếp mất $O(n^2)$.

Trước hết,

$$w(k) - w(k - 1) = 1 + \lfloor \log_2 k \rfloor,$$

nên w là lồi dưới. Bằng quy nạp đồng thời, có thể chứng minh $f(n)$ cũng tăng và lồi dưới, đồng thời điểm quyết định tối ưu k_n thỏa

$$0 \leq k_n - k_{n-1} \leq 1.$$

Ý tưởng là giả sử các kết luận đúng tới n_0 . Khi xét $n_0 + 1$, nếu chọn $k < k_{n_0}$ thì so với chọn k_{n_0} , phần chênh lệch ở trạng thái cũ không âm, còn chênh lệch sai phân của f cũng không âm. Vì vậy k không thể tốt hơn. Tương tự, mọi $k > k_{n_0} + 1$ cũng không tối ưu. Sau khi có quyết định đơn điệu, so sánh công thức sai phân của $f(n_0 + 1)$ và $f(n_0)$ sẽ cho $df(n_0 + 1) \geq df(n_0)$.

Nhờ đó DP có thể tối ưu xuống $O(n)$. Nhưng với n tới 10^{15} , vẫn không thể tính mọi trạng thái. Cần dùng thêm phân tích tiệm cận. Từ định nghĩa,

$$w(n) = \Theta(n \log n).$$

Nếu trong chuyển trạng thái luôn chọn gần $n/2$, ta được một chặn trên $O(n \log^2 n)$ cho $f(n)$. Nếu bỏ hạng $w(k)$ trong chuyển trạng thái để tạo một bài toán dễ hơn, lời giải tối ưu vẫn tách gần $n/2$, cho chặn dưới $\Omega(n \log^2 n)$. Do đó

$$f(n) = \Theta(n \log^2 n).$$

Vì f lồi dưới, sai phân của nó thỏa

$$df(n) = \Theta(\log^2 n).$$

Sai phân $df(n)$ vừa tăng vừa chỉ có $O(\log^2 n)$ giá trị khác nhau. Thuật toán cuối cùng duyệt các giá trị sai phân δ , tìm vị trí lớn nhất có $df(n) = \delta$ bằng tăng gấp bội và kiểm tra bằng thủ tục gần tam phân trên hàm lồi. Khi chọn hệ số gấp bội phù hợp, việc kiểm tra chỉ cần tra các đoạn sai phân đã biết. Độ phức tạp tổng thể là $O(\log^5 n)$. Trong thực tế, với giới hạn đề bài, có thể tiền xử lý các đầu đoạn rồi đưa bảng vào chương trình nộp.

5. Chứng minh tính lồi trong hình học tính toán

Trong hình học tính toán, tính lồi thường không đến từ công thức DP mà đến từ bản thân đối tượng hình học: đa giác lồi, bao lồi, đường tròn, hoặc giao của các hình lồi. Khi bài toán xoay quanh những đối tượng đó, nên thử chứng minh bằng lập luận hình học trước khi đi vào công thức từng đoạn.

5.1. Một ví dụ

Ví dụ 5.1 (Easy Geometry). Cho một đa giác lồi n đỉnh trên mặt phẳng. Cần dựng trong đa giác một hình chữ nhật diện tích lớn nhất, các cạnh song song với trục tọa độ.

Thuật toán tự nhiên là tam phân lồng nhau: tam phân theo chiều rộng w của hình chữ nhật, rồi với w cố định tam phân theo tọa độ trái l . Sau khi biết l và $r = l + w$, có thể tìm biên trên và biên dưới bằng tìm kiếm trên hai đường bao của đa giác. Tính đúng đắn của hai lần tam phân phụ thuộc vào việc các hàm tương ứng là đơn đỉnh; thực ra chúng là lồi trên.

Với w cố định, gọi $h_w(l)$ là chiều cao lớn nhất của hình chữ nhật có biên trái l , và $s_w(l) = wh_w(l)$ là diện tích tương ứng. Hình học của $h_w(l)$ rất rõ: lấy giao của đa giác ban đầu với chính nó sau khi tịnh tiến sang trái w ; độ dài đoạn giao của đường thẳng $x = l$ với vùng giao này chính là $h_w(l)$. Giao của hai tập lồi vẫn lồi, nên độ dài lát cắt theo phương thẳng đứng là một hàm lồi trên theo l . Vì $s_w(l)$ chỉ nhân thêm hằng số w , nó cũng lồi trên. Do đó tam phân bên trong là hợp lệ.

Gọi

$$H(w) = \max_l h_w(l), \quad S(w) = wH(w).$$

Khi w tăng, vùng giao nói trên co lại, nên $H(w)$ không tăng.

Để chứng minh H và S lồi trên theo w , đặt $u(x)$ là biên trên của đa giác và $d(x)$ là biên dưới. Với đa giác lồi, u là lồi trên còn d là lồi dưới. Ta có

$$h_w(l) = \min\{u(l), u(l+w)\} - \max\{d(l), d(l+w)\},$$

hay tương đương là min của bốn hàm:

$$\begin{aligned} &u(l) - d(l), \\ &u(l) - d(l+w), \\ &u(l+w) - d(l), \\ &u(l+w) - d(l+w). \end{aligned}$$

Mỗi hàm trong bốn hàm này là lồi trên theo l .

Lấy $0 < a < b$, đặt $c = (a+b)/2$. Giả sử $H(a)$ đạt tại l_a và $H(b)$ đạt tại l_b , đặt $l_c = (l_a + l_b)/2$. Dùng tính lồi trên của u và lồi dưới của d cho từng trường hợp trong bốn biểu thức trên, ta có

$$2h_c(l_c) \geq h_a(l_a) + h_b(l_b).$$

Vì $H(c) \geq h_c(l_c)$, suy ra

$$2H(c) \geq H(a) + H(b),$$

tức H lồi trên.

Do $H(w)$ không tăng,

$$\begin{aligned} 2S(c) &= c \cdot 2H(c) \\ &\geq c(H(a) + H(b)) \\ &= \frac{a+b}{2}(H(a) + H(b)) \\ &= aH(a) + bH(b) + \frac{b-a}{2}(H(a) - H(b)) \\ &\geq aH(a) + bH(b) = S(a) + S(b). \end{aligned}$$

Vậy $S(w)$ cũng lồi trên, và tam phân ngoài là hợp lệ.

6. Tổng kết

Tính lỗi trong đề OI về bản chất vẫn là tính lỗi trong toán học. Khác biệt là trong bài toán toán học, chứng minh xong thường là kết thúc; còn trong OI, chứng minh chỉ là cơ sở để thiết kế và tối ưu thuật toán.

Bài viết trình bày ba hướng chứng minh: quy về mô hình quen thuộc, phân tích đại số trực tiếp, và khai thác hình học của đối tượng lỗi. Ba hướng này không bao quát mọi khả năng, nhưng là những cách xuất hiện nhiều trong quá trình giải bài. Khi gặp một tính chất lỗi mới, nên vừa quan sát dữ liệu để đoán dạng đúng, vừa tìm một chứng minh thật sự bằng một trong các con đường trên.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi, cảm ơn các huấn luyện viên đội tuyển quốc gia đã chỉ đạo. Xin cảm ơn gia đình đã đồng hành và ủng hộ tác giả. Xin cảm ơn thầy Dong Youming, thầy Li Xiang, Du Yuhao và Jiang Lingyu đã giúp đỡ trong quá trình viết bài. Xin cảm ơn Trường Trung học Nam Khai, Trùng Khánh và Trường THCS Nam Khai Rongqiao, Trùng Khánh đã bồi dưỡng tác giả, cùng các thầy cô và bạn học khác đã giúp đỡ.

Bài 3. Một lời giải dựa trên phân rã chuỗi dài cho bài toán lân cận trên cây

Liu Haifeng, Trường Trung học Tưởng niệm Trung Sơn

Tóm tắt

Bài viết xuất phát từ phân rã chuỗi dài, trình bày cách xử lý bốn dạng bài toán lân cận thường gặp trên cây tĩnh. Các cách xử lý bao gồm cả trường hợp offline và trường hợp bắt buộc online. Nhờ phân rã chuỗi dài, một phần bài toán lân cận được quy về truy vấn đoạn trên dãy và truy vấn đường trên cây trong thời gian tuyến tính.

1. Lời nói đầu

Với bài toán lân cận trên cây tĩnh, các công cụ thường gặp là phân trị điểm và static top tree. Phân rã chuỗi dài cũng là một kỹ thuật quen thuộc, thường dùng để tối ưu các bài toán liên quan đến độ sâu. Vì cấu trúc của lân cận cũng gắn chặt với độ sâu, ta có thể thử dùng phân rã chuỗi dài để xử lý các loại lân cận khác nhau trên cây.

2. Kiến thức chuẩn bị

Bài viết chỉ cần các kiến thức cơ bản về cây.

3. Quy ước và định nghĩa

Nếu không nói rõ, cây trong bài là cây có gốc gồm n đỉnh, gốc là đỉnh 1, mọi cạnh có độ dài 1, và không có thao tác sửa đổi.

Để tiện trình bày, “nửa nhóm” dưới đây mặc định là nửa nhóm giao hoán, còn “nhóm” mặc định là nhóm giao hoán, trừ khi có ghi chú riêng.

Định nghĩa 3.1 (độ sâu). Độ sâu của một đỉnh là khoảng cách ngắn nhất từ đỉnh đó đến gốc cộng 1. Ký hiệu f_{a_x} là cha của đỉnh x .

3.1. Định nghĩa cơ bản của phân rã chuỗi dài

Định nghĩa 3.2 (chiều cao). Chiều cao của một đỉnh là khoảng cách từ đỉnh đó đến đỉnh xa nhất trong cây con của nó, cộng 1. Lá có chiều cao 1; chiều cao của x được ký hiệu là h_x .

Định nghĩa 3.3 (con dài). Con dài của một đỉnh là con có chiều cao lớn nhất. Nếu có nhiều con cùng đạt cực đại thì chọn tùy ý; lá không có con dài.

Định nghĩa 3.4 (con ngắn). Các con không phải con dài được gọi là con ngắn.

Định nghĩa 3.5 (chuỗi dài). Giữ lại, với mỗi đỉnh không phải lá, chỉ cạnh nối tới con dài của nó. Các thành phần đường thu được gọi là chuỗi dài; một chuỗi có thể chỉ gồm một đỉnh. Độ dài của chuỗi là số đỉnh trong chuỗi.

Định lý 3.1 (định lý cơ bản của phân rã chuỗi dài). Gọi $f(x)$ là tổng chiều cao của các con ngắn của x . Khi đó

$$\sum_{i=1}^n f(i) = n - \text{chiều cao của gốc}.$$

Chứng minh. Tách đóng góp của tổng trên: mọi chuỗi dài trừ chuỗi chứa gốc được tính đúng một lần, còn chuỗi chứa gốc không được tính. Vì vậy tổng đúng bằng số đỉnh không nằm trên chuỗi dài của gốc, tức $n -$ chiều cao của gốc.

3.2. Các định nghĩa liên quan tới lân cận

Tên của một số khái niệm trong phần này do tác giả đặt để tiện trình bày.

Định nghĩa 3.6 (lân cận). Lân cận biểu diễn bởi cặp (x, d) là tập các đỉnh có khoảng cách tới x không quá d .

Định nghĩa 3.7 (lân cận trong). Lân cận trong biểu diễn bởi (x, d) là tập các đỉnh nằm trong cây con của x và có khoảng cách tới x không quá d . Một số tài liệu cũng gọi đây là lân cận có hướng.

Định nghĩa 3.8 (lân cận ngoài). Lân cận ngoài biểu diễn bởi (x, d) là tập các đỉnh không nằm trong cây con của x và có khoảng cách tới x không quá d .

Định nghĩa 3.9 (lân cận chuỗi). Với bộ ba (x, y, d) , trong đó y là tổ tiên của x , lân cận chuỗi là tập các đỉnh thỏa:

1. nằm ngoài cây con của x nhưng nằm trong cây con của y ;
2. tồn tại một đỉnh trên đường từ x đến y có khoảng cách tới đỉnh đang xét không quá d .

Hai phần tiếp theo xử lý truy vấn thông tin nhóm và nửa nhóm trên các loại lân cận trong trường hợp offline.

4. Truy vấn thông tin nhóm

Trong phần này, để đơn giản, ta xét bài toán tính tổng trọng số đỉnh trên các loại lân cận. Vì thông tin thuộc nhóm, ta được phép dùng phép trừ hoặc phần tử nghịch đảo. Mục tiêu là xử lý offline mọi truy vấn trong thời gian tuyến tính.

4.1. Lân cận trong

Với một chuỗi dài có độ dài m và đỉnh đầu chuỗi là x , trước hết tiền xử lý tổng trọng số các đỉnh trong cây con của x có khoảng cách tới x bằng i . Theo định lý cơ bản của phân rã chuỗi dài, phần này có thể làm trực tiếp với tổng thời gian tuyến tính.

Sau đó xét từng chuỗi dài. Với mỗi đỉnh x , tiền xử lý một mảng có độ dài bằng chiều cao lớn nhất trong các con ngắn của x ; phần tử thứ i là tổng trọng số các đỉnh nằm trong cây con của các con ngắn và cách x đúng i . Như vậy toàn bộ cây có thể xem như nhiều chuỗi dài, mỗi đỉnh trên chuỗi còn có thể treo thêm một chuỗi phụ chứa thông tin của các con ngắn; tổng kích thước các phần phụ vẫn là $O(n)$.

Với mỗi chuỗi, quét từ đáy lên đỉnh. Khi đang ở vị trí i , các thay đổi chỉ ảnh hưởng tới vị trí $i - 1$ và một đoạn phía sau; tổng số thay đổi trên mọi chuỗi là $O(n)$. Treo truy vấn vào các điểm tương ứng trên chuỗi thì bài toán trở thành nhiều truy vấn tổng đoạn. Do có thể duy trì tổng hậu tố, ta đạt thời gian tuyến tính.

Về bản chất, cấu trúc cần truy vấn giống một lưới: cho điểm (x, y) , cần tổng các điểm nằm ở vùng bên trái phía dưới nó. Một cách nhìn khác là tìm nhanh vị trí có giá trị đầu tiên (x', y) nằm dưới (x, y) , rồi dùng đáp án đã tiền xử lý tại vị trí đó cộng với phần các hàng $[x, x')$. Với truy vấn online bắt buộc, việc tìm vị trí này sẽ được chuyển thành một bài toán riêng ở phần 7.

4.2. Lân cận

Lân cận ngoài có thể tính bằng lân cận trừ lân cận trong, nên trước hết chỉ cần xét lân cận thông thường. Với lân cận (x, d) , nếu thay nó bằng $(fa_x, d - 1)$ thì trạng thái của các đỉnh ngoài cây con x không đổi. Nếu $d \geq h_x + 1$, cả hai lân cận đều chứa toàn bộ cây con x , nên phép thay không làm đổi đáp án. Ta có thể liên tục nhảy lên cha cho tới khi không thể nhảy nữa.

Định nghĩa 4.1 (thao tác nhảy chuỗi). Với (x, d) , tìm tổ tiên gần nhất y của x sao cho độ dài chuỗi dài chứa y nhân 2 không nhỏ hơn d . Nhảy tới y và giảm d theo chênh lệch độ sâu. Cặp mới (y, d') thỏa

$$d' \leq 2 \cdot \text{độ dài chuỗi dài chứa } y.$$

Nếu thay thao tác nhảy chuỗi bằng việc nhảy từng bước từ (x, d) sang $(fa_x, d - 1)$, thì trước khi tới y luôn có $d \geq h_x + 1$. Lý do là với mỗi đỉnh z trên đường đi từ x tới y , trừ y , độ dài chuỗi dài chứa z vẫn đủ lớn so với phần d đã giảm, nên điều kiện bao trọn cây con vẫn đúng.

Bài toán phụ còn lại là: cho x, k , tìm tổ tiên gần nhất y sao cho chuỗi dài chứa y có độ dài ít nhất k . Trong offline, có thể DFS và mở n ngăn xếp; ngăn xếp thứ i lưu chuỗi cần nhảy tới khi $d = i$. Khi DFS vào một con ngắn có chuỗi dài số k , độ dài m , đẩy k vào các ngăn xếp $1, \dots, m$, và khi rời khỏi cây con thì pop lại. Nhờ đó ta biết truy vấn sau khi nhảy sẽ rơi vào chuỗi nào; duy trì thêm vài thông tin đơn giản trong DFS sẽ xác định được cặp (y, d') .

Với đỉnh đầu x của chuỗi dài độ dài m , tiền xử lý các lân cận ngoài $(x, 1), (x, 2), \dots, (x, 2m)$. Nếu một truy vấn ngoài tại chuỗi này có $d \leq 2m$, ta tách đóng góp thành phần ngoài cây con của đỉnh đầu chuỗi và phần trong cây con đỉnh đầu chuỗi; phần thứ nhất đã tiền xử lý, phần thứ hai xử lý như lân cận trong.

Để tiện xử lý lân cận ngoài của đỉnh đầu chuỗi x , gọi $y = fa_x$. Lân cận ngoài (x, d) có thể viết thành lân cận ngoài $(y, d - 1)$ cộng với lân cận chuỗi $(x, y, d - 1)$. Vì đang ở trường hợp nhóm, các phần chồng nhau có thể xử lý bằng bao hàm–loại trừ. Toàn bộ tiền xử lý là $O(n)$, và Q truy vấn offline được xử lý trong $O(n + Q)$.

4.3. Lân cận chuỗi

Cách làm tương tự lân cận thường. Đặt $A(x, d)$ là tổng của lân cận chuỗi $(x, 1, d)$ cộng với lân cận trong (x, d) . Nếu d lớn hơn độ dài chuỗi dài chứa x , thì

$$A(x, d) = A(fa_x, d).$$

Vì vậy ta lại dùng một thao tác nhảy chuỗi, nhưng lần này cần tổ tiên gần nhất y sao cho d không vượt quá độ dài chuỗi dài chứa y .

Với mỗi chuỗi dài có đỉnh đầu x và độ dài m , tiền xử lý

$$(x, 1, 1), (x, 1, 2), \dots, (x, 1, m).$$

Ý tưởng là chuyển $(x, 1, i)$ thành lân cận chuỗi $(fa_x, 1, i)$, cộng lân cận trong (fa_x, i) , rồi trừ lân cận trong $(x, i - 1)$.

Nếu một truy vấn $(x, 1, d)$ thỏa d không vượt quá độ dài chuỗi chứa x , gọi y là đỉnh đầu chuỗi của x . Ta tách truy vấn thành (x, y, d) và $(y, 1, d)$. Phần sau đã được tiền xử lý, còn phần trước là truy vấn đóng góp trên một đoạn của chuỗi, có thể xử lý như cấu trúc lưới ở phần lân cận trong. Cuối cùng lấy $A(x, d)$ trừ lân cận trong (x, d) để thu được lân cận chuỗi cần tìm.

5. Truy vấn thông tin nửa nhóm

Phần này nhằm quy các truy vấn nửa nhóm giao hoán về những bài toán chuẩn hơn trong $O(n + Q)$: lân cận trong và ngoài được quy về truy vấn nửa nhóm trên đoạn của dãy, còn lân cận chuỗi được quy về truy vấn nửa nhóm trên đường của cây. Vì không có nghịch đảo, không thể dùng phép trừ như phần trước.

5.1. Lân cận trong

Quay lại mô hình lưới của lân cận trong. Với truy vấn (x, y) , tìm vị trí đầu tiên (x', y) nằm bên dưới cùng cột và có giá trị, rồi dùng đáp án tiền xử lý tại đó kết hợp với thông tin của các hàng $[x, x')$. Việc tiền xử lý đáp án tại các vị trí có giá trị không cần dùng phép trừ. Trong offline, tìm x' bằng cách quét từ dưới lên và dùng thùng cho từng cột y để ghi vị trí nhỏ nhất đã gặp.

5.2. Lân cận

Ta vẫn thực hiện thao tác nhảy chuỗi như trường hợp nhóm. Sau khi nhảy xong, truy vấn tương đương với ghép một lân cận ngoài và một lân cận trong. Điểm khác biệt là mọi phần phải được tách thành các đoạn không giao nhau hoặc các đường cây độc lập, thay vì trừ phần thừa.

5.3. Lân cận ngoài

Trong tiền xử lý của trường hợp nhóm, phần dùng phép trừ là truy vấn thông tin của fa_x sau khi bỏ cây con x . Không cần nghịch đảo, ta có thể xử lý bằng cách tách các con. Gọi y là con dài của fa_x . Tiền xử

lý lân cận trong của y với các bán kính cần thiết, rồi chỉ còn phải ghép đóng góp của các con ngắn khác của fa_x . Các truy vấn đối với các con ngắn có tổng quy mô tuyến tính nên có thể làm trực tiếp.

Khi truy vấn ngoài (x, d) quá lớn so với chuỗi hiện tại, ta nhảy tới một chuỗi i . Tìm tổ tiên gần nhất y sao cho độ dài chuỗi chứa y nhân 2 đủ lớn, đặt $d' = d - \text{dist}(x, y)$. Ta cần ghép:

- lân cận ngoài (y, d') , đã được tiền xử lý;
- phần trong cây con của y nhưng không thuộc nhánh đi xuống x .

Nếu gọi z là con của y nằm trên đường tới x , phần thứ hai chính là lân cận chuỗi (z, y, d') .

Có một cách cài đặt đơn giản hơn: với mỗi chuỗi dài, đánh số các con ngắn theo độ sâu, con nông hơn có số nhỏ hơn. Với truy vấn (x, d) , xác định y, z, d' và số thứ tự của nhánh z . Sau đó ghép đóng góp của các con ngắn có số lớn hơn z , các con ngắn có số nhỏ hơn z , và phần ngoài cây con của đỉnh đầu chuỗi. Mỗi nhóm lại quy về kiểu truy vấn ở phần 5.1. Nếu từ a nhảy tới b , còn cần cộng thông tin của cây con b sau khi bỏ cây con a ; trên thứ tự DFS, phần này là hợp của hai đoạn. Do đó lân cận trong và ngoài offline được quy về truy vấn đoạn trên dãy trong $O(n + Q)$.

5.4. Lân cận chuỗi

Nếu chuỗi kết thúc tại gốc, cách làm về bản chất giống lân cận ngoài. Trước hết xét trường hợp x, y nằm trên cùng một chuỗi dài. Truy vấn có dạng lấy một dải hàng $[l, r]$ và d cột đầu trong mô hình lưới. Vì không có phép trừ, ta tìm vị trí có giá trị đầu tiên ở cột d phía dưới hàng l , và vị trí có giá trị đầu tiên phía trên hàng r . Với mỗi cột, duy trì thông tin nửa nhóm của các khoảng giữa những vị trí có giá trị; tổng số vị trí có giá trị là $O(n)$.

Đặt g_x là chiều cao lớn nhất trong các con ngắn của x , và $g_x = 0$ nếu không có con ngắn. Xây một rừng sinh có n lớp: với mỗi đỉnh x của cây gốc, tạo các bản sao $(x, 1), \dots, (x, g_x)$. Tổng số bản sao không vượt quá n . Ở lớp i , nối (x, i) tới (y, i) , trong đó y là tổ tiên gần nhất của x thỏa $g_y \geq i$. Trọng số của cạnh này chính là thông tin nửa nhóm của lân cận chuỗi tương ứng.

Sau khi biết thông tin trên các cạnh của rừng sinh, một truy vấn lân cận chuỗi có thể được tách thành phần đã nhảy qua và phần còn lại trong cùng chuỗi. Nếu hai đầu không cùng chuỗi, tìm con ngắn đầu tiên k trên đường từ y xuống x , dùng truy vấn đường trên rừng sinh để lấy (x, fa_k, d) , rồi phần (fa_k, y, d) nằm trên cùng chuỗi và xử lý bằng cách ở trên.

Cách tính thông tin cho mọi cạnh của rừng sinh cũng tuyến tính. Nếu ở lớp i , x nối tới y , thì các đỉnh nằm giữa x và y có giá trị $g \leq i$. Bài toán quy về một cấu trúc dữ liệu hỗ trợ:

- thêm thông tin nửa nhóm cho một tiền tố, đồng thời thêm một thông tin cố định cho phần sau tiền tố;
- truy vấn thông tin của một tiền tố rồi xóa phần đã truy vấn.

Vì tổng lượng sửa đổi là $O(n)$, chỉ cần dùng lazy tag và đẩy tag dần về sau khi thực hiện thao tác tiền tố; tổng thời gian vẫn tuyến tính.

6. Các trường hợp khác

Một số bài toán có thông tin không thể tách thành từng lớp khoảng cách đúng bằng i ; ta chỉ duy trì được thông tin của tập các đỉnh trong cây con x có khoảng cách tới x không quá i . Nếu phép chuyển trạng thái có thể ghép nhanh và có tính kết hợp, phân rã chuỗi dài vẫn xử lý được.

Ví dụ 6.1. Cho cây có trọng số đỉnh. Mỗi truy vấn cho x, d , cần tìm trọng số lớn nhất của tập độc lập trong lân cận trong (x, d) .

Với mỗi chuỗi dài, tính thông tin cho lân cận có bán kính i quanh đỉnh đầu chuỗi. Với mỗi đỉnh x , trước hết tính phần cây con của x sau khi bỏ con dài, rồi dùng DP của con dài để chuyển. Khi quét một chuỗi, sửa đổi tại vị trí cách đỉnh đầu chuỗi i tương đương với cập nhật trực tiếp một đoạn tiền tố ngắn, rồi nhân cùng một ma trận cho phần còn lại. Giống phần 5.4, lazy tag cho phép đẩy các phép nhân này về sau trong tổng thời gian tuyến tính.

Khi truy vấn một vị trí, nếu phía trước còn lazy tag chưa đẩy, giá trị đọc được chưa phải đáp án cuối cùng; cần ghép thêm đóng góp của các hàng trước nó. Nhìn từ góc khác, mỗi lazy tag chính là thông tin của phần cây con sau khi bỏ con dài, nên bản chất vẫn giống cách chuyển DP trên chuỗi dài.

Các loại lân cận còn lại cũng xử lý tương tự, chỉ là chi tiết cài đặt rườm rà hơn.

7. Bất buộc online

Phần này giả sử đã có cấu trúc LCA online $O(n)$ -tiền xử lý, $O(1)$ -truy vấn và cấu trúc tổ tiên cấp k online với cùng độ phức tạp. Mục tiêu là chuyển các bài toán lân cận online về truy vấn nửa nhóm online trên đoạn hoặc trên đường cây.

7.1. Bài toán tiền nhiệm trên dãy

Xét bài toán phụ: cho dãy số nguyên dương $a_1, \dots, a_n$, mỗi truy vấn cho i, x , cần tìm chỉ số lớn nhất $j \leq i$ sao cho $a_j \geq x$, hoặc báo không tồn tại. Gọi $S = \sum_i a_i$; yêu cầu $O(n + S)$ tiền xử lý và $O(1)$ mỗi truy vấn online.

Nếu offline, chỉ cần quét chỉ số và dùng thùng lưu vị trí cuối cùng đã gặp cho mỗi độ cao. Với online, xây đồ thị có S đỉnh: với mỗi i , tạo $(i, 1), (i, 2), \dots, (i, a_i)$. Với mỗi đỉnh (x, y) , tìm $z \leq x$ lớn nhất sao cho $(z, y + 1)$ tồn tại, rồi nối (x, y) tới $(z, y + 1)$ nếu có. Đồ thị thu được là một rừng hướng vào trong. Truy vấn (i, x) trở thành tìm tổ tiên cấp $x - 1$ của $(i, 1)$, nên có thể trả lời trong $O(1)$.

7.2. Bài toán nhảy chuỗi

Trong lân cận ngoài và lân cận chuỗi, ta thường cần: với đỉnh x , tìm tổ tiên gần nhất y sao cho chuỗi dài chứa y có độ dài ít nhất k . Cách offline đã nêu ở phần 4.2; với online, co mỗi chuỗi dài thành một đỉnh. Chuỗi a nối lên chuỗi b nếu cha của đỉnh đầu chuỗi a nằm trên chuỗi b . Khi đó bài toán trở thành tiền nhiệm trên cây.

Cách dựng tương tự bài toán tiền nhiệm trên dãy: với chuỗi x có độ dài a_x , tạo $(x, 1), \dots, (x, a_x)$. Với (x, y) , tìm tổ tiên gần nhất z của chuỗi x sao cho $(z, y + 1)$ tồn tại, rồi nối (x, y) tới $(z, y + 1)$. Truy vấn tại chuỗi i chỉ cần tìm tổ tiên cấp $k - 1$ của $(i, 1)$. Kết quả cho biết truy vấn dừng ở chuỗi nào; để biết chính xác đỉnh dừng, lấy LCA của x với đáy của chuỗi đó.

7.3. Cách xử lý các lân cận

Ta vẫn muốn dùng $O(n)$ tiền xử lý và $O(1)$ để chuyển truy vấn về truy vấn nửa nhóm online trên đoạn hoặc trên đường cây. Với lân cận trong, nhiệm vụ là: cho (x, y) , tìm vị trí đầu tiên nằm dưới cùng cột và có giá trị. Bài toán này cũng chuyển thành tổ tiên cấp k trên cây. Với mỗi vị trí có giá trị (x, y) , tìm i nhỏ nhất sao cho $i \geq x$ và $(i, y + 1)$ có giá trị, rồi nối (x, y) tới $(i, y + 1)$. Khi truy vấn, bắt đầu từ (x, x) và lấy tổ tiên cấp $y - x$.

Các phần còn lại của lân cận trong đều được giải bằng truy vấn nửa nhóm trên dãy hoặc trên cây. Với lân cận ngoài và lân cận chuỗi, trước hết xử lý thao tác nhảy chuỗi online như phần 7.2; sau đó các mảnh còn lại được tách và truy vấn bằng cùng các cấu trúc nửa nhóm.

8. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn Trường Trung học Tưởng niệm Trung Sơn và Trường Sanxin đã cung cấp môi trường học tập tin học tốt. Xin cảm ơn gia đình, bạn bè và huấn luyện viên đã ủng hộ, động viên tác giả. Xin cảm ơn Zhao Haikun, Xu Qiwen và Pang Jiayi đã góp ý quý báu cho bài viết.

Tài liệu tham khảo

1. Liu Rujia. *Nghệ thuật thuật toán và thi đấu tin học*.

Bài 4. Ứng dụng nguyên lý chuyển vị trong một lớp bài toán quy hoạch động

Xu Xiaoyang, Trường Trung học số 1 trực thuộc Đại học Sư phạm Hoa Trung

Tóm tắt

Nguyên lý chuyển vị (*Transposition Principle*) là tư tưởng dùng ma trận chuyển vị để viết lại thuật toán của một biến đổi tuyến tính. Nó có thể tối ưu hiệu quả độ phức tạp thời gian và bộ nhớ của một số bài toán biến đổi tuyến tính. Bài viết đưa nguyên lý này vào quy hoạch động, kết hợp với một số thao tác trên cây đoạn, rồi thông qua hai ví dụ trình bày lợi thế của cách nhìn chuyển vị khi giải các bài toán DP cần tối ưu bằng cây đoạn.

1. Mở đầu

Từ thế kỷ trước, nguyên lý chuyển vị đã được dùng trong tối ưu thuật toán biến đổi tuyến tính. Nó liên hệ chặt chẽ một thuật toán biến đổi tuyến tính với thuật toán của biến đổi chuyển vị. Trước đó, người ta thường tối ưu một biến đổi và chuyển vị của nó một cách riêng rẽ, ví dụ nhân đa thức và bài toán chuyển vị tương ứng. Nguyên lý chuyển vị cho biết hai bài toán có thể được giải trong gần như cùng độ phức tạp thời gian và bộ nhớ. Vì vậy, khi tối ưu được một thuật toán biến đổi tuyến tính, ta cũng có thể thu được thuật toán cho chuyển vị của nó.

Với bài toán đơn giản, thường có thể tối ưu trực tiếp. Giá trị của nguyên lý chuyển vị thể hiện rõ hơn ở những bài toán khó, nhất là khi thuật toán có bản chất tuyến tính nhưng dạng trực tiếp không dễ nhìn ra. Hiện tại, các ứng dụng trong OI chủ yếu liên quan đến hàm sinh và phép toán đa thức. Bài viết này bàn về một hướng ít được nhắc tới hơn: dùng chuyển vị trong DP tối ưu bằng hợp nhất hoặc tách cây đoạn.

2. Nguyên lý chuyển vị

2.1. Giới thiệu

Định nghĩa 2.1 (thuật toán biến đổi tuyến tính). Với ma trận A kích thước $m \times n$, một thuật toán có thể tính

$$b = Aa$$

với mọi vector cột $a \in R^n$ được gọi là thuật toán biến đổi tuyến tính ứng với A .

Nhân ma trận trực tiếp là một ví dụ hiển nhiên, nhưng thuật toán biến đổi tuyến tính không nhất thiết phải cài bằng nhân ma trận thô. Ví dụ, thuật toán tính tổng hậu tố có thể xem là nhân với ma trận tam giác trên toàn số 1. Cách cài đặt của thuật toán chỉ phụ thuộc vào ma trận A , không phụ thuộc giá trị cụ thể của vector đầu vào.

Định nghĩa 2.2 (chuyển vị của thuật toán). Với thuật toán f ứng với ma trận A , thuật toán g ứng với ma trận A^T được gọi là chuyển vị của f . Nghĩa là với mọi vector cột $a' \in R^m$, thuật toán g tính

$$b' = A^T a'.$$

Vector a' ở đây là một đầu vào độc lập, không có quan hệ số trị với vector a của thuật toán gốc.

Vẫn với ví dụ tổng hậu tố, ma trận chuyển vị là ma trận tam giác dưới toàn số 1, tức thuật toán tổng tiền tố. Do đó có thể xem tổng tiền tố là chuyển vị của tổng hậu tố.

Theo nguyên lý chuyển vị, có thể viết lại một thuật toán biến đổi tuyến tính thành thuật toán cho biến đổi chuyển vị mà không đổi bậc độ phức tạp. Nếu bài toán chuyển vị dễ tối ưu hơn, ta có thể tối ưu nó rồi chuyển ngược để giải bài toán ban đầu.

2.2. Viết lại thuật toán

Để tiện xử lý, có thể thêm hàng và cột để biến A thành một ma trận vuông khả nghịch:

$$\begin{pmatrix} A & I_m \\ I_n & 0 \end{pmatrix}.$$

Đồng thời thêm m phần tử vào vector đầu vào và đặt chúng bằng 0; phần mở rộng không ảnh hưởng đến m phần tử đầu của kết quả, còn n phần tử sau chỉ khôi phục lại đầu vào ban đầu. Vì vậy phần mở rộng không làm đổi bản chất thuật toán hay bậc độ phức tạp. Nếu không nói rõ, ta có thể giả sử A là ma trận vuông khả nghịch.

Mọi ma trận khả nghịch có thể phân tích thành tích các ma trận sơ cấp. Ba thao tác sơ cấp tương ứng là:

1. hoán đổi a_i và a_j , ký hiệu $a_i \leftrightarrow a_j$;
2. nhân a_i với hằng số k , ký hiệu $a_i \leftarrow ka_i$;
3. cộng ka_j vào a_i , ký hiệu $a_i \leftarrow a_i + ka_j$.

Hai thao tác đầu khi chuyển vị vẫn giữ nguyên. Thao tác thứ ba chuyển thành

$$a_j \leftarrow a_j + ka_i.$$

Vì $A^T = V_k^T V_{k-1}^T \cdots V_1^T$, muốn lấy chuyển vị của thuật toán thì đổi từng thao tác thành thao tác chuyển vị tương ứng rồi thực hiện theo thứ tự ngược.

3. Nguyên lý chuyển vị và quy hoạch động

3.1. Tư tưởng cơ bản

Muốn dùng nguyên lý chuyển vị, trước hết cần mô tả bài toán dưới dạng một biến đổi tuyến tính. Sau đó dựa trên ý nghĩa cụ thể của ma trận A , ta diễn giải bài toán chuyển vị $b' = A^T a'$. Nếu bài toán chuyển vị có thể giải bằng DP hoặc có DP dễ tối ưu hơn, ta chuyển vị quá trình đó để thu được lời giải cho bài toán gốc.

3.2. Tối ưu quy hoạch động

Với nhiều bài toán, vẫn có thể thiết kế trạng thái và chuyển trực tiếp mà không cần nguyên lý chuyển vị. Nhưng trong các bài cần tối ưu DP bằng cấu trúc dữ liệu phức tạp, cách thiết kế trực tiếp có thể rườm rà và thiếu tự nhiên. Khi đó, nếu bài toán có thể viết thành biến đổi tuyến tính, nguyên lý chuyển vị giúp chuyển bài toán sang dạng dễ nhìn hơn.

Những bài toán chuyển vị hiện thường gặp nhất liên quan tới:

- hàm sinh và đa thức;
- cây đoạn.

Bài viết tập trung vào nhóm thứ hai: các DP cần tối ưu bằng thao tác trên cây đoạn.

4. Chuyển vị của một số thao tác trên cây đoạn

Trước khi chuyển vị một thuật toán DP tối ưu bằng cây đoạn, cần hiểu chuyển vị của các thao tác cây đoạn là gì. Khi làm điều này, phải mô tả thuật toán cây đoạn thành một biến đổi tuyến tính: dữ liệu đầu vào nào được đặt trong vector, dữ liệu nào được xem là phần cố định của ma trận A , và đầu ra nằm ở đâu.

4.1. Cộng đoạn, hỏi điểm

Xét dãy a_i độ dài n , ban đầu toàn 0, hỗ trợ:

1. $l\ r\ x$: cộng x cho mọi a_i với $i \in [l, r]$;
2. $q\ y$: ghi giá trị a_q vào y .

Các giá trị x là đầu vào, còn l, r, q được xem là cố định trong ma trận của biến đổi. Với mỗi truy vấn y_i , nó là tổng của những x_j có thao tác cộng xuất hiện trước truy vấn và thỏa $q_i \in [l_j, r_j]$. Do đó toàn bộ quá trình là một biến đổi tuyến tính.

Chuyển vị của thao tác thô. Nếu cài thô, thao tác cộng đoạn là nhiều phép

$$a_t \leftarrow a_t + x_i \quad (t = l_i, \dots, r_i),$$

còn thao tác hỏi điểm là

$$y_i \leftarrow y_i + a_{q_i}.$$

Sau khi chuyển vị và đảo thứ tự:

- thao tác $l\ r\ x$ trở thành lấy tổng $a_l + a_{l+1} + \dots + a_r$ và cộng vào x ;
- thao tác $q\ y$ trở thành cộng y vào a_q .

Nói cách khác, chuyển vị của “cộng đoạn, hỏi điểm” là “cộng điểm, hỏi tổng đoạn”.

Chuyển vị của thao tác trên cây đoạn. Dùng đánh dấu vĩnh viễn trên cây đoạn: mỗi nút pos lưu s_{pos} , nghĩa là mọi vị trí trong đoạn của nút đều được cộng thêm s_{pos} . Vector trung gian không còn là n giá trị a_i , mà là $2n - 1$ giá trị s_{pos} .

Thao tác cộng đoạn tách $[l_i, r_i]$ thành các nút cây đoạn S_i , rồi với mỗi $pos \in S_i$ thực hiện $s_{pos} \leftarrow s_{pos} + x_i$. Thao tác hồi điểm lấy các nút trên đường chứa q_i , gọi là S'_i , rồi thực hiện $y_i \leftarrow y_i + s_{pos}$ với mọi $pos \in S'_i$. Chuyển vị cho ta:

- thao tác l_i, r_i, x_i : cộng các s_{pos} của $pos \in S_i$ vào x_i ;
- thao tác q_i, y_i : cộng y_i vào mọi s_{pos} trên đường chứa q_i .

Ý nghĩa thực tế vẫn đúng là “cộng điểm, hồi tổng đoạn”. Như vậy có hai con đường để lấy chuyển vị của một thao tác cây đoạn: chuyển vị trực tiếp thao tác trên cây đoạn, hoặc trước hết chuyển vị thao tác thô rồi tối ưu bài toán mới bằng cây đoạn. Nội dung của hình gốc chỉ minh họa hai con đường này; bài viết không cần chèn lại hình để dùng kết luận.

4.2. Hai cách giải chuyển vị của thao tác cây đoạn

Từ phân tích trên, có hai phương pháp:

1. phân tích trực tiếp các thao tác trên cây đoạn rồi lấy chuyển vị;
2. tìm chuyển vị của thao tác thô, sau đó dùng cây đoạn tối ưu bài toán chuyển vị.

Phương pháp thứ nhất phức tạp hơn khi phân tích nhưng áp dụng được cho mọi thao tác trên cây đoạn. Phương pháp thứ hai dễ hơn khi thuật toán thô đơn giản, nhưng khi thuật toán gốc phức tạp, việc diễn giải và tối ưu bài toán chuyển vị có thể khó hơn, thậm chí cần thêm chứng minh về độ phức tạp và tính đúng.

4.3. Chuyển vị của hợp nhất cây đoạn

Phần này tạm không xét giá trị số, chỉ xét quá trình đảo ngược thao tác hợp nhất cây đoạn. Trong các bài toán thực tế, ta thường chạy hợp nhất cây đoạn bình thường rồi xử lý ngược lại; quá trình ngược này rất giống thao tác rollback và được gọi là tách cây đoạn. Khái niệm này khác với tách cây đoạn thông thường theo giá trị. Ở đây hai cây sau khi tách có thể có miền giá trị trùng nhau, vì chúng là hai cây trước khi hợp nhất.

Khi hợp nhất hai cây T_1, T_2 , xét hai gốc rt_1, rt_2 :

- nếu một trong hai rỗng, trả về cây không rỗng;
- nếu cả hai không rỗng, hợp nhất con trái với con trái, con phải với con phải, cập nhật thông tin và trả về rt_1 .

Muốn tách cây T trở lại T_1, T_2 , ta đảo quá trình trên: nút sinh ra từ trường hợp một bên rỗng được trả lại đúng cây con cũ; nút sinh ra từ trường hợp cả hai không rỗng thì đệ quy tách hai con rồi cập nhật hai nút gốc. Nếu cần rollback, chỉ cần ghi lại những thông tin bị sửa khi hợp nhất; thời gian và bộ nhớ vẫn giữ $O(n \log n)$.

4.4. Hợp nhất cây đoạn để tối ưu MAX-convolution

4.4.1. MAX-convolution. Với dãy viết thành chuỗi lũy thừa hình thức

$$F = \sum_{i=0}^n f_i x^i, \quad G = \sum_{i=0}^n g_i x^i,$$

định nghĩa $H = F \otimes G$ nếu với mọi k :

$$h_k = \sum_{\max(i,j)=k} f_i g_j.$$

4.4.2. Chuyển vị của MAX-convolution. Xem F là đầu vào, G là hằng số tạo ma trận A , và H là đầu ra. Chuyển vị được ký hiệu $F = H \otimes^T G$. Theo định nghĩa, thao tác gốc với mỗi cặp i, j là

$$h_{\max(i,j)} \leftarrow h_{\max(i,j)} + f_i g_j.$$

Chuyển vị của nó là

$$f_i \leftarrow f_i + h_{\max(i,j)} g_j.$$

Do đó

$$f_i = \sum_{j=0}^n h_{\max(i,j)} g_j = h_i \sum_{j=0}^i g_j + \sum_{j=i+1}^n h_j g_j.$$

4.4.3. Tính MAX-convolution. Để tính $H = F \otimes G$, duyệt $i = 0, \dots, n$, đồng thời duy trì

$$s_f = \sum_{j < i} f_j, \quad s_g = \sum_{j < i} g_j.$$

Nếu $f_i = g_i = 0$ thì $h_i = 0$. Nếu chỉ một trong hai khác 0, h_i bằng tích của giá trị khác 0 với tổng tiền tố của dãy còn lại. Nếu cả hai khác 0, ta có

$$h_i = s_f g_i + s_g f_i + f_i g_i,$$

rồi cập nhật s_f, s_g .

4.4.4. Tối ưu MAX-convolution bằng hợp nhất cây đoạn. Nếu trong DP trên cây có

$$D_i \leftarrow D_i \otimes D_v,$$

và mỗi D_i ban đầu chỉ có $O(1)$ vị trí khác 0, thì từ định nghĩa thấy vị trí k của kết quả khác 0 chỉ khi $f_k \neq 0$ hoặc $g_k \neq 0$. Vì vậy tập vị trí khác 0 của D_i luôn nằm trong hợp của các tập vị trí ban đầu thuộc cây con. Ta chỉ cần dùng cây đoạn mở nút động để lưu các vị trí khác 0.

Khi hợp nhất hai cây đoạn tương ứng với $F = D_i$ và $G = D_v$, duy trì hai tổng tiền tố s_f, s_g . Nếu một bên rỗng, đánh tag nhân với tổng tiền tố bên kia và trả về cây con đó; nếu cả hai có nút, đệ quy vào con trái rồi con phải và cập nhật. Quá trình có cùng độ phức tạp với hợp nhất cây đoạn thường, $O(n \log n)$.

4.4.5. Chuyển vị của cách tối ưu trên. Với DP có thêm mảng C_i :

$$D_i \leftarrow D_i \otimes D_v, \quad C_i \leftarrow C_i \otimes D_v + C_v \otimes D_i,$$

xem D là hằng số và bài toán là biến đổi tuyến tính theo C . Xét riêng

$$C_i \leftarrow C_i \otimes D_v.$$

Khi đảo quá trình hợp nhất, phải tách phần thuộc v khỏi cây đoạn đã hợp nhất. Viết $C_i = F, D_v = G$. Vì G là hằng số, tổng s_g giữ nguyên ý nghĩa; chỉ cần xử lý s_f theo chiều ngược.

Khi thăm nút pos được hợp nhất từ pos_1, pos_2 :

- nếu pos rỗng, trả về;

- nếu pos_1 không rỗng và pos_2 rỗng, với mọi f_i trong đoạn của pos_1 , thực hiện $f_i \leftarrow s_g f_i + s_f$;
- nếu pos_1 rỗng và pos_2 không rỗng, phép toán không liên quan đến F trong nhánh này;
- nếu cả hai không rỗng, xử lý hai con theo thứ tự ngược với lúc hợp nhất, rồi cập nhật nút hiện tại.

Nếu đi từ chuyển vị thô của MAX-convolution rồi mới tối ưu bằng cây đoạn, ta cũng có thể thu được kết quả tương tự, nhưng cần chứng minh thêm rằng chỉ những vị trí nằm trong tập đang được cây đoạn duy trì mới có ý nghĩa.

5. Ví dụ

5.1. Ví dụ 1

Nguồn. NOI2024 D2T2, *Đăng sơn*.

Tóm tắt đề. Cho cây gốc 1, cha của đỉnh i là p_i , độ sâu d_i là số cạnh từ i tới 1. Mỗi đỉnh có tham số l_i, r_i, h_i hạn chế đường leo núi hợp lệ. Một đường leo núi từ x là dãy

$$a_1 = x, a_2, \dots, a_m = 1$$

sao cho mỗi bước hoặc nhảy tới tổ tiên cấp k của đỉnh hiện tại với $l_{a_i} \leq k \leq r_{a_i}$, hoặc đi theo một quan hệ cha đặc biệt trong đề; đồng thời các lần nhảy phải thỏa ràng buộc về độ sâu h_i . Gọi f_x là số đường leo hợp lệ từ x tới 1; cần tính $f_2, \dots, f_n$.

Đặt

$$A = (f_2, f_3, \dots, f_n)^T, \quad a = (1),$$

tức cần tính $b = Aa$. Xét bài toán chuyển vị: cho hệ số

$$a' = (v_2, v_3, \dots, v_n)^T,$$

tính $b' = A^T a'$. Diễn giải thực tế là: mỗi đường leo hợp lệ có trọng số bằng trọng số v_s của điểm xuất phát s ; cần tính tổng trọng số của mọi đường leo hợp lệ.

Vì các điểm được “nhảy tới” luôn là tổ tiên của điểm nhảy trước đó, đặt g_x là tổng trọng số của mọi đường hiện đang nhảy tới x . Dùng thêm $c_{i,j}$ là số đường trong cây con i có thể nhảy lên tổ tiên ở độ sâu j , và $t_{i,j}$ là tổng trọng số của các đường như vậy. Nếu S_x là tập con của x , có dạng chuyển:

$$g_x = v_x + \sum_{i \in S_x} t_{i,d_x},$$

$$c_{x,j} = \begin{cases} [d_x - r_x \leq j \leq d_x - l_x] + \sum_{i \in S_x} c_{i,j}, & j < d_x - h_x, \\ 0, & j \geq d_x - h_x, \end{cases}$$

$$t_{x,j} = \begin{cases} g_x + \sum_{i \in S_x} t_{i,j}, & j < d_x - h_x, \\ 0, & j \geq d_x - h_x. \end{cases}$$

Có thể dùng hợp nhất cây đoạn để duy trì các mảng c, t , đạt $O(n \log n)$.

Khi chuyển vị quá trình trên, mảng c chỉ đóng vai trò hệ số nên có thể rollback trực tiếp. Bắt đầu từ $d_{1,0} = 1$, đảo các thao tác tại đỉnh x :

$$g_x \leftarrow \sum_{j < d_x - h_x} c_{x,j} t_{x,j},$$

rồi đặt $f_x \leftarrow g_x$, $t_{x,d_x} \leftarrow g_x$, và truyền $t_{x,j}$ xuống các con. Kết quả cuối cùng là mảng f . Cách nhìn chuyển vị giúp tránh việc phải tự thiết kế trực tiếp một DP khó tối ưu theo các hệ số c .

5.2. Ví dụ 2

Tóm tắt đề. Cho một cây, đỉnh i có trọng số p_i . Trọng số của một đồ thị con liên thông là giá trị lớn nhất trong các p_i của nó. Với mỗi đỉnh k , cần tính tổng trọng số của mọi đồ thị con liên thông chứa k .

Gọi $a_{i,j}$ là số đồ thị con liên thông chứa i và có trọng số lớn nhất bằng j , $A = (a_{i,j})$, và

$$a = (1, 2, \dots, n)^T.$$

Bài toán gốc là tính $b = Aa$. Xét chuyển vị với hệ số

$$a' = (v_1, v_2, \dots, v_n)^T.$$

Khi đó bài toán trở thành: mỗi đỉnh có trọng lượng v_i , trọng lượng của một đồ thị con liên thông là tổng trọng lượng các đỉnh của nó; với mỗi j , cần tính tổng trọng lượng của mọi đồ thị con liên thông có trọng số lớn nhất bằng j .

Bài toán chuyển vị này có DP tự nhiên. Gốc cây tại 1, gốc của một đồ thị con liên thông là đỉnh có độ sâu nhỏ nhất trong nó. Đặt $f_{i,j}$ là số đồ thị con liên thông có gốc i và trọng số lớn nhất j , còn $g_{i,j}$ là tổng trọng lượng của các đồ thị con đó. Ban đầu

$$F_i = x^{p_i}, \quad G_i = v_i x^{p_i}.$$

Khi gộp một con v vào i :

$$F_i \leftarrow F_i \otimes (1 + F_v),$$

$$G_i \leftarrow G_i \otimes (1 + F_v) + G_v \otimes F_i.$$

Sau mỗi lần gộp, cộng G_i vào đáp án theo trọng số. Các phép $\otimes$ là MAX-convolution và có thể tối ưu bằng hợp nhất cây đoạn, nên bài toán chuyển vị giải được trong $O(n \log n)$.

Để quay lại bài toán gốc, đảo quá trình trên. Các chuyển của F không phụ thuộc hệ số đầu vào nên chỉ cần rollback. Nếu đáp án chuyển vị nằm trong mảng y , thì với bài toán gốc cần tính

$$y = \sum_i ix_i.$$

Khi xử lý ngược tại đỉnh i :

- $G_i \leftarrow G_i + y$;
- tách các con theo thứ tự ngược: $G_v \leftarrow G_i \otimes^T F_i$, $G_i \leftarrow G_i \otimes^T (F_v + 1)$;
- sau khi tách xong, lấy đóng góp tại p_i để gán cho v_i .

Nhờ cách xử lý $\otimes^T$ bằng tách cây đoạn ở phần trước, bài toán gốc cũng được giải trong $O(n \log n)$. Với ví dụ này, nếu thiết kế DP trực tiếp cho bài toán gốc thì trạng thái, chuyển và tối ưu chuyển đều kém tự nhiên hơn đáng kể; chuyển vị đem lại lợi thế rõ ràng.

6. Tổng kết

Bài viết giới thiệu nguyên lý chuyển vị, khảo sát chuyển vị của một số thao tác trên cây đoạn, rồi dùng hai ví dụ để minh họa ứng dụng trong các bài toán quy hoạch động có thể mô tả thành biến đổi tuyến tính. Với những bài toán cần hợp nhất hoặc tách cây đoạn để tối ưu chuyển, nguyên lý chuyển vị có thể giảm đáng kể độ khó khi thiết kế trạng thái, thiết kế chuyển và chứng minh độ phức tạp, đồng thời vẫn đạt được bậc $O(n \log n)$ quen thuộc của hợp nhất cây đoạn.

7. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia đã giúp đỡ và chỉ đạo. Xin cảm ơn thầy Xiang Qizhong và thầy Hu Weidong đã quan tâm, hướng dẫn. Xin cảm ơn gia đình đã ủng hộ và động viên tác giả. Xin cảm ơn Zhang Ruichen, Wu Ruiqi và các bạn khác đã đọc kiểm và góp ý cho bài viết.

Tài liệu tham khảo

1. A. Bostan, G. Lecerf, and E. Schost. Tellegen's principle into practice. *Proceedings of the 2003 International Symposium on Symbolic and Algebraic Computation*, pages 37–44, 2003.
2. Gilbert Strang. *Linear Algebra*, 5th edition. Tsinghua University Press, 2019.
3. Chen Yu. Giới thiệu đơn giản về nguyên lý chuyển vị. Tập luận văn đội tuyển quốc gia 2020, 2020.

Bài 5. Bàn về ứng dụng cơ sở tuyến tính trong OI

Chen Xinyang, Trường Trung học số 2 Hàng Châu

Tóm tắt

Cơ sở tuyến tính là công cụ mạnh trong các bài toán về xor của dãy con, đồng thời có nhiều mở rộng hữu dụng trong thi tin học. Bài viết trình bày cách xây dựng cơ sở tuyến tính và các ứng dụng cơ bản, sau đó hệ thống hóa một số kỹ thuật tối ưu thường gặp. Phần sau giới thiệu cơ sở tuyến tính trực giao, cách lấy giao của các không gian sinh bởi xor, và cuối cùng thảo luận cơ sở tuyến tính có hỗ trợ xóa. Với biến thể có xóa, bài viết đưa ra hai cách cài đặt, trong đó có một phiên bản giữ được đầy đủ tính chất “tiền tố”, nhằm xử lý một số bài toán trước đây khó giải bằng kỹ thuật quen thuộc.

1. Dẫn nhập

Trong bài viết này, ký hiệu $\oplus$ chỉ phép xor theo bit. Với một dãy số nguyên không âm S , giả sử chỉ cần xét m bit thấp. Ta gọi dãy T là một cơ sở tuyến tính của S nếu nó thỏa ba điều kiện sau.

1. Các giá trị xor của mọi dãy con của T đôi một khác nhau, kể cả dãy con rỗng.
2. Tập các giá trị xor thu được từ dãy con của S trùng với tập các giá trị xor thu được từ dãy con của T .
3. Nếu bỏ các phần tử bằng 0, vị trí bit cao nhất bằng 1 của các phần tử trong T giảm nghiêm ngặt.

Nhìn theo đại số tuyến tính, đây chính là một cơ sở của không gian sinh bởi các véc-tơ bit trên trường $\mathbb{F}_2$. Tuy vậy, khi dùng trong OI, ta thường diễn đạt bằng thao tác xor để thuận tiện hơn.

Bố cục của bài viết như sau. Phần 2 nhắc lại cách xây dựng và các ứng dụng cơ bản. Phần 3 bàn về các kỹ thuật tối ưu khi phải xây dựng hoặc truy vấn nhiều cơ sở tuyến tính. Phần 4 giới thiệu cơ sở tuyến tính trực giao và phép lấy giao. Phần 5 tập trung vào cơ sở tuyến tính có hỗ trợ xóa, gồm một cách cài đặt đơn giản nhưng còn khuyết điểm và một cách cài đặt giữ đầy đủ tính chất tiền tố.

2. Xây dựng và ứng dụng cơ bản

Mấu chốt của cơ sở tuyến tính là phép biến đổi bảo toàn toàn bộ tập giá trị xor của dãy con. Nếu chọn hai vị trí $i \neq j$ và thay S_j bằng $S_i \oplus S_j$, thì dãy mới tương đương với dãy cũ: với mỗi dãy con ở một bên, ta

luôn đổi được sang một dãy con ở bên kia có cùng giá trị xor. Lặp lại thao tác khử bit giống khử Gauss, ta đưa dãy ban đầu về một dãy tương đương có các bit cao nhất khác nhau.

Khi chèn từng phần tử, ta duy trì mảng w_p , trong đó w_p là phần tử cơ sở có bit cao nhất bằng p , và một biến c đếm số phần tử phụ thuộc tuyến tính bị khử thành 0. Để chèn x , duyệt các bit từ cao xuống thấp. Nếu bit j của x bằng 0, bỏ qua. Nếu $w_j = 0$, đặt $w_j = x$ và dừng. Nếu $w_j \neq 0$, gán $x \leftarrow x \oplus w_j$ rồi tiếp tục. Nếu cuối cùng $x = 0$, tăng c . Độ phức tạp là $O(|S|m)$, bộ nhớ $O(m)$.

Một hệ quả quan trọng là các phần tử cơ sở khác 0 sinh ra đúng 2^r giá trị xor khác nhau, với r là số phần tử cơ sở. Nếu trong quá trình xây dựng có c phần tử bị khử thành 0, thì mỗi giá trị biểu diễn được xuất hiện đúng 2^c lần khi đếm theo dãy con của dãy gốc.

Giá trị xor lớn nhất. Sau khi có cơ sở, để tìm giá trị xor lớn nhất của một dãy con, khởi tạo $ans = 0$. Duyệt j từ cao xuống thấp và thay ans bằng $\max(ans, ans \oplus w_j)$. Cách này cũng áp dụng khi cần tối đa hóa $v \oplus x$, với x là xor của một dãy con: chỉ cần khởi tạo $ans = v$.

Đếm số dãy con có xor bằng một giá trị. Với giá trị cần kiểm tra x , thử khử x bằng cơ sở hiện có. Nếu khử được về 0, x biểu diễn được và số dãy con có xor bằng x là 2^c . Nếu không, đáp án bằng 0.

Đếm số giá trị nhỏ hơn một ngưỡng. Sau khi bỏ phần tử phụ thuộc, các giá trị xor sinh bởi cơ sở đôi một khác nhau. Ta duyệt bit từ cao xuống thấp, giữ tiền tố hiện tại và so sánh với lim . Khi bit của lim bằng 1, có thể cộng số lựa chọn làm tiền tố nhỏ hơn ngay tại bit đó; phần còn lại được chọn tùy ý từ các cơ sở ở bit thấp hơn. Nhánh còn lại buộc phải giữ tiền tố bằng lim và tiếp tục khử. Cuối cùng nhân thêm 2^c nếu đếm dãy con của dãy gốc. Cùng ý tưởng này cũng cho phép tìm giá trị xor nhỏ thứ k .

Đường đi xor lớn nhất trong đồ thị vô hướng. Bài WC2011 “Đường đi xor lớn nhất” là ví dụ kinh điển. Chọn một đường đi bất kỳ từ 1 đến n , có xor là v . Mỗi khi chèn thêm một chu trình vào đường đi, hai đầu mút không đổi và giá trị đường đi bị xor thêm giá trị xor của chu trình đó. Vì thế đáp án là giá trị lớn nhất của $v \oplus x$, trong đó x thuộc không gian sinh bởi các xor chu trình.

Không cần liệt kê mọi chu trình. Chọn một cây khung, đặt d_u là xor trên đường từ gốc đến u trong cây. Với mỗi cạnh ngoài cây (u, v, w) , giá trị $d_u \oplus d_v \oplus w$ là xor của chu trình cơ bản tương ứng. Các chu trình cơ bản này sinh ra toàn bộ không gian chu trình, nên chỉ cần đưa chúng vào cơ sở tuyến tính rồi tối đa hóa v . Độ phức tạp là $O(|V| + |E|m)$.

3. Các kỹ thuật tối ưu thường gặp

Nếu S' là cơ sở của S , T' là cơ sở của T , thì cơ sở của dãy ghép $S + T$ có thể xây từ $S' + T'$. Vì mỗi cơ sở có nhiều nhất m phần tử, phép gộp tốn $O(m^2)$. Đây là nền tảng cho cây đoạn lưu cơ sở tuyến tính trên mỗi nút: cập nhật một điểm và truy vấn một đoạn đều làm việc qua các phép gộp, đạt độ phức tạp kiểu $O(\log n \cdot m^2)$ mỗi thao tác. Nhược điểm là hằng số và nhân tử m^2 khá nặng.

Một kỹ thuật khác là đổi dãy đang duy trì sang một dãy tương đương thuận lợi hơn. Chẳng hạn với bài toán cập nhật xor trên đoạn và hỏi xor dãy con lớn nhất trên đoạn, đặt $b_i = a_i \oplus a_{i-1}$. Khi đó đoạn $a_l, \dots, a_r$ tương đương với phần tử a_l cùng dãy $b_{l+1}, \dots, b_r$, còn cập nhật xor trên đoạn chỉ làm đổi b_l và b_{r+1} . Cây đoạn trên dãy b , kèm thêm thao tác chèn a_l khi trả lời, cho lời giải tự nhiên hơn. Với cập nhật theo bước k , có thể dùng $b_i = a_i \oplus a_{i-k}$ tương tự.

Cơ sở tuyến tính tiền tố. Khi xây cơ sở từ trái sang phải, ta có thể ghi lại thời điểm mỗi phần tử cơ sở được đưa vào. Tổng quát hơn, gán cho mỗi giá trị a_i một độ ưu tiên t_i . Trong mảng cơ sở, ngoài w_j còn

lưu độ ưu tiên p_j của phần tử đang chiếm bit j . Khi chèn một cặp (t, x) , nếu gặp một bit đã có w_j nhưng độ ưu tiên của x cao hơn, ta hoán đổi x với w_j rồi tiếp tục khử phần còn lại. Nhờ đó cơ sở giữ các phần tử có độ ưu tiên cao nhất có thể.

Ứng dụng tiêu biểu là CF1100F *Ivan and Burgers*: với mỗi truy vấn tính $[l, r]$, sau khi đã xử lý đến r , chỉ cần lấy các phần tử cơ sở có độ ưu tiên ít nhất l . Cách này trả lời mọi truy vấn trong $O((n + q)m)$. Trên cây, bài SCOI2016 “Lucky Numbers” dùng độ sâu làm độ ưu tiên để lấy cơ sở trên đường tổ tiên; đường đi bất kỳ được tách thành hai đoạn quanh LCA rồi gộp. Với tập động nhưng biết trước thời điểm xóa, có thể dùng thời điểm xóa làm độ ưu tiên để tránh thao tác xóa thật sự. Tuy nhiên cách này chỉ dùng tốt trong bối cảnh offline và khi bài toán không cần thêm một thứ tự ưu tiên khác.

Khử tiếp các bit thấp. Sau khi có cơ sở, ta thường khử tiếp các bit thấp khỏi các phần tử cơ sở cao: duyệt i từ thấp lên cao, nếu $w_i \neq 0$ thì với mọi $j > i$ có bit i bằng 1, gán $w_j \leftarrow w_j \oplus w_i$. Khi đó mỗi bit thấp chỉ xuất hiện trong phần tử cơ sở tương ứng của nó. Dạng chuẩn hóa này giúp một số hàm trên cơ sở trở nên tuyến tính hơn.

Ví dụ, đặt $g(x, S)$ là giá trị lớn nhất của $x \oplus$ xor dãy con của S , và $g'(x, S)$ là giá trị nhỏ nhất. Sau khi chuẩn hóa, g' có tính tuyến tính theo xor:

$$g'(x_1 \oplus \dots \oplus x_k, S) = g'(x_1, S) \oplus \dots \oplus g'(x_k, S).$$

Ngoài ra $g(x, S) \oplus g'(x, S)$ bằng xor của các phần tử cơ sở. Trong bài “Xor Master”, tính chất này cho phép duy trì các giá trị đã biến đổi khi cơ sở của một tập S_0 mở rộng, tránh cập nhật lại toàn bộ với chi phí $O(nm)$ mỗi lần.

Kết hợp cơ sở tiền tố với cấu trúc dữ liệu. Với bài toán duy trì nhiều dãy $S_1, \dots, S_n$, hỗ trợ thêm phần tử vào một dãy và hỏi giá trị xor lớn nhất từ dãy ghép $S_l + \dots + S_r$, có thể dùng cây đoạn. Ở các nút con trái lưu cơ sở hậu tố, ở các nút con phải lưu cơ sở tiền tố. Khi truy vấn, tách đoạn tại nút đầu tiên mà hai đầu rơi sang hai phía, rồi gộp cơ sở hậu tố bên trái và cơ sở tiền tố bên phải. Cập nhật thêm phần tử tốn $O(m \log n)$, truy vấn tốn $O(m^2)$. Cách làm này cho thấy sức mạnh của cơ sở tiền tố, nhưng chưa xử lý được sửa đổi tùy ý bên trong dãy; vấn đề đó được đưa sang phần về cơ sở có xóa.

4. Cơ sở tuyến tính trực giao

Xét dãy điểm-giá trị $v(S)$ độ dài 2^m , trong đó $v(S)_i$ là số dãy con của S có xor bằng i . Phép ghép hai dãy tương ứng với tích chập xor:

$$c_k = \sum_{i \oplus j = k} a_i b_j.$$

Với hai số m -bit, định nghĩa

$$i \odot j = \text{popcount}(i \& j) \bmod 2.$$

Biến đổi Walsh–Hadamard cho xor là

$$\hat{a}_i = \sum_j (-1)^{i \odot j} a_j.$$

Nó biến tích chập xor thành nhân theo điểm.

Với một phần tử đơn x , dãy điểm-giá trị có hai lựa chọn: không lấy hoặc lấy x . Vì vậy

$$\widehat{v(S)}_k = \prod_{x \in S} (1 + (-1)^{x \odot k}).$$

Giá trị này bằng $2^{|S|}$ khi k trực giao với mọi phần tử của S , và bằng 0 trong các trường hợp còn lại. Do trực giao với S tương đương trực giao với bất kỳ cơ sở nào của S , ta chỉ cần xét cơ sở.

Điều này hữu ích khi số véc-tơ trực giao ít. Trong CF1336E2, cần đếm dãy con theo số bit 1 của xor. Nếu cơ sở ban đầu nhỏ, ta liệt kê các xor do cơ sở sinh ra. Nếu cơ sở lớn, không gian trực giao nhỏ hơn; ta liệt kê cơ sở trực giao rồi dùng FWT để suy ra đáp án. Cân bằng hai phía cho độ phức tạp khoảng $O(nm + 2^{m/2} + m^3)$.

Dựng cơ sở trực giao. Trước hết chuẩn hóa cơ sở bằng cách khử các bit thấp. Với mỗi vị trí bit i mà $w_i = 0$, bit i của một véc-tơ trực giao được chọn tự do. Với mỗi $w_i \neq 0$, bit i của véc-tơ trực giao bị xác định duy nhất để tích vô hướng với w_i bằng 0. Vì vậy, với mỗi ô trống i , tạo một véc-tơ

$$w'_i = 2^i + \sum_{\substack{j > i \\ \text{bit } i \text{ của } w_j \text{ bằng } 1}} 2^j.$$

Các w'_i sinh đúng toàn bộ không gian trực giao. Chi phí dựng là $O(m^2)$, hoặc $O(sm)$ nếu chỉ có s vị trí cần xét.

Giao của nhiều cơ sở. Muốn kiểm tra một giá trị x có biểu diễn được đồng thời bởi nhiều dãy $S_1, \dots, S_k$ hay không, ta cần giao của các không gian sinh. Dựa trên quan hệ $(A \cap B)^\perp = A^\perp + B^\perp$, có thể lấy trực giao từng cơ sở, gộp tất cả cơ sở trực giao, rồi lấy trực giao lần nữa. Nói ngắn gọn: giao bằng trực giao của hợp các không gian trực giao.

Trong UOJ698, với các tập S_i , mỗi truy vấn hỏi chỉ số nhỏ nhất i sao cho S_i không có dãy con xor bằng x . Nếu S'_i là cơ sở trực giao của S_i , thì x không biểu diễn được bởi S_i khi và chỉ khi có một phần tử của S'_i không trực giao với x . Ta quét các i , chèn các phần tử của S'_i vào một cơ sở và ghi lại thời điểm chúng xuất hiện. Truy vấn trở thành tìm thời điểm đầu tiên có phần tử không trực giao với x , xử lý trong $O(m)$ sau tiền xử lý. Tổng độ phức tạp là $O(nm^2 + \sum |S_i|m + qm)$.

5. Cơ sở tuyến tính có xóa

Các phần trước thường tránh xóa thật sự: hoặc chuyển bài toán thành gộp cơ sở trên cây đoạn, hoặc dùng thời điểm xóa làm độ ưu tiên trong một bài toán offline. Những chuyển hóa này khá mong manh. Khi bài toán bắt buộc online, hoặc khi độ dài bit lớn khiến phép gộp $O(m^2)$ trở thành nút thắt, ta cần đối mặt trực tiếp với thao tác xóa.

Một phiên bản còn khuyết điểm. Xét bài toán động trên các cặp (t_i, a_i) : chèn một cặp, xóa một cặp đã có, và với một ngưỡng lim , hỏi số giá trị xor khác nhau sinh bởi các a_i có $t_i \leq \text{lim}$, modulo 998244353. Ở đây $n, m \leq 2000$. Cơ sở tiền tố thông thường không đủ, vì điều kiện vừa theo thời gian tồn tại, vừa theo độ ưu tiên, tạo thành một truy vấn hai chiều.

Ta đánh số phần tử theo thứ tự chèn. Mỗi giá trị a_i có một biểu diễn resp_i bằng một bitset trên các phần tử chuẩn. Mỗi ô cơ sở w_j cũng lưu wresp_j , tức biểu diễn của w_j bằng các phần tử chuẩn. Nếu xóa một phần tử không nằm trong cơ sở chuẩn, chỉ cần bỏ nó. Nếu xóa một phần tử chuẩn i , ta tìm phần tử ngoài chuẩn có độ ưu tiên nhỏ nhất phù hợp để thay thế, cụ thể là một phần tử j có biểu diễn chứa i . Khi tìm được j , dùng các bitset để thay i bằng j trong mọi biểu diễn; nếu không tìm được, loại i khỏi cơ sở và cập nhật các biểu diễn liên quan. Thao tác chèn xử lý đối xứng: phần tử mới có thể đẩy một phần tử ưu tiên thấp hơn ra khỏi cơ sở.

Chi phí mỗi chèn hoặc xóa là $O((n+m)^2/w)$, với w là độ rộng từ máy, vì phải thao tác trên các bitset dài. Cách cài đặt này online và hữu ích trong một số bài, nhưng có hai khuyết điểm: chi phí phụ thuộc

vào số phần tử đang duy trì, và nó chỉ giữ được một phần tính chất tiền tố, chưa đủ để thay thế hoàn toàn cơ sở tiền tố trong mọi cấu trúc dữ liệu.

Kết hợp phiên bản có xóa với cây đoạn. Quay lại bài toán duy trì mảng a , hỗ trợ sửa một điểm $a_x = y$ và hỏi giá trị xor lớn nhất của dãy con trong đoạn. Cách ở phần 3 dùng cây đoạn và gộp cơ sở tiền tố $O(m^2 \log n)$. Với cơ sở có xóa ở trên, ta có thể tối ưu vì tại mỗi nút cây đoạn chỉ cần duy trì các phần tử chuẩn của hai con; số lượng phần tử chuẩn không vượt quá $2m$.

Cụ thể, mỗi nút lưu hai cơ sở tiền tố có xóa, một cho hướng xuôi và một cho hướng ngược. Ở lá, cơ sở chuẩn có nhiều nhất một phần tử. Ở nút trong, cơ sở được xây từ các phần tử chuẩn của hai con. Khi sửa một điểm, thay đổi ở lá tạo ra một số sự kiện chèn/xóa trong cơ sở chuẩn; các sự kiện này được truyền lên cha. Vì mỗi nút chỉ duy trì $O(m)$ phần tử, mỗi tầng chỉ phát sinh lượng nhỏ thao tác. Truy vấn vẫn tách đoạn ở điểm chia đầu tiên như phần 3: lấy cơ sở hậu tố từ bên trái và cơ sở tiền tố từ bên phải rồi gộp để trả lời.

Để giảm chi phí dựng và bộ nhớ, có thể chặn đáy cây bằng phân khối: mọi nút có độ dài đoạn không quá m được xem như lá giả, bỏ các hậu duệ của nó. Khi sửa bên trong lá giả, dừng lại trực tiếp trong $O(m^2)$, rồi truyền thay đổi lên trên. Khi truy vấn rơi hoàn toàn trong một lá giả, dựng cơ sở bằng vét cạn trên đoạn ngắn. Nhờ vậy, bài toán có lời giải online $O(nm + qm(\log n + m))$ thời gian và $O(n)$ bộ nhớ, tốt hơn phương pháp cây đoạn truyền thống về bậc độ phức tạp, dù ý tưởng và cài đặt phức tạp hơn.

Phiên bản giữ đầy đủ tính chất tiền tố. Phần này đưa ra một cài đặt mạnh hơn. Mục tiêu là dù chèn các cặp theo thứ tự nào, hình dạng cuối cùng của cơ sở vẫn giống hệt cơ sở thu được khi chèn $a_1, a_2, \dots, a_n$ theo đúng thứ tự chỉ số bằng thuật toán cơ bản ở phần 2. Tính “giống hệt” này rất quan trọng: khi xóa, ta có thể xem phần tử bị xóa như phần tử vừa được chèn cuối cùng, rồi đảo ngược quá trình chèn đó.

Để mô tả trạng thái, ngoài w_i và $wresp_i$, đặt idx_i là chỉ số cao nhất xuất hiện trong biểu diễn $wresp_i$; nếu $w_i = 0$ thì xem $idx_i = +\infty$. Khi chèn một phần tử ở vị trí p , duy trì ba đại lượng: cur là giá trị đang xử, $cresp$ là biểu diễn của nó bằng các phần tử gốc, và q là chỉ số đại diện hiện tại. Ban đầu $cur = a_p$, $cresp = 2^p$, $q = p$. Duyệt bit từ cao xuống thấp. Nếu bit hiện tại của cur bằng 0, ta vẫn có thể phải dùng $cresp$ để loại q khỏi các biểu diễn $wresp$ nhằm giữ ràng buộc mạnh hơn. Nếu bit bằng 1, ta so sánh q với idx_i ; khi cần, hoán đổi cur với w_i , $cresp$ với $wresp_i$, rồi xor để khử bit. Nếu sau một bước $q = +\infty$, có thể dừng.

Khi chèn xong, có hai khả năng. Nếu $q = +\infty$, phần tử p chỉ được thêm vào cơ sở chuẩn và không có phần tử chuẩn nào bị loại. Nếu $1 \leq q \leq n$, thì p đi vào cơ sở chuẩn còn q bị đẩy ra; khi đó cần dùng $cresp$ để sửa mọi biểu diễn có chứa q .

Thao tác xóa được thực hiện bằng cách khôi phục trạng thái trước lần chèn cuối. Trước hết phải xác định lần chèn đó kết thúc ở trường hợp nào. Nếu có một phần tử ngoài cơ sở có biểu diễn chứa p , lấy phần tử có chỉ số nhỏ nhất như vậy làm q ; khi đó có thể khôi phục $cur = 0$ và suy ra $cresp$ từ biểu diễn của q . Nếu không tồn tại, ta ở trường hợp $q = +\infty$ và cần tìm vị trí bit mà quá trình chèn đã dừng.

Dấu hiệu quan trọng là bit p trong các biểu diễn. Trước khi chèn a_p , không biểu diễn nào chứa bit này; trong quá trình chèn, bit p chỉ lan sang các $wresp$ thông qua hoán đổi hoặc qua bước khử đặc biệt ở nhánh bit 0. Vì thế khi đảo ngược từng bit, ta dùng việc $wresp_i$ có chứa p hay không để biết bước trước đó có hoán đổi hoặc có khử bằng $cresp$ hay không. Trường hợp khó nhất là phân biệt bit hiện tại của cur trước bước đó bằng 0 hay 1; bài viết chỉ ra có thể quyết định bằng cách kiểm tra các bit cao trong $cresp$ và quan hệ với idx_i . Nhờ các dấu hiệu này, mọi bước chèn đều đảo ngược được một cách xác định.

Cả chèn và xóa đều bị chi phối bởi $O(n)$ phép xor trên bitset dài n , nên mỗi sửa điểm tốn $O(n^2/w)$. Sau khi duy trì được dạng cơ sở mạnh hơn cơ sở tiền tố thông thường, truy vấn các thông tin kinh điển của cơ sở tuyến tính cũng xử lý trong $O(n^2/w)$. Tính cả thời gian xây dựng, ví dụ trong bài viết được

giải online trong $O((n + q)n^2/w)$.

Tổng kết phần cơ sở có xóa. Phiên bản ở mục đầu đơn giản hơn nhưng chỉ giữ một phần tính tiền tố. Phiên bản sau phức tạp hơn, song giữ được đầy đủ tính tiền tố và vì vậy dùng được trong những bài toán động khó hơn. Tác giả nhận xét rằng chi phí của cơ sở có xóa khó tránh khỏi việc phụ thuộc vào số phần tử được duy trì; vì thế khi số phần tử lớn hơn nhiều so với độ dài bit, các kỹ thuật truyền thống đôi khi vẫn phù hợp hơn. Tuy nhiên, trong những bài mà số phần tử cùng bậc với độ dài bit và bắt buộc online, hướng tiếp cận này mở ra các lời giải mới.

6. Tổng kết

Bài viết lần lượt trình bày cách xây dựng cơ sở tuyến tính, các ứng dụng cơ bản, những kỹ thuật tối ưu thường dùng, cơ sở trực giao, phép lấy giao và cơ sở tuyến tính có hỗ trợ xóa. Ba phần đầu chủ yếu hệ thống hóa các kỹ thuật đã quen thuộc trong thi tin học; phần cuối đưa ra thêm một số lý thuyết và cách cài đặt mới cho hướng cơ sở tuyến tính có xóa, qua đó tối ưu một số bài toán cổ điển. Tác giả hy vọng các ý tưởng này gợi mở thêm nhiều bài toán và mở rộng thú vị liên quan đến cơ sở tuyến tính.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Peng Sijin và Yang Yaoliang đã hướng dẫn. Xin cảm ơn cha mẹ, nhà trường, thầy Li Jian, cùng các bạn học đã giúp đỡ. Xin cảm ơn Zhang Shaojia và Wang Suyi đã trao đổi, thảo luận và đọc kiểm bài viết.

Bài 6. Tổng kết một số ý tưởng cho các bài toán liên quan tới dãy

Fan Sizhe, Trường THPT Hải Lượng, Chư Ky, Chiết Giang

Tóm tắt

Bài viết khảo sát các bài toán liên quan tới dãy trong thi tin học, tổng kết một số ý tưởng giải thường gặp và kỹ thuật phân tích hữu dụng. Các ví dụ được dùng để minh họa tác dụng của từng ý tưởng cũng như hoàn cảnh nên áp dụng.

1. Lời nói đầu

Bài toán trong thi tin học rất đa dạng, lời giải cũng nhiều hình thái, khiến nhiều thí sinh khó xác định điểm bắt đầu. Dù vậy, đằng sau những đề khác nhau thường có các ý tưởng chung. Nếu tách được những kỹ thuật nằm sau quá trình giải, ta có thể dùng lại chúng trong các bài sau.

Với bài toán về dãy, có thể chia thô thành hai loại. Loại thứ nhất cho sẵn một dãy và yêu cầu thống kê một đối tượng tổ hợp nào đó được định nghĩa trên dãy. Loại thứ hai cho một số tính chất mà dãy phải thỏa, rồi yêu cầu đếm hoặc xây dựng mọi dãy như vậy. Phần 2 và phần 3 lần lượt tổng kết các hướng xử lý cho hai loại này. Ngoài ra còn có nhiều bài liên quan tới thao tác trên dãy; khi đó cần chuyển hóa thao tác hoặc tìm một đại lượng tổ hợp cốt lõi để mô tả chúng. Phần 4 trình bày một số cách suy nghĩ theo hướng này.

2. Với dãy đã cho

2.1. Thống kê đóng góp của đoạn

Khi cần thống kê đóng góp của mọi đoạn, câu hỏi đầu tiên là đóng góp đó có tính trừ được hay không. Với các đại lượng như tổng đoạn hoặc tích đoạn, có thể dùng tiền tố rồi lấy hiệu, đưa đóng góp đoạn về đóng góp tại hai đầu mút. Khi đó ta thường cố định một đầu mút và thống kê tổng đóng góp của đầu còn lại.

Nếu đóng góp không có tính trừ được, các hướng quen thuộc là quét đường hoặc chia để trị. Chẳng hạn bài *Độ sâu của biển* cho dãy $a_1, \dots, a_n$, có m lần sửa một điểm, và sau mỗi lần sửa cần tính tổng giá trị nhỏ nhất của mọi đoạn. Giá trị nhỏ nhất của đoạn không thể tách bằng tiền tố; hơn nữa có sửa đổi nên quét đường trực tiếp khá khó. Ta offline toàn bộ thao tác rồi chia để trị theo thời gian.

Trong một bước chia để trị, chỉ cần tính các đoạn băng qua điểm giữa. Thay vì xử lý trực tiếp giá trị nhỏ nhất của đoạn, ta xét từng ngưỡng x và đếm số đoạn có giá trị nhỏ nhất ít nhất x . Với mỗi thời điểm i , duy trì p_i là khoảng cách từ điểm giữa sang trái đến phần tử gần nhất không vượt x , q_i là đại lượng tương tự ở bên phải, và w_i là đóng góp hiện tại. Khi x tăng, các mảng p, q chỉ chịu thao tác lấy min trên đoạn, còn w_i tăng thêm $p_i q_i$. Các thao tác này có thể duy trì bằng Segment Tree Beats; kết hợp với chia để trị cho độ phức tạp $O(n \log^2 n)$. Bài này cũng có lời giải $O(n\sqrt{n})$ dựa trên chia khối theo dãy thao tác.

Với bài toán liên quan tới cực trị của đoạn, cây Descartes cũng là một công cụ tự nhiên. Trong cây Descartes min-heap của dãy, giá trị nhỏ nhất trên $[l, r]$ chính là giá trị tại LCA của hai vị trí l, r . Vì vậy những đóng góp chứa $\min(a_l, \dots, a_r)$ thường có thể chuyển thành thống kê trên các cặp đỉnh có LCA xác định.

Ví dụ trong CF1580D *Subsequence*, cần chọn m vị trí và tối đa hóa một biểu thức gồm tổng trọng số của các vị trí được chọn trừ đi tổng giá trị nhỏ nhất trên các đoạn giữa từng cặp vị trí. Dựng cây Descartes, mỗi cặp vị trí đóng góp phần cực tiểu tại LCA của chúng. Do đó có thể làm quy hoạch động ba lô trên cây: tại mỗi đỉnh, gộp số phần tử được chọn ở hai cây con và cộng thêm phần đóng góp do các cặp có LCA tại đỉnh này. Thời gian tổng thể là $O(n^2)$.

2.2. Thống kê đóng góp của cặp

Với bài toán thống kê theo cặp, một hướng thường dùng là cố định một thành phần của cặp rồi tính tổng đóng góp của thành phần còn lại. Đôi khi cần tận dụng tính đơn điệu của đóng góp để loại bỏ các trường hợp không thể ảnh hưởng tới đáp án.

Trong CF1285F *Classical?*, cho n số nguyên và cần tìm giá trị lớn nhất của

$$\text{lcm}(a_i, a_j) = \frac{a_i a_j}{\gcd(a_i, a_j)}.$$

Ta có thể liệt kê $g = \gcd(a_i, a_j)$, chỉ xét các số là bội của g , rồi quy bài toán về tìm tích lớn nhất của hai số nguyên tố cùng nhau. Sau khi sắp xếp và xóa trùng, duyệt các giá trị từ lớn xuống. Nếu đã tìm được $a_j > a_i$ nguyên tố cùng nhau với a_i , thì mọi giá trị nằm giữa a_i và a_j sẽ không tạo ra đáp án tốt hơn với cùng nhánh duyệt.

Vì thế dùng một ngăn xếp lưu các ứng viên còn có thể đóng góp. Mỗi khi xét a_i , thử loại dần phần tử đỉnh ngăn xếp; nếu phần còn lại không còn số nào nguyên tố cùng nhau với a_i thì dừng. Bài toán còn là động: chèn/xóa phần tử trong tập S , hỏi có bao nhiêu phần tử nguyên tố cùng nhau với a_i . Dùng đảo Mobius:

$$\sum_{x \in S} [\gcd(x, a_i) = 1] = \sum_{d|a_i} \mu(d) \sum_{x \in S} [d | x].$$

Chỉ cần duy trì số phần tử trong S chia hết cho từng d . Cách trực tiếp có chi phí $O(A \log^2 A)$, với

$A = \max a_i$. Có thể tối ưu về $O(A \log A)$ bằng cách trước hết thêm mọi ước của các a_i vào dãy, vì nếu đáp án là $\text{lcm}(a_i, a_j)$, tồn tại $x \mid a_i, y \mid a_j$ sao cho $\text{gcd}(x, y) = 1$ và xy bằng đáp án đó.

3. Với dãy chưa xác định

Khi dãy cần được xác định trong quá trình giải, điểm then chốt thường là chọn thứ tự sinh hợp lý: ta nên quyết định các phần tử theo thứ tự nào? Cách trực tiếp là theo chỉ số từ trái sang phải, tức trạng thái là một tiền tố. Nhưng trong nhiều bài, ảnh hưởng của tiền tố hoặc hậu tố lên phần chưa xác định khó biểu diễn bằng ít đại lượng, nên cần thứ tự khác.

Khi đưa phần tử mới vào dãy, cũng có hai góc nhìn. Góc nhìn “tuyệt đối” là đặt phần tử vào một ô trống của dãy hoàn chỉnh. Góc nhìn “tương đối” là chèn phần tử vào giữa hai phần tử đã có, hoặc ở biên. Thường chỉ một trong hai góc nhìn cho lời giải gọn.

3.1. Xác định theo giá trị

Thay vì xác định theo chỉ số, ta có thể xác định các phần tử theo thứ tự giá trị. Trong ZJOI2012 *Sóng*, giá trị của một hoán vị $p_1, \dots, p_n$ là

$$\sum_{i=1}^{n-1} |p_{i+1} - p_i|,$$

và cần đếm số hoán vị độ dài n có giá trị bằng m . Do biểu thức tuyệt đối phụ thuộc mạnh vào quan hệ lớn nhỏ của hai số kề nhau, ta chèn các số $1, 2, \dots, n$ theo thứ tự tăng dần. Dùng góc nhìn tương đối: các số đã chèn tạo thành một số đoạn liên tiếp, và cần biết còn bao nhiêu vị trí biên chưa điền.

Đặt $dp_{i,j,k,s}$ là số cách sau khi đã chèn các số 1 đến i , tạo ra j đoạn liên tiếp, còn k vị trí biên chưa điền, và tổng đóng góp hiện tại là s . Khi chèn i , phân loại theo hai phía của vị trí chèn: kề biên hay không, kề đoạn đã có hay không. Năm trường hợp tương ứng làm tăng, giữ nguyên hoặc giảm số đoạn, đồng thời cộng các giá trị $-i, +i, -2i, 0, +2i$ vào tổng đóng góp. Cách này giải bài trong $O(n^4)$.

Có những bài cũng xác định theo giá trị nhưng không cần quyết định mọi vị trí, mà chỉ cần xét một số vị trí then chốt. Trong CF1437F *Emotional Fishermen*, cần đếm số cách sắp xếp các số sao cho với mọi $i \geq 2$, nếu

$$b_i = \max_{j < i} a_j,$$

thì $2a_i \leq b_i$ hoặc $2b_i \leq a_i$. Điều kiện gắn chặt với các giá trị cực đại tiền tố. Xét hai cực đại tiền tố liên tiếp $a_j < a_i$: phải có $2a_j \leq a_i$, còn mọi phần tử nằm giữa chúng phải không vượt quá $a_j/2$. Vì vậy chỉ cần DP trên dãy các cực đại tiền tố.

Đặt f_i là số cách khi cực đại tiền tố hiện tại là a_i và đã chèn mọi số trong $[1, a_i/2]$. Khi chuyển từ cực đại trước a_j sang a_i , phần tử a_i buộc đặt vào ô trống sớm nhất; các số trong $(a_j/2, a_i/2] \setminus \{a_j\}$ có thể đặt tự do vào các ô còn lại. Công thức chuyển chỉ phụ thuộc vào số lượng phần tử trong các khoảng, nên tối ưu bằng tiền tố và đạt $O(n)$.

3.2. Đồng thời theo giá trị và chỉ số

Đôi khi cần xác định đồng thời theo giá trị và theo chỉ số. Trong ABC134F *Permutation Oddness*, giá trị của hoán vị là

$$\sum_{i=1}^n |i - p_i|,$$

và cần đếm số hoán vị có giá trị bằng m . Đặt p' là hoán vị nghịch đảo. Khi duyệt i tăng dần, ta xét đồng thời quan hệ của p_i với i và của p'_i với i . Về bản chất, đây là bài toán ghép n phần tử với n vị trí.

Đặt $dp_{i,j,s}$ là số cách sau khi đã xác định mọi cặp (k, p_k) với $k \leq i$ và $p_k \leq i$, trong đó có j chỉ số như vậy, và tổng đóng góp của các cặp không vượt quá i là s . Khi thêm i , có bốn khả năng: i ghép với chính nó; cả phần tử lẫn vị trí ghép sang phía sau; cả hai đã ghép về phía trước; hoặc một phía trước, một phía sau. Các hệ số lần lượt là 1 , $(i-1-j)^2$, 1 , $2(i-1-j)$, và đóng góp giá trị thay đổi tương ứng. Tổng thời gian là $O(n^4)$.

4. Với thao tác trên dãy

4.1. Phân tích bản thân thao tác

Trong các bài có thao tác, tính chất của thao tác thường là điểm quyết định. Nếu có nhiều loại thao tác, trước hết nên xem chúng có thể thay thế nhau hay không, từ đó loại bớt các phương án không cần thiết.

Trong phần con của NOI2022 *Remove Stones*, ta có một dãy và hai loại thao tác: giảm một phần tử ít nhất 2, hoặc chọn một đoạn dài ít nhất 3 mà mọi phần tử đều dương rồi giảm cả đoạn đi 1. Cần quyết định có thể đưa mọi phần tử về 0 hay không trong $O(n)$. Có thể giả sử mọi thao tác đoạn được thực hiện trước, sau đó mới dùng thao tác đơn; mỗi phần tử cần thao tác đơn nhiều nhất một lần; thao tác đoạn dài ít nhất 6 có thể tách thành các đoạn dài 3, 4, 5; và không cần dùng nhiều lần cùng một thao tác đoạn. Nhờ đó, DP chỉ phải xét số rất nhỏ các đoạn đang băng qua biên hiện tại. Nếu tiếp tục loại những tổ hợp đoạn có thể thay bởi thao tác khác, số trạng thái giảm xuống còn $9n$, phù hợp với DP lồng DP của bài gốc.

Xét thao tác ngược. Khi thao tác xuôi khó mô tả, thao tác ngược đôi khi đơn giản hơn. Trong bài *Cơ sở cấu trúc vòng lặp*, ban đầu $a_i = i$, cần biến thành dãy đích b_i bằng không quá 2001 thao tác: cộng cùng một số không âm cho mọi phần tử, hoặc lấy mọi phần tử modulo một số không âm x . Đảo ngược quá trình, hai thao tác trở thành: trừ cùng một số không âm không vượt quá giá trị nhỏ nhất; hoặc cộng cho từng phần tử một bội không âm tùy chọn của x , với x lớn hơn giá trị lớn nhất hiện tại.

Từ dãy b , thao tác tăng sau khi đảo ngược rất tự do. Nếu giá trị nhỏ nhất là mn , lớn nhất là mx , ta có thể trừ mn , rồi chọn $x = mx - mn + 1$ để chỉ tăng các vị trí ban đầu đạt mn , đưa chúng vượt lên trên giá trị lớn nhất cũ. Lặp lại sẽ xử lý được trường hợp b tăng. Nếu b không tăng, trước hết dùng một thao tác với số rất lớn S để biến a_i thành $a_i + iS$, đưa bài toán về trường hợp tăng. Đảo ngược lại các bước, ta có lời giải trong $2n + 1$ thao tác.

Nghiên cứu chức năng của nhiều thao tác. Khi từng thao tác riêng lẻ không gần với mục tiêu, có thể ghép một chuỗi thao tác thành một chức năng dễ hiểu. Trong CF1748F *Circular Xor Reversal*, có một dãy vòng $a_i = 2^i$ và thao tác chọn i , gán $a_i \leftarrow a_i \oplus a_{i+1}$. Cần đảo ngược dãy. Phép xor gợi đến cách hoán đổi hai biến bằng ba phép xor. Vì vậy ta tìm cách thực hiện chức năng $a_i \leftarrow a_i \oplus a_j$ bằng một chuỗi thao tác dọc theo cung từ i đến j .

Gọi $f(i, j)$ là chuỗi: đi ngược từ $j-1$ đến i , đi xuôi từ $i+1$ đến $j-1$, đi ngược từ $j-2$ đến i , rồi đi xuôi từ $i+1$ đến $j-2$. Nếu khoảng cách vòng từ i đến j là l , chuỗi này dùng $4l-4$ thao tác và giúp mô phỏng hoán đổi. Chọn chiều ngắn hơn cho mỗi cặp cho số thao tác khoảng $2.5n^2 + O(n)$. Quan sát thêm rằng ba bước đầu của $f(i, n-i-1)$ trùng với bước đầu của $f(i-1, n-i)$, nên sắp xếp thứ tự thao tác hợp lý sẽ loại được lặp, giảm xuống khoảng $1.25n^2 + O(n)$.

4.2. Tìm đại lượng tổ hợp

Đôi khi thao tác tác động lên dãy theo cách khó phân tích trực tiếp. Khi đó nên tìm một đại lượng tổ hợp mô tả dãy, hoặc thay thế dãy trong quá trình phân tích.

Nghịch thế. Với các thao tác hoán đổi, số nghịch thế thường cho điều kiện cần hoặc cận dưới sắc. Hai kết luận kinh điển là: số lần đổi chỗ kề nhau ít nhất để sắp xếp một dãy bằng đúng số nghịch thế; và khi các phần tử đôi một khác nhau, hoán đổi hai phần tử bất kỳ luôn đổi tính chẵn lẻ của số nghịch thế.

Trong CF804E *The same permutation*, phải sắp xếp thứ tự thực hiện tất cả $n(n-1)/2$ phép hoán đổi khác nhau sao cho cuối cùng dãy trở lại ban đầu. Mỗi hoán đổi đổi tính chẵn lẻ nghịch thế, nên điều kiện cần là $n(n-1)/2$ chẵn, tức $n \bmod 4 \in \{0, 1\}$. Với hai trường hợp này có thể nhóm bốn phần tử và dựng thứ tự thao tác bằng liệt kê; chi tiết cấu trúc không phải trọng tâm của bài viết.

Dựng dãy mới. Trong NOIP2021 *Variance*, dãy ban đầu không giảm và thao tác chọn $1 < i < n$, thay a_i bằng $a_{i-1} + a_{i+1} - a_i$. Công thức phương sai được đưa về

$$n^2 D = n \sum_i a_i^2 - \left(\sum_i a_i \right)^2.$$

Thao tác trên a khó nhìn, nhưng nếu đặt $b_i = a_i - a_{i-1}$ thì thao tác chỉ hoán đổi b_i và b_{i+1} . Vì vậy mọi kết quả có thể đạt được tương ứng với một hoán vị của dãy sai phân. Khi phương sai nhỏ nhất, dãy sai phân tối ưu có dạng một dãy: giảm rồi tăng. Sắp xếp b , rồi DP quyết định đặt từng sai phân ở đầu trái hay đầu phải. Nếu $dp_{i,j}$ là giá trị tốt nhất sau khi đã đặt i sai phân và tổng các phần tử a hiện tại là j , mỗi bước có hai lựa chọn đặt trái/phải. Độ phức tạp là $O(nA \min(n, A))$, với $A = \max a_i$.

Trong bài *Thiên bách*, thao tác trên dãy vòng đổi dấu một phần tử và trừ giá trị cũ của nó khỏi hai láng giềng; mục tiêu là làm mọi cặp kề nhau trái dấu. Xét đại lượng tiền tố

$$b_i = \sum_{j=0}^i (-1)^j a_j.$$

Một thao tác tương đương với đổi chỗ hai giá trị kề nhau trong dãy b , còn yêu cầu cuối cùng tương đương với b đơn điệu nghiêm ngặt. Sau khi cắt vòng thành dãy và xét các lớp chỉ số modulo n , bài toán trở thành sắp xếp các lớp bằng hoán đổi kề nhau. Số lần hoán đổi cần thiết được tính bằng một dạng đếm nghịch thế hai chiều; thao tác sửa trên a chỉ cộng cùng một lượng vào cả một lớp trong b . Do đó dùng cây trong cây hoặc chia để trị offline xử lý được trong $O(n \log^2 n)$.

Duy trì tập bộ giá trị. Một số bài có thao tác giống “biến hoán vị thành hoán vị nghịch đảo”. Khi đó duy trì tập các bộ (i, a_i) , thay vì chỉ duy trì dãy a , thường làm thao tác trở nên rõ ràng hơn. Trong CF1458C *Latin Square*, ma trận $n \times n$ có mỗi hàng, mỗi cột là một hoán vị. Sáu thao tác gồm dịch vòng các hàng/cột và biến mỗi hàng hoặc mỗi cột thành hoán vị nghịch đảo. Nếu duy trì tập bộ ba (x, y, z) với $z = a_{x,y}$, bốn thao tác dịch chỉ cộng/trừ 1 vào một tọa độ modulo n , còn hai thao tác nghịch đảo lần lượt hoán đổi (y, z) hoặc (x, z) . Vì thế chỉ cần lưu một hoán vị trên ba tọa độ và độ lệch của từng tọa độ, cập nhật mỗi thao tác trong $O(1)$, rồi phục hồi ma trận cuối trong $O(n^2)$.

Dựng đồ thị. Dựng đồ thị từ dãy là một chiến lược mạnh, đặc biệt với hoán vị. Cách quen thuộc nhất là xét các chu trình của hoán vị. Trong CF1787F *Inverse Transformation*, phép biến đổi thay a_i bằng a_{a_i} . Xem hoán vị như đồ thị mỗi đỉnh có một cạnh ra, ta có các chu trình rời nhau. Hàm $f(x)$ trong đề đúng bằng độ dài chu trình chứa x , nên mục tiêu tối thiểu hóa tổng $\sum 1/f(i)$ chính là tối thiểu hóa số chu trình trong hoán vị gốc. Một lần biến đổi làm chu trình chẵn tách đôi, còn chu trình lẻ giữ nguyên. Đảo ngược quá trình, ta cố gắng gộp các chu trình cuối cùng nhiều nhất có thể: chu trình lẻ có thể gộp theo nhóm 2^i với $i \leq k$, còn chu trình chẵn phải gộp theo nhóm 2^k . Khi biết cấu trúc chu trình ban đầu, thứ tự các phần tử trên chu trình dựng lại được bằng công thức, tổng thời gian $O(n)$.

Trong THUPC2024 sơ tuyển *Sorting Master*, thao tác hoán đổi hai đoạn con rời nhau của một hoán vị và cần tối thiểu hóa số thao tác. Nếu chỉ nối cạnh $a_i \rightarrow a_{i+1}$, mọi trạng thái đều là một đường đi nên

khó phân tích. Thêm 0 ở đầu và $n + 1$ ở cuối, rồi nối cạnh $a_i + 1 \rightarrow a_{i+1}$. Trạng thái đã sắp xếp có $n + 1$ vòng tự khép. Một thao tác tương đương với hai lần đổi đầu mút cạnh: đổi cạnh vào của hai đỉnh và đổi cạnh ra của hai đỉnh. Mỗi lần như vậy làm số chu trình đổi nhiều nhất 1, nên một thao tác làm số chu trình đổi nhiều nhất 2 và bảo toàn tính chẵn lẻ. Từ đó suy ra điều kiện vô nghiệm và cận dưới; phân dựng đạt cận bằng cách luôn tăng số chu trình thêm 2.

Trong CF1458D *Flip and Reverse*, với dãy nhị phân, một thao tác chọn đoạn có số 0 và 1 bằng nhau, đảo bit rồi lật ngược đoạn; cần tìm dãy cuối cùng nhỏ nhất theo từ điển. Đặt 0 là $+1$, 1 là -1 , và xét dãy tiền tố s . Một thao tác chọn đoạn $[l, r]$ với $s_{l-1} = s_r$, rồi đảo ngược $s_l, \dots, s_{r-1}$. Dựng đồ thị có cạnh từ s_i đến s_{i+1} . Đoạn hợp lệ chính là một chu trình, thao tác tương đương đảo hướng mọi cạnh trên chu trình đó. Sau nhiều thao tác, dãy s là một đường đi Euler trong đồ thị vô hướng thu được khi bỏ hướng các cạnh. Dãy nhị phân nhỏ nhất tương đương với dãy s nhỏ nhất theo từ điển, nên dùng tham lam tìm đường Euler nhỏ nhất trong $O(n)$.

5. Tổng kết

Bài viết tổng kết sơ bộ một số ý tưởng và kỹ thuật cho các bài toán liên quan tới dãy trong thi tin học, đồng thời dùng nhiều ví dụ để chỉ ra cách áp dụng. Nhiều ý tưởng trong số này không chỉ dùng cho dãy mà còn mở rộng được sang các cấu trúc khác. Do số lượng bài toán rất lớn và trình độ tác giả có hạn, bản tổng kết khó tránh khỏi chưa đầy đủ; hy vọng nó đóng vai trò gợi mở để người đọc tiếp tục tích lũy và phát triển thêm các phương pháp của riêng mình.

Bài 7. Bàn về tính chất và ứng dụng của dãy Thue–Morse

Zheng Jun, Trường Trung học số 2 Cù Châu, Chiết Giang

Tóm tắt

Xoay quanh dãy Thue–Morse, bài viết giới thiệu một số định nghĩa tương đương của dãy, khảo sát các tính chất của nó khi xem như một xâu vô hạn, tính chất của hàm sinh, và ứng dụng trong bài toán tổng lũy thừa bằng nhau, xây dựng xâu vô hạn không chứa bình phương cùng một vài tình huống khác trong OI.

1. Dãy Thue–Morse

Dãy Thue–Morse, còn gọi là dãy Prouhet–Thue–Morse, là một dãy vô hạn trên bảng chữ cái $\{0, 1\}$. Dãy này đã được Prouhet nghiên cứu từ năm 1851. Dù định nghĩa rất đơn giản, nó xuất hiện trong số học, tổ hợp trên từ và trò chơi tổ hợp, cho thấy dãy có nhiều tính chất tinh tế.

Vì dãy chỉ gồm hai ký tự 0, 1, trong nhiều bài toán ta xem nó như một xâu vô hạn, gọi là xâu Thue–Morse hoặc xâu TM. Các ký tự đầu tiên là

01101001100101101001011001101001 ...

Trong phần còn lại, ký hiệu T_i là ký tự thứ i , đánh số từ 0, và T là toàn bộ xâu vô hạn.

2. Các định nghĩa tương đương

Định nghĩa bằng chẵn lẻ chữ số. Đặt T_i bằng tổng các chữ số trong biểu diễn nhị phân của i , lấy modulo 2. Ví dụ $11 = (1011)_2$, nên $T_{11} = 1$.

Từ định nghĩa này suy ra ngay

$$T_{2n} = T_n, \quad T_{2n+1} = 1 - T_n.$$

Đây là định nghĩa đệ quy của dãy.

Định nghĩa bằng thay thế. Xét phép thế μ trên xâu:

$$\mu(0) = 01, \quad \mu(1) = 10,$$

và mở rộng theo quy tắc $\mu(w_1 w_2 \cdots w_n) = \mu(w_1) \mu(w_2) \cdots \mu(w_n)$. Khi đó

$$\mu^0(0) = 0, \quad \mu^1(0) = 01, \quad \mu^2(0) = 0110, \quad \mu^3(0) = 01101001, \dots$$

Dãy Thue–Morse là điểm bất động

$$T = \mu^\infty(0).$$

Việc $\mu^k(0)$ luôn là tiền tố của $\mu^{k+1}(0)$ bảo đảm giới hạn này được xác định tốt.

Định nghĩa bằng đảo bit. Nếu đã biết 2^n ký tự đầu của T , thì 2^n ký tự tiếp theo là phần đảo bit của chúng. Đặt $TM_0 = 0$ và

$$TM_{2^{n+1}} = TM_{2^n} \overline{TM_{2^n}},$$

trong đó dấu gạch ngang chỉ đảo $0 \leftrightarrow 1$. Khi $n \rightarrow \infty$, ta thu được lại T . Định nghĩa này đặc biệt hữu ích khi cần khai thác tính đối xứng của dãy.

3. Dãy Thue–Morse và bài toán tổng lũy thừa bằng nhau

Bài toán tổng lũy thừa bằng nhau hỏi: với số nguyên dương n, k , tìm hai tập rời nhau A, B , mỗi tập có n phần tử, sao cho với mọi $i = 0, 1, \dots, k$,

$$\sum_{a \in A} a^i = \sum_{b \in B} b^i.$$

Khi n là lũy thừa của 2, dãy Thue–Morse cho một cấu trúc rất tự nhiên.

Với $k \in \mathbb{N}$, đặt

$$X_k = \{n \in [0, 2^{k+1}) \mid T_n = 0\}, \quad Y_k = \{n \in [0, 2^{k+1}) \mid T_n = 1\}.$$

Theo định nghĩa đảo bit, hai tập có cùng kích thước 2^k . Hơn nữa, với mọi đa thức f có bậc không quá k ,

$$\sum_{n \in X_k} f(n) = \sum_{n \in Y_k} f(n).$$

Chứng minh dùng quy nạp theo k . Với $k = 0$ hiển nhiên đúng. Với $k \geq 1$, đặt $g(n) = f(n + 2^k) - f(n)$. Khi đó $\deg g \leq k - 1$; áp dụng giả thiết quy nạp cho g , rồi chuyển các nửa 2^k phần tử bằng định nghĩa đảo bit, ta thu được đẳng thức cho f . Lấy $f(n) = n^i$ chính là lời giải cho bài toán tổng lũy thừa bằng nhau.

Ví dụ ứng dụng. Trong bài *Angry Little N*, cho n dưới dạng tập các bit 1 và một đa thức f bậc không quá 500, cần tính

$$\sum_{i=0}^{n-1} T_i f(i) \pmod{10^9 + 7}.$$

Ta chuyển về tổng

$$\sum_{i=0}^{n-1} (-1)^{T_i} f(i),$$

còn tổng $\sum f(i)$ xử lý bằng nội suy Lagrange cổ điển. Với mỗi bit 2^k của n , phần đóng góp tương ứng có dạng

$$(-1)^{T_s} \left(\sum_{i \in X_{k-1}} f(i+s) - \sum_{i \in Y_{k-1}} f(i+s) \right).$$

Nếu $\deg f \leq k-1$, phần này bằng 0 theo định lý trên; các trường hợp còn lại xử lý bằng khai triển nhị thức. Độ phức tạp đạt $O(\log n + \deg^3 f)$.

Mở rộng cơ số. Với cơ số $b \geq 2$, đặt $s_b(n)$ là tổng chữ số của n trong hệ cơ số b , lấy modulo b . Với

$$S_{k,i} = \{n \in [0, b^{k+1}) \mid s_b(n) = i\},$$

ta cũng có, với mọi đa thức f bậc không quá k ,

$$\sum_{n \in S_{k,0}} f(n) = \sum_{n \in S_{k,1}} f(n) = \dots = \sum_{n \in S_{k,b-1}} f(n).$$

Chứng minh tương tự: xét hiệu $f(n+C_1) - f(n+C_2)$, bậc giảm đi một, rồi gom các chữ số theo tổng modulo b .

4. Tính chất xâu của Thue–Morse

Trong phần này, ta dùng một số ký hiệu chuẩn. Với xâu w , ký hiệu $|w|$ là độ dài, w^R là xâu đảo ngược, $\bar{w}$ là xâu đảo bit. Xâu con $w(l:r)$ gồm các ký tự từ vị trí l đến r . Xâu rỗng ký hiệu là ε .

Từ định nghĩa đảo bit, các khối TM_{2^n} có đối xứng mạnh:

$$TM_{2^{2n}} = TM_{2^{2n}}^R, \quad TM_{2^{2n+1}} = \overline{TM_{2^{2n+1}}^R}.$$

Suy ra nếu w là xâu con của T , thì $\bar{w}$, w^R và $\overline{w^R}$ cũng đều là xâu con của T .

4.1. Tiền tố gần TM và phân tách TM

Gọi w là **tiền tố gần TM** nếu w là tiền tố của T hoặc của $\bar{T}$. Nếu w là tiền tố gần TM thì $\bar{w}$ cũng vậy; mọi tiền tố của w cũng là tiền tố gần TM.

Một tiêu chuẩn tiện dụng là: w là tiền tố gần TM khi và chỉ khi với mọi $0 \leq i < |w|$,

$$w(0) \oplus w(i) = T_i.$$

Điều này nói rằng một tiền tố gần TM được xác định bởi ký tự đầu tiên và cấu trúc xor tương đối của T .

Với xâu w , gọi cặp (u, v) là một **phân tách TM** của w nếu

$$uv = w,$$

và cả u^R lẫn v đều là tiền tố gần TM. Khi đó có một mô tả chính xác: nếu $|w| = 1$, w hiển nhiên là xâu con của T ; nếu $|w| > 1$, w là xâu con của T khi và chỉ khi w có ít nhất một phân tách TM.

Ý tưởng chứng minh chiều cần là chọn trong đoạn xuất hiện của $w = T(l : r)$ một vị trí $i \in (l, r]$ có $\text{lowbit}(i)$ lớn nhất, rồi tách ở đó. Hai phần hai bên, sau khi đảo ngược bên trái, đều khớp với tiền tố của T hoặc $\bar{T}$. Chiều đủ dùng một khối $T(0 : 7 \cdot 2^k - 1)$ đủ lớn; với mọi lựa chọn u^R, v là tiền tố gần TM, một trong vài vị trí đối xứng trong khối này sẽ khớp, nên uv xuất hiện trong T . Nhờ vậy, số xâu con TM độ dài n không vượt quá $4n$.

4.2. Số phân tách TM

Nếu một xâu w có hai phân tách TM, viết $w = xyz$ sao cho hai phân tách là (x, yz) và (xy, z) . Khi đó y và y^R đều là tiền tố gần TM. Từ tiêu chuẩn xor ở trên suy ra $|y|$ phải là một lũy thừa của 2. Cụ thể, nếu $|y|$ không phải lũy thừa của 2, chọn bit 0 cao nhất trong biểu diễn nhị phân của $|y| - 1$; so sánh $T(i) \oplus T(|y| - 1 - i)$ tại vị trí tương ứng sẽ mâu thuẫn với việc biểu thức này phải là hằng số.

Từ đó suy ra một xâu chỉ có nhiều nhất hai phân tách TM. Nếu có ba phân tách, hai đoạn chồng lấn liên tiếp v, x và vx đều phải có độ dài là lũy thừa của 2, nên $|v| = |x|$. Áp dụng tính chất tiền tố gần TM cho các cặp đoạn liên quan, ta buộc được ba ký tự đầu của ba đoạn đôi một khác nhau, điều không thể trên bảng chữ cái nhị phân.

Hơn nữa, hai phân tách (x, yz) , (xy, z) tồn tại khi và chỉ khi

$$|x|, |z| \leq |y|,$$

và cả $(xy)^R$ lẫn yz là tiền tố gần TM. Điều kiện này cho phép nhận diện chính xác trường hợp một xâu có hai phân tách.

Ứng dụng cho truy vấn xâu. Trong một bài do tác giả thiết kế, với $S \in \{0, 1, ?\}^*$, đặt $h(S)$ là tổng, trên mọi cách thay dấu hỏi bằng 0 hoặc 1, của số xâu con đồng thời là xâu con của T . Cần trả lời nhiều truy vấn đoạn. Dựa vào phân tách TM, ta đếm mỗi xâu con bằng

số xâu con độ dài 1 + tổng số phân tách TM – số xâu con có đúng hai phân tách TM.

Phần khớp với T hoặc $\bar{T}$ có thể xử lý bằng nhảy đôi theo định nghĩa $TM_{2^{n+1}} = TM_{2^n} \overline{TM_{2^n}}$. Với xâu có hai phân tách, liệt kê độ dài phần chồng lấn là 2^k , rồi chỉ cần khớp tối đa 2^k ký tự về hai phía. Kết hợp quét đường và cây đoạn lưu tổng lịch sử sẽ xử lý được các truy vấn đoạn.

4.3. Tính không chồng lấn

Một xâu vô hạn w gọi là **không chồng lấn** nếu không chứa hai xâu con bằng nhau có giao nhau. Tương đương, w không chứa xâu con dạng

$$auaua,$$

với a là một ký tự và u là một xâu hữu hạn, có thể rỗng. Một phía là vì hai bản sao aua chồng lên nhau; phía còn lại thu gọn hai xâu con chồng lấn bất kỳ cho đến khi phần giao có độ dài một ký tự.

Định lý quan trọng là T không chồng lấn. Bài viết chứng minh bằng cách giả sử tồn tại $auaua$, lấy một phân tách TM của nó và chuyển mọi trường hợp về dạng trong đó nửa phải của phân tách là một tiền tố của T . Đặt

$$f(n) = \text{LCP}(T, T(n : \infty)),$$

và $g(n)$ là số nguyên dương lớn nhất sao cho

$$T(n - g(n) : n - 1)^R$$

là tiền tố gần TM. Từ giả thiết chồng lấn suy ra tồn tại $n \geq 1$ với

$$f(n) + g(n) > n.$$

Sau đó chứng minh loại bỏ đề:

$$g(n) = n \iff n \text{ là lũy thừa của } 2,$$

$$g(n) \neq n - 1, \quad f(2n) = 2f(n), \quad g(2n) = 2g(n),$$

và $f(2n + 1) \leq 2$. Tách trường hợp $n = 1, 2, n$ lẻ, n chẵn, ta thu được

$$f(n) + g(n) \leq n,$$

mâu thuẫn. Vì vậy T không chồng lấn.

Một hệ quả trực tiếp là T không chứa lập phương, tức không chứa xâu con dạng uuu , vì lập phương luôn tạo ra chồng lấn.

Xâu vô hạn không chứa bình phương trên ba ký tự. Một xâu gọi là **không chứa bình phương** nếu không chứa uu với $u \neq \varepsilon$. Trên bảng chữ cái hai ký tự không có xâu vô hạn không chứa bình phương, nhưng với ba ký tự có thể xây dựng từ T . Gọi

$$X = \{n \mid T_n = 0\} = \{x_0 < x_1 < x_2 < \dots\},$$

và đặt

$$c(n) = x_{n+1} - x_n - 1.$$

Vì T không chứa ba ký tự 1 liên tiếp, $c(n) \in \{0, 1, 2\}$. Các ký tự đầu của c là

$$210201210120210201202101 \dots$$

Xâu này cũng là điểm bất động của phép thế $2 \mapsto 210, 1 \mapsto 20, 0 \mapsto 1$, nên còn gọi là dãy Thue–Morse ba ký tự. Nếu c chứa một bình phương uu , khi chuyển ngược sang các khoảng giữa các số 0 trong T , ta thu được một xâu chồng lấn trong T , mâu thuẫn. Do đó c không chứa bình phương.

Trong bài *Red*, cần dựng một xâu độ dài n trên $\{0, 1, 2\}$ có đúng m xâu con bình phương. Ý tưởng là lấy tiền tố của xâu không bình phương c , rồi kéo giãn ký tự thứ i thành một đoạn liên tiếp độ dài l_i . Số bình phương sinh ra bên trong một đoạn chỉ phụ thuộc vào l_i ; chọn các l_i bằng ba lô để đạt m với độ dài nhỏ nhất, sau đó nối thêm các đoạn độ dài 1 cho đủ n .

4.4. Xâu con đặc trưng ngắn nhất

Với một xâu vô hạn w , gọi u là **n -đặc trưng** của w nếu mọi xâu con của w có độ dài không vượt quá n đều xuất hiện trong u . Nếu u còn là tiền tố của w , gọi là tiền tố **n -đặc trưng**.

Đặt $\text{pre}(n)$ là độ dài tiền tố n -đặc trưng ngắn nhất của T . Các giá trị đầu là 2, 7, 8, 15, 16, 29, Bài viết chứng minh công thức

$$\text{pre}(n) = \begin{cases} 2, & n = 1, \\ 7, & n = 2, \\ 3 \cdot 2^{\lceil \log_2(n-1) \rceil} + n - 1, & n \geq 3. \end{cases}$$

Chứng minh chặn trên dùng phân tách TM của một xâu con độ dài n : với $k = \lceil \log_2(n-1) \rceil$, trong khối $T(0 : 4 \cdot 2^k - 1)$ luôn có một vị trí cho phép khớp phần trái đảo ngược và phần phải của phân tách. Chặn dưới xét xâu đặc biệt

$$T(2^{k-1} - 1) T(0 : n - 2)$$

và dùng hàm $f(n) = \text{LCP}(T, T(n : \infty))$ để chứng minh lần xuất hiện đầu tiên của nó không thể kết thúc quá sớm.

Nếu không yêu cầu là tiền tố, đặt $\text{sub}(n)$ là độ dài xâu n -đặc trưng ngắn nhất. Khi đó

$$\text{sub}(n) = \begin{cases} 2, & n = 1, \\ 5, & n = 2, \\ 2^{\lceil \log_2(n-1) \rceil + 1} + 2(n-1), & n \geq 3. \end{cases}$$

Chặn trên lấy từ chứng minh của $\text{pre}(n)$ bằng cách rút ngắn phần khớp bên trái xuống đúng $n-1$. Chặn dưới phức tạp hơn và được lược bỏ trong bài gốc.

5. Hàm sinh của Thue–Morse

Đặt

$$A = \{n \in \mathbb{N} \mid T_n = 0\}, \quad B = \{n \in \mathbb{N} \mid T_n = 1\},$$

và các hàm sinh

$$A(x) = \sum_{a \in A} x^a, \quad B(x) = \sum_{b \in B} x^b.$$

Rõ ràng

$$A(x) + B(x) = \frac{1}{1-x}.$$

Từ đệ quy $T_{2n} = T_n$, $T_{2n+1} = 1 - T_n$, ta có

$$A(x) - B(x) = (1-x)(A(x^2) - B(x^2)).$$

Lặp lại với $x^2, x^4, \dots$, rồi dùng hệ số hằng bằng 1, thu được tích vô hạn

$$A(x) - B(x) = \prod_{k=0}^{\infty} (1 - x^{2^k}) = \sum_{n=0}^{\infty} (-1)^{T_n} x^n.$$

Đây là một cách nhìn khác của định nghĩa theo tổng chữ số: mỗi bit của n góp một thừa số $1 - x^{2^k}$.

Chứng minh lại bài toán tổng lũy thừa. Với X_k, Y_k , hiệu tổng lũy thừa bậc m là

$$\sum_{x \in X_k} x^m - \sum_{y \in Y_k} y^m = \sum_{n=0}^{2^{k+1}-1} (-1)^{T_n} n^m.$$

Viết $n^m = [x^m] e^{nx} m!$, biểu thức này trở thành hệ số bậc m của

$$\prod_{j=0}^k (1 - e^{2^j x}).$$

Tích có $k+1$ thừa số đều có hệ số hằng bằng 0, nên mọi hệ số bậc $\leq k$ đều bằng 0. Điều này đúng với kết luận ở phần 3.

Một bài toán chia tự nhiên. Chia $\mathbb{N}$ thành A, B sao cho với mọi n , số cách biểu diễn n thành tổng của hai số khác nhau trong A bằng số cách tương ứng trong B . Từ các đẳng thức hàm sinh ở trên suy ra

$$A(x)^2 - A(x^2) = B(x)^2 - B(x^2).$$

Hệ số của x^n chính là số cách biểu diễn n thành tổng hai phần tử khác nhau trong tập tương ứng. Ngược lại, nếu C, D là một phân hoạch khác thỏa tính chất này và $0 \in C$, thì hệ phương trình hàm sinh buộc

$$C(x) - D(x) = \prod_{k=0}^{\infty} (1 - x^{2^k}),$$

nên $C = A, D = B$. Vì vậy phân hoạch Thue–Morse là nghiệm duy nhất, sau khi cố định $0 \in A$.

6. Các tính chất và ứng dụng khác

6.1. Phỏng đoán Shevelev

Đặt $u(n) = (-1)^{T_n}$. Với số nguyên dương a , chia các số tự nhiên thành hai dãy chẵn lẻ:

$$B_a = \{\ell \mid u(\ell + a) = -u(\ell)\}, \quad C_a = \{m \mid u(m + a) = u(m)\}.$$

Nếu $B_a = \{\ell_0 < \ell_1 < \dots\}$, $C_a = \{m_0 < m_1 < \dots\}$, đặt

$$\beta_a(n) = u(\ell_n), \quad \gamma_a(n) = u(m_n).$$

Định lý Shevelev nói rằng β_a, γ_a có chu kỳ nhỏ nhất

$$2 \text{ lowbit}(a),$$

và $\beta_a(n) = -\gamma_a(n)$. Hơn nữa, trên một chu kỳ, γ_a chính là $\{u(n)\}$ hoặc $\{-u(n)\}$, tùy $u(a) = +1$ hay -1 .

Với a lẻ, xét tổng tiền tố của

$$w_a(n) = u(a + n) + u(n).$$

Các vị trí thuộc C_a cho giá trị ± 2 , các vị trí còn lại cho 0. Dùng $u(2n) + u(2n + 1) = 0$, phân biệt n chẵn/lẻ, ta thấy tổng tiền tố chỉ nhận hai giá trị $\{0, +2\}$ nếu $u(a) = +1$, hoặc $\{0, -2\}$ nếu $u(a) = -1$. Điều này xác định chu kỳ của γ_a . Với a chẵn, từ đệ quy của u suy ra tính chất cho $2a$ từ tính chất của a . Thay $+$ bằng $-$ trong $u(n + a) \pm u(n)$ cho kết luận về β_a .

6.2. Va chạm băm

Băm xâu bằng tràn tự nhiên rất phổ biến vì không cần phép modulo rõ ràng và chạy nhanh hơn modulo số nguyên tố lớn. Tuy nhiên, nếu modulo thật sự là $p = 2^k$, cơ sở là b , giá trị băm của w là

$$\sum_{i=0}^{|w|-1} w(i)b^i \pmod{p},$$

thì dãy Thue–Morse cho cách tạo va chạm.

Nếu b chẵn, $b^k \equiv 0 \pmod{p}$, nên va chạm rất dễ dựng. Nếu b lẻ, xét hai khối TM_{2^n} và $\overline{TM_{2^n}}$. Hiệu hai giá trị băm là

$$\sum_{i=0}^{2^n-1} (-1)^{T_i} b^i = \prod_{i=0}^{n-1} (1 - b^{2^i}).$$

Vì $1 - b$ và $1 + b^{2^i}$ đều chẵn, tích chứa rất nhiều thừa số 2. Với modulo 2^{64} , chỉ cần lấy $n = 11$ là đủ làm hai xâu có cùng băm theo tràn tự nhiên.

7. Tổng kết

Bài viết giới thiệu bốn định nghĩa tương đương của dãy Thue–Morse, rồi trình bày ứng dụng trong bài toán tổng lũy thừa bằng nhau và các mở rộng của nó. Tiếp đó, thông qua khái niệm tiền tố gần TM và phân tách TM, bài viết phân tích cấu trúc sâu con của Thue–Morse, chứng minh tính không chồng lấn, xây dựng sâu vô hạn không chứa bình phương trên bảng chữ cái ba ký tự, và khảo sát độ dài sâu đặc trưng ngắn nhất. Phần cuối dùng hàm sinh để chứng minh lại một số kết quả, trình bày phỏng đoán Shevelev và ứng dụng tạo va chạm cho băm sâu bằng tràn tự nhiên. Hy vọng những ý tưởng này giúp người đọc hiểu sâu hơn về dãy Thue–Morse.

8. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn thầy Ye Minhong của Trường Trung học số 2 Cù Châu đã quan tâm và hướng dẫn. Xin cảm ơn gia đình, bạn bè đã ủng hộ và động viên. Xin cảm ơn Fang Xintong đã thảo luận, gợi ý và giúp chỉnh sửa bài viết.

Tài liệu tham khảo

1. OEIS A010060. <https://oeis.org/A010060>.
2. OEIS A036577. <https://oeis.org/A036577>.
3. Christopher Williamson. *An Overview of the Thue–Morse Sequence*, 2012.
4. Vladimir Shevelev. *Equations of the Form $t(x + a) = t(x)$ and $t(x + a) = 1 - t(x)$ for Thue–Morse Sequence*, 2012.
5. Jean-Paul Allouche. *Thue, Combinatorics on Words, and Conjectures Inspired by the Thue–Morse Sequence*. *Journal de Theorie des Nombres de Bordeaux*, 2015.
6. Hs-black. <https://www.cnblogs.com/Hs-black/p/12219270.html>, 2020.
7. Ricardo Astudillo. *On a Class of Thue–Morse Type Sequences*, 2003.

Bài 8. Bàn về nhân ma trận và phép quy giảm

Chen Nuo, Trường Văn Uyên thuộc Tập đoàn Giáo dục Trung học Học Quân, Hàng Châu

Tóm tắt

Tối ưu độ phức tạp của nhân ma trận là một vấn đề kinh điển; thuật toán tốt nhất hiện nay giải được nhân hai ma trận nguyên $n \times n$ trong thời gian khoảng $O(n^{2.371339})$. Việc quy giảm giữa một bài toán và nhân ma trận có thể dùng để chứng minh cận trên hoặc cận dưới về độ phức tạp. Bài viết giới thiệu khái niệm quy giảm, một số quy giảm kinh điển giữa nhân ma trận và các bài toán dữ liệu, cũng như vài thuật toán nhanh hơn cho bài toán cổ điển nhờ nhân ma trận.

1. Định nghĩa và quy ước

Với một bài toán, ta xem nó như một hàm A : đầu vào là x , đáp án là $A(x)$. Giải bài toán nghĩa là thiết kế thuật toán tính $A(x)$. Trong ngữ cảnh OI, ta chỉ xét các bài toán trên máy Turing xác định và so sánh thuật toán bằng thời gian tính toán.

Với hai bài toán A, B , nếu tìm được hàm dễ tính f sao cho

$$A(x) = B(f(x)),$$

thì ta nói A quy giảm về B , ký hiệu $A \leq B$. Nếu chi phí tính f nhỏ hơn chi phí giải B , thì mọi thuật toán cho B đều cho thuật toán cho A . Khi $A \leq B$ và $B \leq A$, hai bài toán có độ khó tương đương, bỏ qua chi phí quy giảm.

Với hai ma trận $N \times N$, tích $C = A \times B$ được định nghĩa bởi

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}.$$

Tùy tập phần tử và hai phép toán, ta có nhân ma trận nguyên, thực, modulo, ma trận 01 với cộng–nhân số học, hoặc nhân ma trận bool với $\vee, \wedge$. Nếu xem mỗi phép toán cơ bản tốn $O(1)$, độ phức tạp tốt nhất đã biết của nhân ma trận nguyên là $O(n^\omega)$, với $\omega \approx 2.371339$ tại thời điểm bài viết.

Trong OI, các thuật toán nhân ma trận nhanh hơn $O(n^3)$ thường khó cài và hằng số lớn. Vì vậy, nếu một bài toán không yếu hơn nhân ma trận kích thước n , ta thường xem lời giải cỡ $O(n^{1.5})$ sau các phép chia khối là gần như tối ưu thực dụng, dù về lý thuyết có thể còn thuật toán dựa trên n^ω .

2. Quy giảm bên trong nhân ma trận

2.1. Từ ma trận 01 tới ma trận nguyên

Một số nguyên không âm có thể viết

$$a = \sum_i a_i 2^i, \quad a_i \in \{0, 1\}.$$

Khi nhân hai số, ta có

$$ab = \sum_{i,j} 2^{i+j} a_i b_j.$$

Vì vậy nhân ma trận nguyên với giá trị không quá 2^k có thể tách thành k^2 lần nhân ma trận 01. Chiều ngược lại hiển nhiên vì ma trận 01 là trường hợp riêng của ma trận nguyên. Khi k là hằng số, hai bài toán có thể xem là quy giảm hai chiều.

2.2. Từ ma trận bool tới ma trận 01

Nhân ma trận 01 mạnh hơn nhân ma trận bool: nếu kết quả số học $(A \times B)_{i,j} > 0$, thì kết quả bool tại (i,j) bằng 1. Chiều ngược lại không đúng trực tiếp, nên không thể dùng nó để nói nhân ma trận bool khó hơn nhân ma trận 01; tuy nhiên theo hiểu biết của tác giả, độ phức tạp tốt nhất đã biết của nhân ma trận bool cũng là $O(n^\omega)$.

3. Một số kỹ thuật quy giảm về nhân ma trận

3.1. Bài toán tĩnh: dựng giao thông tin tập hợp

Giả sử có n tập S_i và k tập T_j , đều là tập con của $\{1, \dots, m\}$. Ta muốn biết $|S_i \cap T_j|$ cho mọi i, j . Dựng

$$A_{i,t} = [t \in S_i], \quad B_{t,j} = [t \in T_j].$$

Khi đó $(A \times B)_{i,j} = |S_i \cap T_j|$. Với $N = \max(n, m, k)$, bài toán tương đương nhân ma trận 01 kích thước N .

Mô hình này xuất hiện trong nhiều truy vấn đoạn. Nếu mỗi khối độ dài n có hai loại thông tin độ dài n , và đóng góp giữa hai khối là

$$C_{i,j} = \sum_k A_{i,k} B_{k,j},$$

thì muốn biết mọi đóng góp giữa các khối chính là phải biết tích $A \times B$. Do đó các bài toán truy vấn đoạn mà thông tin hai nửa sau tiền xử lý vẫn cần hợp nhất với giá thấp nhất $\Omega(n)$ thường có cận dưới liên quan tới nhân ma trận.

Ví dụ, trong *Count on a tree II*, cần hỏi số màu khác nhau trên đường đi (x, y) . Dựng cây dạng một gốc nối nhiều dây chuyền đại diện cho các tập S_i, T_j ; mỗi truy vấn đường qua hai dây và gốc cho thông tin giao giữa hai tập. Khi chọn các tham số cùng bậc, nhân ma trận 01 kích thước $\sqrt{N}$ quy giảm về bài toán kích thước N , nên cận dưới tương ứng là $O(N^{\omega/2})$.

Với bài *Little Z's Socks*, truy vấn số cặp bằng nhau trong đoạn có tính trừ được. Chia dãy thành khối; đóng góp giữa khối x và y được rút từ

$$F(x, y) - F(x, y - 1) - F(x + 1, y) + F(x + 1, y - 1).$$

Như vậy đóng góp giao giữa hai tập cũng được lấy từ các truy vấn đoạn, dẫn đến cận dưới tương tự. Với *Yuno loves sqrt technology III*, thông tin mode đoạn có thể mã hóa thành kiểm tra giao rỗng, tức nhân ma trận bool.

3.2. Bài toán động: dựng thông tin của tập con tùy ý

Với bài toán có cập nhật, có thể chia dãy thành n khối, xây thông tin loại $A_{j,i}$ cho khối i , rồi dựng n tập truy vấn B_j gồm một số khối. Nếu truy vấn hỏi tổng thông tin trên một tập khối, đáp án có dạng

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}.$$

Do đó nhân ma trận quy giảm về bài toán.

Trong bài “cộng đoạn và đếm số không toàn cục”, mỗi khối biểu diễn một tập con $c_i \subseteq \{1, \dots, n\}$. Với mỗi nhóm truy vấn, ta tạm cộng $+\infty$ vào các khối không thuộc tập con đang xét, rồi nhiều lần giảm toàn cục 1 và hỏi số phần tử bằng 0. Nhờ vậy ta khôi phục được số lần xuất hiện của từng giá trị trong tập khối. Tổng số thao tác chỉ $O(n^2)$, nhưng thông tin thu được là tích ma trận $A \times B$.

Bài “cộng hàng, hỏi tổng cột” trên tập điểm thưa trong mặt phẳng cũng tương tự. Mỗi hàng là một tập cột; một nhóm thao tác cộng trên một tập hàng rồi hỏi từng cột sẽ cho tích giữa ma trận hàng–cột và ma trận tập truy vấn–hàng.

4. Quy giảm giữa các bài toán khác

4.1. Từ nghịch thế hình chữ nhật tới nghịch thế đoạn

Bài *Thời đại nước mắt* hỏi số cặp

$$l \leq x < y \leq r, \quad u \leq a_x \leq a_y \leq d$$

trong một hoán vị. Quy giảm nghịch thế đoạn về bài toán hình chữ nhật là dễ; chiều ngược lại dùng chia để trị trên cây đoạn hai chiều.

Ta giữ các hình chữ nhật có chứa điểm; số hình chữ nhật và tổng số điểm trong chúng chỉ $O(n \text{ polylog } n)$. Một truy vấn hình chữ nhật được phân thành $O(\log^2 n)$ hình chữ nhật con. Các cặp điểm được chia thành: cùng hình chữ nhật, khác hình nhưng chung khoảng theo một chiều, hoặc khác cả hai chiều. Loại cùng

hình và khác cả hai chiều tính trực tiếp; hai loại còn lại dùng bao hàm–loại trừ qua từng hàng/cột của cây đoạn hai chiều, cuối cùng quy về nhiều truy vấn nghịch thế đoạn. Nếu phân tích kỹ, gọi nhân ma trận ở mỗi nút cho truy hồi $T(n) = O(n^\omega) + 2T(n/2) = O(n^\omega)$, không cần ẩn thêm polylog.

4.2. Quy giảm hai chiều cho bài toán truy vấn đoạn

Bài viết định nghĩa **đa thức nửa mặt phẳng không trực giao** có dạng

$$F(x, y) = \sum_i P_i(x, y) [A_i x + B_i y \geq 0],$$

với bậc và số hạng không quá polylog. Bài toán truy vấn đoạn hỏi tổng $F(a_x, a_y)$ trên mọi cặp $l \leq x < y \leq r$. Bài toán hai đoạn hỏi tổng tương tự với x nằm trong đoạn thứ nhất và y nằm trong đoạn thứ hai.

Hai bổ đề kỹ thuật, lấy từ tài liệu tham khảo, nói rằng những hàm dạng trên có thể biểu diễn bằng một số polylog các bài toán đếm cặp bằng nhau $[g_i(x) = h_i(y)]$, và chiều ngược lại cũng đúng. Từ đó suy ra bài toán truy vấn đoạn, bài toán hai đoạn và bài toán hai đoạn đếm cặp bằng nhau đều quy giảm hai chiều với nhau, bỏ qua nhân tử polylog. Những hàm thường gặp như $[x = y]$, $[x > y]$, $\max(x, y)$, $|x - y|$ đều thuộc mô hình này.

4.3. Đếm tam giác và cặp bằng nhau hai đoạn

Bài toán đếm tam giác theo cạnh: với mỗi cạnh (v, w) , hỏi có bao nhiêu đỉnh u kề với cả v và w . Với mỗi đỉnh i , ghi dãy các đỉnh kề E_i ; ghép các dãy lại. Giao giữa E_u và E_v chính là một truy vấn cặp bằng nhau giữa hai đoạn, nên đếm tam giác quy giảm về bài toán hai đoạn.

Chiều ngược lại dựng đồ thị ba phía có trọng số. Hai phía V, W biểu diễn các đoạn cây đoạn của dãy, phía U biểu diễn các giá trị. Cạnh từ giá trị sang đoạn có trọng số là số lần giá trị xuất hiện trong đoạn; cạnh giữa hai đoạn được đặt khi cặp đoạn đó cần truy vấn. Đếm tam giác có trọng số trong đồ thị này cho ra tổng cặp bằng nhau. Tách trọng số bằng biểu diễn nhị phân, bài toán quy về đếm tam giác thường. Kết luận: đếm tam giác và cặp bằng nhau hai đoạn quy giảm hai chiều với nhau, bỏ qua polylog, và đều nhận được cận liên quan tới nhân ma trận 01.

5. Dùng nhân ma trận cải thiện độ phức tạp

Quy giảm không chỉ cho cận dưới; nó cũng gợi cách dùng nhân ma trận làm cận trên. Với các bài vốn có lời giải $O(n^{1.5})$, ta thường chia dãy thành B khối, chia giá trị theo ngưỡng, dùng một lần nhân ma trận kích thước B , còn mỗi truy vấn xử lý phần lẻ trong $O(N/B)$.

5.1. Nghịch thế đoạn

Trong *Yuno loves sqrt technology II*, cần hỏi số cặp $x < y$ trong đoạn sao cho $a_x > a_y$. Chia dãy thành B khối, mọi truy vấn tách thành phần lẻ và các khối nguyên. Đóng góp trong một khối tiền xử lý trực tiếp. Với đóng góp giữa hai khối nguyên, lại chia giá trị thành B khối. Gọi $F_{x,y}$ là số phần tử ở khối dãy x nằm trong khối giá trị y , còn $G_{y,z}$ là số phần tử ở khối dãy z nằm trong các khối giá trị nhỏ hơn y . Khi đó $(F \times G)_{x,z}$ chính là đóng góp nghịch thế từ khối x sang khối z . Cân bằng chi phí cho độ phức tạp

$$O(B^\omega + N^2/B^2) = O(N^{2\omega/(\omega+1)}).$$

Vì nghịch thế hình chữ nhật và nghịch thế đoạn quy giảm hai chiều, kết quả này cũng áp dụng cho bài toán hình chữ nhật và các bài truy vấn đoạn tương đương.

5.2. Đếm tổng số tam giác

Với đồ thị N đỉnh, M cạnh, cần đếm tổng số tam giác. Lấy B đỉnh có bậc lớn nhất, dựng ma trận kề trên chúng; tổng các phần tử của G^2 theo cạnh cho số tam giác toàn nằm trong tập đỉnh lớn, tốn $O(B^\omega)$. Các đỉnh còn lại có bậc không quá $O(M/B)$, nên có thể liệt kê hai cạnh kề trong $O(M^2/B)$. Cân bằng cho thời gian $O(M^{2\omega/(\omega+1)})$, không có nhân tử polylog.

5.3. Cộng hàng, hồi tổng cột

Giữ lại B hàng và B cột có nhiều điểm nhất, rồi rạc hóa thành ma trận $B \times B$. Các hàng/cột còn lại ít điểm nên xử lý thẳng trong $O(N^2/B^2)$. Chia thao tác thành B khối, với phần khối nguyên dựng ma trận $A_{i,j} = [(i,j) \in V]$ và $B_{i,j}$ là số lần trong khối thao tác có cập nhật hàng i . Tích $A \times B$, kèm tiền tố theo cột, cho mọi đóng góp khối nguyên trong $O(B^\omega)$. Tổng thời gian cân bằng cũng là $O(N^{2\omega/(\omega+1)})$.

5.4. Số màu trên đường cây

Bài *Count on a tree II* cũng có thể được tăng tốc bằng phân cụm cây. Dùng khái niệm **cluster**: một đồ thị con liên thông của cây với tối đa hai đỉnh biên. Top Tree hoặc một thuật toán phân khối đơn giản chia cây thành $O(B)$ cluster, mỗi cluster kích thước $O(N/B)$, và mọi đường đi tách thành $O(N/B)$ điểm lẻ cộng với một đường giữa hai đỉnh biên.

Nếu biết số màu trên mọi đường giữa hai đỉnh biên, phần lẻ xử lý trực tiếp. Với các màu xuất hiện nhiều, dùng phân trị cạnh theo số đỉnh biên và nhân ma trận để tính đóng góp của B màu lớn trên mọi cặp biên. Với các màu ít, dựng cây ảo trên các vị trí của từng màu, rồi dùng tiền tố hai chiều theo thứ tự DFS của các đỉnh biên để cộng đóng góp. Tổng thời gian đạt

$$O(B^\omega + N^2/B^2 + N \log N) = O(N^{2\omega/(\omega+1)}).$$

6. Tổng kết

Bài viết giới thiệu khái niệm quy giảm, các kỹ thuật quy giảm từ nhân ma trận đến một số bài toán dữ liệu, các quy giảm giữa truy vấn đoạn, đếm cặp bằng nhau và đếm tam giác, cũng như một số cách dùng nhân ma trận để cải thiện độ phức tạp của bài toán cổ điển. Vẫn còn nhiều bài chưa rõ có tương đương với nhân ma trận hay không, và nhiều bài chưa đạt được thuật toán tốt hơn $O(n^{1.5})$; đây là hướng đáng tiếp tục khảo sát.

7. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Zhang Yibin, Peng Sijin và Yang Yaoliang đã hướng dẫn. Xin cảm ơn các thầy của Trường Học Quân như Xu Xianyou đã quan tâm và chỉ bảo. Xin cảm ơn Li Xinlong, Zhou Kangyang và Shi Qingyu đã góp ý cho bài viết. Xin cảm ơn gia đình, các bạn và anh chị trong đội tin học đã đồng hành, giúp đỡ.

Tài liệu tham khảo

1. Przemyslaw Uznanski, Karim Labib, and Daniel Wolleb-Graf. *Hamming distance completeness*. Annual Symposium on Combinatorial Pattern Matching, 2019.
2. Lech Duraj, Krzysztof Kleiner, Adam Polak, and Virginia Vassilevska Williams. *Equivalences between triangle and range query problems*. Symposium on Discrete Algorithms, 2020.

3. Renato F. Werneck and Robert E. Tarjan. *Self-adjusting top trees*. Symposium on Discrete Algorithms, 2005.

Bài 9. Cách làm một số bài toán cổ điển modulo lũy thừa của số nguyên tố nhỏ

Wu Lin, Trường Thập Nhất Bắc Kinh

Tóm tắt

Các bài toán đếm modulo lũy thừa của số nguyên tố nhỏ chiếm một vị trí quan trọng trong số học thuật toán. Bài viết này tổng kết một số phương pháp kinh điển cho các bài toán như vậy, đặc biệt là các thủ pháp dùng hàm sinh, thao tác giá trị của đa thức và tính chất số học của modulo p^k . Trọng tâm không phải là liệt kê mã mẫu, mà là giải thích vì sao các tính chất của p^k giúp giảm số lượng giá trị hoặc hệ số cần tính.

1. Quy ước

Trong bài, modulo đặc biệt luôn có dạng p^k , với p là số nguyên tố, $k \in \mathbb{Z}_{>0}$, và p tương đối nhỏ. Ký hiệu $v_p(n)$ là số mũ lớn nhất sao cho $p^{v_p(n)} \mid n$. Ký hiệu $f_p(n)$ là phần còn lại của $n!$ sau khi bỏ hết các thừa số p .

2. Thao tác giá trị của đa thức

Nhiều bài modulo p^k cuối cùng cần tính giá trị của một đa thức tại một số điểm. Phần này nhắc lại nội suy và đa điểm trong bối cảnh modulo có ước nhỏ.

Bổ đề 1 (công thức Legendre).

$$v_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor.$$

Bổ đề 2. Với mọi số nguyên n và số tự nhiên k , ta có $k! \mid n^k$.

Để chứng minh, chỉ cần xét từng số nguyên tố p . Tương tự công thức Legendre,

$$v_p(n^k) = \sum_{i=1}^{\infty} \left(\left\lfloor \frac{n}{p^i} \right\rfloor - \left\lfloor \frac{n-k}{p^i} \right\rfloor \right).$$

Do đó

$$v_p(n^k) - v_p(k!) = \sum_{i=1}^{\infty} \left(\left\lfloor \frac{n}{p^i} \right\rfloor - \left\lfloor \frac{n-k}{p^i} \right\rfloor - \left\lfloor \frac{k}{p^i} \right\rfloor \right),$$

mà mỗi hạng có dạng $\lfloor a+b \rfloor - \lfloor a \rfloor - \lfloor b \rfloor \geq 0$. Vì vậy kết quả không âm.

2.1. Nội suy đa thức

Bài toán nội suy: cho các điểm $(x_1, y_1), \dots, (x_n, y_n)$ của một đa thức $F(x)$ có $\deg F < n$, với các x_i đôi một khác nhau, cần khôi phục F . Trên trường số thực luôn có nghiệm duy nhất, nhưng modulo p^k thì không phải lúc nào cũng khôi phục được toàn bộ đa thức. Tuy vậy, nếu các điểm cho trước là một đoạn số nguyên liên tiếp, ta vẫn xác định được giá trị tại mọi điểm nguyên.

Định lý 1. Biết các giá trị nguyên của $F(x)$, $\deg F < n$, tại $1, 2, \dots, n$ theo modulo p^k , thì với mọi số nguyên x_0 , giá trị $F(x_0) \bmod p^k$ được xác định duy nhất.

Theo công thức nội suy Lagrange,

$$F(x_0) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x_0 - j}{i - j} = \sum_{i=1}^n (-1)^{n-i} y_i \frac{(x_0 - 1)^{i-1}}{(i-1)!} \frac{(x_0 - i - 1)^{n-i}}{(n-i)!}.$$

Hai thương trong biểu thức cuối là số nguyên theo bổ đề 2, nên $F(x_0)$ là tổ hợp tuyến tính nguyên của các y_i . Vì thế chỉ cần biết $y_i \bmod p^k$. Lập luận này cũng áp dụng cho các đa thức hữu tỉ lấy giá trị nguyên, chẳng hạn $x(x+1)/2$.

Ví dụ 1. Tự nhiên. Cho n, m , cần tính

$$\sum_{0 \leq i < n} i^m \pmod{2^k},$$

trong đó đề gốc có $k = 64$. Tổng này là đa thức theo n bậc $m+1$. Khi $m < k$, nội suy trực tiếp với $O(k)$ điểm và lũy thừa nhanh cho độ phức tạp $O(k \log m)$.

Khi $m \geq k$, mọi hạng chẵn đều bằng 0 modulo 2^k . Đặt $t = \lfloor (n-1)/2 \rfloor$, phần hạng lẻ là

$$\sum_{0 \leq i \leq t} (2i+1)^m = \sum_{0 \leq i \leq t} \sum_{0 \leq j < k} \binom{m}{j} (2i)^j,$$

vì các hạng $j \geq k$ triệt tiêu. Do đó đáp án vẫn biểu diễn được bằng một đa thức bậc k theo t .

2.2. Đa điểm của đa thức

Với một đa thức nguyên $F(x)$, bổ đề sau làm giảm mạnh miền giá trị cần xét.

Bổ đề 3. Với mọi số nguyên x , ta có $x^{p^k} \equiv 0 \pmod{p^k}$.

Thật vậy, theo bổ đề 2, $(p^k)! \mid x^{p^k}$, còn $\nu_p((p^k)!) \geq k$. Vì thế, khi tính giá trị của một đa thức hệ số nguyên, ta có thể trước hết rút gọn theo $x \bmod p^k$, hoặc tiền xử lý các giá trị tại $0, 1, \dots, p^k - 1$ rồi dùng nội suy liên tiếp để trả lời mọi điểm hỏi.

Ví dụ 2. Bears and Juice. Trong lời giải bài này xuất hiện tổng

$$\sum_{j=0}^{\min(m, n-1)} \binom{n}{j} i^j \pmod{2^k}$$

cho mọi $1 \leq i \leq q$. Nếu làm thẳng sẽ tốn ít nhất $O(mq)$. Áp dụng cách trên, tính thô vài giá trị đầu rồi nội suy các giá trị còn lại, tổng thời gian giảm còn $O((m+q)k)$.

2.3. Tính tổ hợp

Bài toán là tính $\binom{n}{m} \bmod p^k$. Chỉ cần tách

$$n! = p^a b, \quad (b, p) = 1.$$

Phần $a = \nu_p(n!)$ tính được bằng công thức Legendre trong $O(\log_p n)$; phần khó là $b = f_p(n)$.

Ta có

$$f_p(n) = f_p(\lfloor n/p \rfloor) \prod_{\substack{1 \leq i \leq n \\ p \nmid i}} i.$$

Đặt $n = mp + v$. Phần đuôi từ $mp + 1$ tới $mp + v$ có thể nhân trực tiếp trong $O(p)$. Với phần nguyên khối, xét

$$F_m(x) = \prod_{0 \leq i < m} \prod_{1 \leq j < p} \left(1 + \frac{i}{j}x\right),$$

trong đó $1/j$ là nghịch đảo của j modulo p^k . Tích khối bằng $(p-1)!^m F_m(p)$. Vì cần $F_m(p) \bmod p^k$, chỉ phải quan tâm tới các hệ số bậc 0 đến $k-1$.

Đặt $G_m(x) = \ln F_m(x)$. Khi khai triển logarit,

$$G_m(x) = \sum_{t \geq 1} \frac{(-1)^{t-1}}{t} \left(\sum_{0 \leq i < m} i^t \right) \left(\sum_{1 \leq j < p} j^{-t} \right) x^t.$$

Hệ số $[x^t]G_m(x)$ là đa thức theo m bậc $t+1$. Từ

$$F'_m(x) = G'_m(x)F_m(x)$$

suy ra đệ quy

$$[x^t]F_m(x) = \frac{1}{t} \sum_{0 \leq i < t} (t-i)[x^i]F_m(x)[x^{t-i}]G_m(x).$$

Do $[x^0]F_m(x) = 1$, quy nạp cho thấy $[x^t]F_m(x)$ là đa thức theo m bậc không quá $2t$. Vì vậy $F_m(p) \bmod p^k$ có thể lấy bằng nội suy liên tiếp từ $F_0(p), F_1(p), \dots, F_{2k-2}(p)$, các giá trị này tiền xử lý thô trong $O(pk)$. Mỗi truy vấn tính $f_p(n)/f_p(\lfloor n/p \rfloor)$ tốn $O(p+k+\log n)$, rồi đệ quy cho tổng thời gian $O((p+k)\log_p n + pk + \log n)$.

3. Tìm hệ số của đa thức

Nhiều bài đếm quy về tìm một hệ số xa của một đa thức. Khi modulo là p^k , các đồng dư kiểu Frobenius cho phép chia nhỏ số mũ lớn.

3.1. Hệ số xa của lũy thừa cao của đa thức ngắn

Bài toán: cho đa thức

$$F(x) = \sum_{i=0}^n a_i x^i,$$

với n nhỏ nhưng m, t rất lớn, cần tính $[x^t]F^m(x) \bmod p^k$.

Trường hợp $k = 1$. Với số nguyên tố p và đa thức nguyên F , ta có

$$F^p(x) \equiv F(x^p) \pmod{p}.$$

Đây là hệ quả của định lý Fermat nhỏ và việc các hệ số đa thức trong khai triển p -lần đều chia hết cho p , trừ các hạng thuần.

Xét bài tổng quát hơn $[x^t]F^m(x)G(x) \bmod p$. Viết

$$m = a_1 p + b_1, \quad t = a_2 p + b_2.$$

Ta có

$$[x^t]F^m(x)G(x) = [x^t]F^{a_1}(x^p)F^{b_1}(x)G(x).$$

Tách $F^{b_1}(x)G(x)$ theo phần dư của bậc modulo p :

$$F^{b_1}(x)G(x) = \sum_{0 \leq r < p} x^r H_r(x^p).$$

Khi đó

$$[x^t]F^m(x)G(x) = [x^{a_2}]F^{a_1}(x)H_{b_2}(x).$$

Như vậy m, t cùng chia cho p sau một bước, còn bậc của đa thức phụ không tăng. Tiềm xử lý các lũy thừa $0, \dots, p-1$ của F cho độ phức tạp tiềm xử lý $O(n^2 p)$, và truy vấn $O(n^2 \log_p m)$.

Trường hợp $k > 1$. Tương tự có đồng dư

$$F^{p^k}(x) \equiv F^{p^{k-1}}(x^p) \pmod{p^k}.$$

Chứng minh bằng quy nạp theo k : giả sử

$$F^{p^{k-1}}(x) \equiv F^{p^{k-2}}(x^p) + p^{k-1}G(x) \pmod{p^k},$$

nâng lên lũy thừa p thì mọi hạng có chứa G đều có đủ số mũ p để triệt tiêu modulo p^k .

Đặt

$$F_0(x) = F^{p^{k-1}}(x), \quad G_0(x) = F^{m \bmod p^{k-1}}(x)G(x), \quad m' = \left\lfloor \frac{m}{p^{k-1}} \right\rfloor.$$

Khi đó bài toán chuyển thành $[x^t]F_0^{m'}(x)G_0(x)$. Viết tiếp $m' = a_1 p + b_1$, $t = a_2 p + b_2$, rồi áp dụng đồng dư trên:

$$[x^t]F_0^{m'}(x)G_0(x) = [x^{a_1}]F_0^{a_2}(x)H_{b_2}(x).$$

Phân tích độ phức tạp giống trường hợp $k = 1$, nhưng bậc hiệu dụng tăng lên khoảng np^{k-1} , nên đạt tiềm xử lý $O(n^2 p^{2k})$ và truy vấn $O(n^2 p^{2k-2} \log_p m)$.

3.2. Hệ số xa của nghịch đảo hợp thành

Cho đa thức

$$F(x) = \sum_{1 \leq i \leq n} f_i x^i, \quad f_1 \neq 0,$$

và G thỏa $F(G(x)) = x$, cần tính $[x^t]G(x) \bmod p^k$ với n nhỏ, t lớn. Công thức đảo Lagrange quen thuộc có dạng

$$n[x^n]G^m(x) = m[x^{-m}]F^{-n}(x),$$

nhưng khi làm modulo p^k , phép chia cho n có thể không hợp lệ. Dùng dạng thay thế:

$$[x^n]G^m(x) = [x^{-m-1}]F'(x)F^{-n-1}(x) = [x^{n-m}]F'(x) \left(\frac{F(x)}{x} \right)^{-n-1}.$$

Biểu thức này quy về bài hệ số xa của lũy thừa cao ở trên. Điểm cần chú ý là số mũ có thể âm, nên trạng thái kết thúc của đệ quy có dạng $[x^0]A^{-1}(x)B(x)$.

3.3. Nhị thức chập

Với hai dãy $a_0, \dots, a_n$ và $b_0, \dots, b_n$, bài toán nhị thức chập yêu cầu tính

$$c_i = \sum_{0 \leq j \leq i} \binom{i}{j} a_j b_{i-j}.$$

Thông thường ta đưa về tích chập thường bằng EGF, tức chia cho $i!$. Khi $p \leq n$, các nghịch đảo này không luôn tồn tại modulo p^k .

Ta tách

$$\binom{i}{j} = p^{\nu_p(i!) - \nu_p(j!) - \nu_p((i-j)!)} \frac{f_p(i)}{f_p(j)f_p(i-j)}.$$

Đặt $\hat{a}_i = a_i / f_p(i) \pmod{p^k}$. Theo định lý Kummer,

$$\nu_p(i!) - \nu_p(j!) - \nu_p((i-j)!)$$

bằng số lần mượn khi trừ j khỏi i trong hệ cơ số p , nên không vượt quá $\lfloor \log_p n \rfloor$. Vì thế có thể làm các tích chập theo từng số mũ của p , tính dưới vài modulo NTT lớn rồi ghép CRT về đáp án trước khi rút modulo p^k . Do p khả nghịch trong các modulo NTT lớn, phần tích chập trở lại là tích chập thường, đạt $O(n \log n)$.

3.4. Hàm mũ của đa thức

Bài toán: với $F(0) = 0$, tính $G(x) = \exp F(x)$ đến bậc n . Cách chuẩn dùng Newton cho phương trình

$$H(G) = \ln G - F = 0.$$

Giả sử đã có $G_0(x) \equiv G(x) \pmod{x^m}$, khai triển Taylor quanh G_0 cho

$$G(x) \equiv G_0(x)(1 + F(x) - \ln G_0(x)) \pmod{x^{2m}}.$$

Vấn đề trong modulo có ước nhỏ là phép tính $\ln G_0$ thường cần tích phân, tức chia hệ số cho chỉ số. Thay vì duy trì trực tiếp $[x^i]G(x)$, ta duy trì $G^{(i)}(0) = i![x^i]G(x)$. Khi đó đạo hàm và tích phân chuyển thành thao tác dịch chỉ số, còn tích chập thường chuyển thành nhị thức chập ở mục trước. Nhờ vậy vẫn có thể đạt độ phức tạp $O(n \log n)$.

4. Tổng kết

Bài viết khảo sát các bài toán đếm modulo lũy thừa của số nguyên tố nhỏ, rút ra hai nhóm công cụ chính: dùng tính chất số học của p^k để giảm miền giá trị hoặc số hệ số cần xét, và dùng hàm sinh/đa thức để chuyển bài đếm thành nội suy, hệ số xa, nhị thức chập hoặc hàm mũ đa thức. Phần tính tổ hợp modulo p^k cũng đưa ra một cách làm dựa trên nội suy của các hệ số trong tích khối.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang, Peng Sijin và Zhang Yibin đã hướng dẫn. Xin cảm ơn cha mẹ đã nuôi dưỡng và giáo dục trong nhiều năm. Xin cảm ơn thầy Wang Xingming đã bồi dưỡng và chỉ bảo. Xin cảm ơn các anh Wu Chang, Dong Yurun, Liu Ziyuan đã góp ý quý báu cho bài viết. Xin cảm ơn Zhao Haikun, Zhu Pengrui đã cùng thảo luận, và cảm ơn các thầy cô, bạn bè khác đã giúp đỡ.

Tài liệu tham khảo

1. Tang Jingzhe. *Tính tổ hợp theo modulo*. CSDN article 52553048, 2020.
2. 2023 Beijing Collegiate Programming Contest. <https://qoj.ac/contest/1464/problem/7992>.
3. Codeforces 643F, *Bears and Juice*. <https://codeforces.com/problemset/problem/643/F>.
4. LibreOJ NOI Round #2, *Simple Arithmetic*. <https://loj.ac/p/577>.
5. Leukocyte. Ghi chú về đồng dư $FP^k(x)$. <https://www.cnblogs.com/leukocyte/p/14552848.html>.
6. 2020–2021 bài tập đội tuyển, bài *Dai Jiaqiang he Duo Xiangmu*. <https://loj.ac/p/3398>.
7. Bài *Công thức mũ*. <https://loj.ac/p/6737>.

Bài 10. Bàn về một lớp phương pháp duy trì thông tin trên cấu trúc cây động dễ cài đặt

Xiao Daien, Trường Trường Quận

Tóm tắt

Duy trì thông tin trên cấu trúc cây động là một nhóm bài toán dữ liệu quan trọng. Nhóm bài toán này có hai hướng giải tự nhiên với độ phức tạp logarithm: lời giải amortized dùng splay để duy trì phân rã chuỗi đặc, và lời giải có bảo đảm chặt hơn dùng cây cân bằng theo trọng lượng để duy trì phân rã chuỗi nặng tuyệt đối động. Bài viết giới thiệu hai cách làm này, cách cài đặt và cách nhìn thể năng dùng để phân tích độ phức tạp.

1. Mở đầu

Bài toán duy trì thông tin trên cây động yêu cầu thao tác trên một rừng có thể thêm hoặc xóa cạnh. Trong khi hình dạng cây thay đổi, ta vẫn phải sửa thông tin trên một thành phần liên thông hoặc trên một đường đi, đồng thời truy vấn tổng thông tin tương ứng. Các lời giải lý thuyết với độ phức tạp logarithm theo trường hợp xấu đã có từ lâu, nhưng thường khó cài và hằng số lớn. Bài viết xuất phát từ các cấu trúc quen thuộc hơn để đưa ra hai phương án cài đặt thực dụng.

2. Quy ước

Giả sử người đọc đã quen với splay, cây cân bằng theo trọng lượng và phân rã chuỗi nặng. Bài viết dùng cây có gốc khi cần nói tới cha, con và cây con.

3. Định nghĩa cơ bản

Cây là đồ thị vô hướng liên thông, đơn và không có chu trình; rừng là hợp của một hoặc nhiều cây. Với cây có gốc, cha của một đỉnh là đỉnh đầu tiên trên đường từ đỉnh đó tới gốc; các đỉnh có cha là u gọi là con của u .

Con nặng tuyệt đối của u là một con có kích thước cây con lớn hơn một nửa kích thước cây con của u . Mỗi đỉnh có nhiều nhất một con như vậy. Các con còn lại là con nhẹ tuyệt đối. Cạnh nối tới con nặng gọi là cạnh nặng; cạnh nối tới con nhẹ gọi là cạnh nhẹ.

Bài viết mô hình hóa dữ liệu bằng một cấu trúc hai nửa nhóm. Có hai kiểu dữ liệu D, O . Kiểu O tác động lên D , kiểu O có phép hợp thành, còn kiểu D có phép cộng giao hoán:

$$\times : O \times D \rightarrow D, \quad \times : O \times O \rightarrow O, \quad + : D \times D \rightarrow D.$$

Các phép toán có phần tử đơn vị ϵ_D, ϵ_O , phép $+$ trên D giao hoán và kết hợp, phép $\times$ trên O kết hợp, và tác động của O phân phối qua phép cộng trên D . Mô hình này đủ để biểu diễn các thao tác gán/cộng lazy và truy vấn tổng thường gặp.

4. Bài toán

Cần duy trì một rừng; mỗi đỉnh x có thông tin d_x . Các thao tác gồm:

- thêm hoặc xóa một cạnh, luôn bảo đảm đồ thị còn là rừng;
- với $o \in O$ và một thành phần liên thông cực đại hoặc một đường đi S , gán $d_x \leftarrow o \times d_x$ cho mọi $x \in S$;
- với một thành phần hoặc đường đi S , truy vấn $\sum_{x \in S} d_x$.

5. Phân tích thể năng của splay có trọng số

Thể năng là công cụ chuẩn để phân tích độ phức tạp amortized. Một thao tác được xem là có chi phí bằng số bước thực hiện cộng với phần tăng thể năng; nếu tổng chi phí này bị chặn, ta thu được độ phức tạp amortized.

Đặt

$$F(k) = \lceil \log_2 k \rceil.$$

Một splay có trọng số là splay trong đó mỗi nút mang thêm một trọng số. Với mỗi nút x , size_x là tổng kích thước cây con splay của x cộng với trọng số phụ của các nút trong cây con đó. Nút “con giữa” trong mô tả gốc chỉ biểu diễn trọng số phụ này, không phải một con thật trong splay.

Thể năng của x là

$$h_x = F(\text{size}_x),$$

và thể năng toàn cục là $\phi = \sum_x h_x$. Nếu sau thao tác thể năng chuyển từ ϕ_{old} sang ϕ_{new} , phần tăng thể năng là

$$\Delta\phi = \phi_{\text{new}} - \phi_{\text{old}}.$$

Với các phép xoay đôi *zig-zig*, *zag-zag*, *zig-zag* và *zag-zig*, phân tích trong bài gốc xét các khối cây con bị hoán đổi. Các khối đối xứng được gộp trọng số, nên chỉ cần quan tâm tổng kích thước của chúng. Trong mọi trường hợp, chi phí xoay cộng với tăng thể năng không vượt quá một hằng số nhân với độ tăng thể năng giữa cây lớn sau xoay và cây nhỏ trước xoay, cụ thể có thể chặn bởi dạng $3(F(n) - F(s))$. Phép xoay đơn cuối cùng trong một lần splay có thêm một hằng số 1. Do đó toàn bộ một lần splay vẫn được phân tích bằng thể năng của splay có trọng số.

6. Phân rã chuỗi đặc bằng splay

Ta phân rã một cây có gốc thành các chuỗi đặc. Mỗi đỉnh có nhiều nhất một con đặc; cạnh nối tới con đặc là cạnh đặc, còn các cạnh khác là cạnh ảo. Mỗi chuỗi đặc được duy trì bằng một splay: bên trái một nút là các đỉnh nông hơn trên chuỗi, bên phải là các đỉnh sâu hơn. Gốc của splay vẫn ghi cha ngoài splay, tức cha của đỉnh đầu chuỗi trong cây gốc.

Thao tác then chốt là $\text{splice}(x)$, biến đỉnh đầu chuỗi chứa x thành con đặc của cha nó. Cách làm:

1. splay cha của đỉnh đầu chuỗi;
2. ngắt con phải hiện tại của cha, tức con đặc cũ;
3. đặt gốc splay chứa x làm con phải mới.

Hai bước cuối có thể cài bằng một phép đổi con. Việc ngắt con đặc cũ chỉ làm giảm thể năng nên không gây vấn đề trong phân tích.

Thao tác $\text{access}(x)$ biến đường từ x tới gốc thành một chuỗi đặc. Ta ngắt con đặc hiện tại của x , rồi lặp $\text{splice}(x)$ cho tới khi x cùng chuỗi với gốc. Trong mỗi lần splice, các kích thước có trọng số trước và sau splay tăng theo dạng lồng nhau:

$$\Delta\phi = h(x_2) - h(x_1) + h(x_4) - h(x_3) + \dots + h(x_{2k}) - h(x_{2k-1}),$$

với $h(x_i) \leq h(x_{i+1})$. Vì vậy tổng chặn bởi $h(x_{2k}) - h(x_1) = O(\log n)$. Phần hằng số của số lần splice được xử lý bằng cách cấp thêm một đơn vị thể năng cho mỗi cạnh vừa là cạnh ảo vừa là cạnh nặng: nếu splice mở một cạnh nặng đang ảo, thể năng đó trả cho chi phí; nếu là cạnh nhẹ, số lần như vậy trên một đường chỉ là $O(\log n)$.

Từ đó, access có chi phí amortized $O(\log n)$. Các thao tác link, cut, đổi gốc, sửa đường và hỏi đường đều có thể xây trên access , nên đều có độ phức tạp amortized $O(\log n)$.

7. Duy trì thông tin cây con

Để duy trì thông tin của các con ảo, bài viết dùng **splay leafy tree (SLT)**, tức splay mà chỉ các lá đại diện dữ liệu thật. Với mỗi đỉnh, toàn bộ con ảo của nó được gom trong một SLT riêng; các nút ghép do SLT sinh ra vẫn mang thể năng $F(\text{size}_x)$. Khi đó một nút ngoài hai con trái/phải còn có một “con giữa” trở tới SLT của các con ảo.

Khi splice thay đổi con đặc, có hai trường hợp. Nếu cha đang có con đặc cũ, ta đưa con cũ về vị trí tương ứng trong SLT rồi splay cha trong SLT. Nếu cha không có con đặc cũ, ta xóa nút ghép chứa x , đưa anh em của nó lên thay, rồi splay ông của x trong SLT. Trong cả hai trường hợp, ảnh hưởng thể năng bị chặn bởi $O(h(\text{fa}_x) - h(x))$, nên cộng lại trên một access vẫn là $O(\log n)$. Việc ngắt con đặc ban đầu của access chỉ là chèn con đó vào “con giữa”, cũng có chi phí $O(\log n)$.

Cách duy trì thông tin là:

- trên mỗi nút thuộc splay chuỗi đặc, lưu d_1, d_2, o_1, o_2 , tương ứng với tổng trên chuỗi, tổng cây con ảo, lazy trên chuỗi và lazy trên cây con ảo;
- trên mỗi nút của SLT, lưu d_1, o_1 , tức tổng cây con và lazy của cây con đó.

Như vậy bài toán ở mục 4 được giải trong $O((n + q) \log n)$ thời gian amortized.

8. Cây lá cân bằng theo trọng lượng

Một phương án khác là dùng **weight-balanced leafy tree (WBLT)** để duy trì cả chuỗi đặc lẫn các con ảo. Gọi ls_x, rs_x là con trái và con phải của x . Khi gộp hai WBLT L, R , giả sử $\text{size}_L \leq \text{size}_R$ sau khi đổi vai nếu cần. Có ba trường hợp chính:

- nếu $\text{size}_R / (\text{size}_L + \text{size}_R) \leq \alpha$, hoặc R là lá, tạo nút mới nối hai cây;
- nếu $\text{size}_{\text{rs}_R} / (\text{size}_L + \text{size}_R) \geq 1 - \alpha$, hoặc ls_R là lá, đệ quy gộp L với ls_R , rồi trả về R ;
- ngược lại, đệ quy gộp $(L, \text{ls}_{\text{ls}_R})$ và $(\text{rs}_{\text{ls}_R}, \text{rs}_R)$, rồi nối lại.

Phân tích cân bằng cho thấy chi phí gộp là

$$O(\log \text{size}_R - \log \text{size}_L + 1),$$

và khi đi lên ba tầng thì kích thước tăng ít nhất một hệ số $1/\alpha$. Trong thực nghiệm, α thường lấy khoảng 0.65–0.85, tùy dữ liệu.

9. Phân rã chuỗi nặng tuyệt đối động

Có thể dùng WBLT thay splay và SLT trực tiếp. Khi chỉ cần tính chất cân bằng theo trọng lượng, cách này đã đủ cho một số bài. Để khống chế số lần splice một cách chặt, bài viết xét phân rã chuỗi nặng tuyệt đối động: sau mỗi thao tác, mọi cạnh nặng đều phải là cạnh đặc. Nếu một đỉnh không có con nặng thì cạnh đặc có thể là cạnh nhẹ hoặc không tồn tại.

Sau một lần access, việc sửa/truy vấn thông tin đã xong nhưng phân rã có thể không còn tuyệt đối nặng. Chỉ $O(1)$ chuỗi đặc bị ảnh hưởng. Với một đỉnh không nằm ở đáy chuỗi, nếu nó có con nặng nhưng con đó không phải con đặc hiện tại, ta ngắt cạnh đặc hiện tại, tìm con nặng trong cây “con giữa”, rồi ghép lại.

Trên WBLT của chuỗi đặc, ta nhị phân để tìm vị trí đầu tiên mà kích thước vượt một nửa; cạnh tại đó là cạnh nhẹ đặc đầu tiên từ trên xuống. Cắt WBLT tại đó, rồi nhìn vào WBLT của các con ảo của cha cạnh bị cắt. Chỉ cần quét $O(1)$ tầng đầu để kiểm tra có lá nào biểu diễn một con có kích thước vượt một nửa cây con của cha hay không. Nếu có, lá đó chính là con nặng; ta tách nó khỏi WBLT con ảo, rồi ghép với phần chuỗi cũ. Nếu không, chỉ cần ghép cây chứa con đặc nhẹ vào phần con ảo.

Gọi các cây liên quan là x, y, z , trong đó y là cây chứa con đặc nhẹ cũ, còn z là cây chứa con nặng mới. Chi phí ghép với chuỗi của con nặng có dạng

$$O(|\log \text{size}_x - \log \text{size}_z|).$$

Vì $\text{size}_y \leq \min(\text{size}_x, \text{size}_z)$, chi phí này bị chặn bởi

$$O(\log \max(\text{size}_x, \text{size}_z) - \log \text{size}_y).$$

Sau mỗi bước, kích thước mới của x, z không vượt quá kích thước cũ của y , nên cộng dồn giống phân tích access ở mục 6, tổng vẫn $O(\log n)$. Con nặng tại đáy chuỗi được xử lý tương tự bằng cách quét vài tầng đầu của “con giữa”, tách ra nếu tồn tại rồi ghép vào chuỗi.

Nhờ vậy, mỗi thao tác đạt thời gian chặt $O(\log n)$. Phần phân tích gộp cây con ảo phức tạp hơn nên bài gốc không triển khai hết chi tiết. Ưu điểm của cách này là vừa có tính cân bằng theo trọng lượng, vừa có thể duy trì thông tin dạng khối trên cây, ví dụ những bài tương tự *CTS2022 WBTT*, và cài đặt vẫn tương đối trực tiếp.

10. Tổng kết

Bài viết đưa ra hai cách giải logarithm cho bài toán duy trì thông tin trên cây động. Cách thứ nhất dùng phân rã chuỗi đặc và splay, cho độ phức tạp đúng theo nghĩa amortized. Cách thứ hai dùng phân rã chuỗi nặng tuyệt đối động với WBLT, cho bảo đảm chặt hơn nhưng cài đặt phức tạp và hằng số lớn hơn. Tác giả hy vọng các phương pháp này là điểm xuất phát để tìm ra những cách duy trì cây động đơn giản và hiệu quả hơn.

Ghi chú trích xuất

Bài này được dịch từ trang PDF 84–88, tương ứng các trang in 161–168. PDF được dàn trang hai trang nội dung trên một trang vật lý; trang PDF 84 chứa nửa trái là tài liệu tham khảo của bài trước và nửa phải

là phần đầu bài này, còn trang PDF 88 chứa nửa phải là phần đầu bài kế tiếp. Các hình minh họa phép xoay splay và cây phụ được diễn giải bằng lời, không để lại chú thích rời.

Bài 11. Bàn về một lớp bài toán lân cận trên đồ thị

Jiao Siyuan, Trường Trung học số 1 Tiêu Tác

Tóm tắt

Bài viết bắt đầu từ một bài toán kinh điển về duy trì thông tin của lân cận trên đồ thị, phân tích sâu hơn bản chất của lời giải, rồi mở rộng vấn đề theo một hướng khác. Thông qua các ví dụ, bài viết giới thiệu một số cách áp dụng góc nhìn này cho đồ thị thưa, đồ thị có mật độ lớn nhất nhỏ, và các bài toán truy vấn trên dãy lân cận.

1. Mở đầu và quy ước

Lý thuyết cấu trúc dữ liệu đã được nghiên cứu rất nhiều trên dãy và trên cây. Với một số đồ thị có cấu trúc đặc biệt như cactus hoặc đồ thị chuỗi–song song tổng quát, ta cũng thường chuyển chúng về cây rồi xử lý. Nhưng với đồ thị tổng quát, cấu trúc phức tạp khiến việc duy trì nhanh trở nên khó hơn. Bài viết lấy một bài toán cổ điển làm điểm xuất phát, rồi từ lời giải cho các trường hợp đặc biệt rút ra một cách nhìn sâu hơn.

Trong bài, đồ thị luôn là đồ thị không có khuyên và không có cạnh song song. Ký hiệu $G = (V, E)$ nghĩa là V là tập đỉnh và E là tập cạnh của G . Lân cận của đỉnh x là tập các đỉnh kề trực tiếp với x , ký hiệu $R(x)$. Bậc của x là $\deg x = |R(x)|$.

2. Bắt đầu từ một bài toán kinh điển

Ví dụ 1. Project Management. Cho một đồ thị vô hướng liên thông n đỉnh, m cạnh, có q thao tác:

- cộng v vào trọng số của đỉnh x ;
- hỏi tổng trọng số của các đỉnh kề với x .

Lời giải kinh điển là chia đỉnh theo bậc. Chọn ngưỡng B ; đỉnh có $\deg_i \leq B$ gọi là nhẹ, còn đỉnh có $\deg_i > B$ gọi là nặng. Vì tổng bậc là $2m$, số đỉnh nặng không quá $\lfloor 2m/B \rfloor$. Khi hỏi một đỉnh nhẹ, ta duyệt trực tiếp lân cận của nó, tốn $O(B)$. Với mỗi đỉnh nặng x , duy trì s_x là tổng trọng số lân cận của x ; khi sửa một đỉnh, duyệt các đỉnh nặng kề với nó để cập nhật các s_y , tốn $O(m/B)$. Lấy $B = \sqrt{2m}$ cho độ phức tạp $O(q\sqrt{m})$.

2.1. Một số đồ thị đặc biệt

Cách trên chỉ dùng tổng số cạnh, không tận dụng hình dạng đồ thị. Trên một số lớp đồ thị đặc biệt, có lời giải tuyến tính hơn.

Cây. Gốc hóa cây tùy ý. Gọi fa_x là cha của x , và với mỗi đỉnh duy trì s_x , tổng trọng số các con của x . Khi sửa trọng số của x , chỉ s_{fa_x} bị ảnh hưởng. Khi hỏi lân cận của x , lấy s_x cộng thêm trọng số của cha x . Tổng thời gian là $O(n + q)$.

Cactus. Một đồ thị cactus cạnh là đồ thị vô hướng liên thông trong đó mỗi cạnh nằm trên nhiều nhất một chu trình đơn. Cactus đỉnh yêu cầu mỗi đỉnh nằm trên nhiều nhất một chu trình đơn. Ta dùng Tarjan để co các chu trình đơn của cactus thành nút, thu được một cây. Phần lân cận khác chu trình của x xử lý như trên cây; các đỉnh cùng chu trình với x chỉ có nhiều nhất hai đỉnh cần xét trực tiếp. Độ phức tạp vẫn là $O(n + m + q)$.

Đồ thị chuỗi–song song tổng quát. Đồ thị chuỗi–song song tổng quát là đồ thị liên thông không chứa đồ thị con đồng phân với K_4 . Loại đồ thị này có thể thu nhỏ bằng ba phép:

- xóa đỉnh bậc 1;
- co đỉnh bậc 2: nếu x nối với y, z , xóa hai cạnh đó và thêm cạnh (y, z) ;
- gộp các cạnh song song nối cùng một cặp đỉnh thành một cạnh.

Một đồ thị chuỗi–song song tổng quát có thể bị co về một đỉnh bằng các phép trên. Quá trình co tạo ra một cấu trúc cây, nên áp dụng cùng tư tưởng sẽ xử lý được trong $O(n + m + q)$. Ba ví dụ đặc biệt này đều gợi ý rằng yếu tố quyết định không hẳn là m , mà là một đại lượng đo độ dày cục bộ của đồ thị.

3. Cách làm trên đồ thị thưa

3.1. Mật độ của đồ thị

Với đồ thị vô hướng liên thông $G = (V, E)$, định nghĩa mật độ

$$\rho(G) = \frac{|E|}{|V|}.$$

Một đồ thị con mật độ lớn nhất là đồ thị con $G' = (V', E')$ làm cực đại $\rho(G')$. Mật độ lớn nhất của G ký hiệu $\rho_{\max}(G)$.

Luôn tồn tại một đồ thị con mật độ lớn nhất liên thông và là đồ thị con cảm sinh bởi V' . Nếu chưa cảm sinh, thêm các cạnh còn thiếu chỉ làm tăng mật độ. Nếu nó không liên thông, thành phần có mật độ lớn nhất trong các thành phần cũng đạt mật độ không nhỏ hơn toàn bộ.

Với cây, mọi đồ thị con liên thông khác rỗng có mật độ $(|V'| - 1)/|V'| < 1$. Với cactus, số cạnh nhỏ hơn $2n$, và đồ thị con của cactus vẫn là cactus, nên $\rho_{\max} < 2$. Với đồ thị chuỗi–song song tổng quát, quy nạp theo các phép co ở trên cũng cho mật độ luôn nhỏ hơn 2.

3.2. Cách làm dựa trên mật độ lớn nhất

Bổ đề 3.2. Trong một đồ thị G có mật độ lớn nhất $\rho_{\max}(G)$, mọi đồ thị con đều có ít nhất một đỉnh bậc không quá $2\rho_{\max}(G)$.

Nếu tồn tại một đồ thị con $G' = (V', E')$ mà mọi đỉnh đều có bậc lớn hơn $2\rho_{\max}(G)$, tổng bậc của nó lớn hơn $2\rho_{\max}(G)|V'|$, nên $|E'| > \rho_{\max}(G)|V'|$, mâu thuẫn với định nghĩa mật độ lớn nhất.

Ký hiệu $d_{\min}(G)$ là giá trị lớn nhất, trên mọi đồ thị con khác rỗng, của bậc nhỏ nhất trong đồ thị con đó. Khi đó

$$\rho_{\max}(G) \leq d_{\min}(G) \leq 2\rho_{\max}(G).$$

Hơn nữa, $d_{\min}(G) \leq r$ khi và chỉ khi có thể xóa rỗng đồ thị bằng cách lặp lại thao tác xóa một đỉnh bậc không quá r .

Ví dụ 2. The Last Cumulonimbus Cloud. Cho một đồ thị thỏa $d_{\min}(G) \leq 10$, hỗ trợ hai thao tác: cộng vào trọng số một đỉnh, và hỏi tổng trọng số các đỉnh kề nó. Theo bổ đề trên, ta có thể xóa lần lượt các đỉnh bậc nhỏ để lấy thứ tự xóa. Định hướng mỗi cạnh từ đỉnh bị xóa sớm hơn sang đỉnh bị xóa muộn hơn; khi đó mỗi đỉnh có số cạnh đi ra không quá $d_{\min}(G)$.

Với mỗi đỉnh x , duy trì s_x , tổng trọng số các đỉnh nối tới x bằng cạnh đi vào x . Khi sửa x , ta sửa trọng số của x và cập nhật s_y cho các cạnh đi ra $x \rightarrow y$. Khi hỏi x , lấy s_x cộng trọng số của các đỉnh y mà $x \rightarrow y$. Mỗi thao tác tốn $O(d_{\min}(G))$, nên tổng thời gian là

$$O(n + m + q d_{\min}(G)) = O(n + m + q \rho_{\max}(G)).$$

3.3. Khi được xử lý offline

Bản chất lời giải trên là định hướng mỗi cạnh và chỉ duy trì thông tin từ một phía. Nếu được biết trước mọi thao tác, đặt c_x là tổng số lần đỉnh x tham gia thao tác loại sửa/hỏi. Một cạnh (x, y) nên được định hướng từ đầu có c nhỏ sang đầu có c lớn; tổng chi phí là

$$\sum_{(x,y) \in E} \min(c_x, c_y).$$

Xét trường hợp xấu nhất khi tổng c_x là q . Nếu tập đỉnh nhận trọng số là V_1 , thì ở tối ưu mọi đỉnh trong V_1 có cùng trọng số $q/|V_1|$. Nếu không, tách theo mức trọng số nhỏ nhất sẽ cho hai lớp; khi lớp sau có mật độ lớn hơn, dồn trọng số sang lớp sau không tệ hơn, còn khi lớp đầu có mật độ lớn hơn thì gộp hai mức thành một mức tốt hơn. Lặp lại lập luận này làm giảm số mức trọng số, nên cuối cùng các c_x trong V_1 bằng nhau.

Khi đó chi phí bằng

$$\frac{q}{|V_1|} |E_1| = q \rho(G_1),$$

nên G_1 phải là đồ thị con mật độ lớn nhất. Vì vậy cận trên offline vẫn là $O(n + m + q \rho_{\max}(G))$, nhưng hằng số nhỏ và thường khó bị chặn sát hơn trong thực tế.

3.4. Đồ thị dày và đồ thị có hướng

Với mọi đồ thị $G = (V, E)$, luôn có $d_{\min}(G) \leq \sqrt{2m}$. Do đó cách định hướng không tệ hơn lời giải căn bậc hai cổ điển. Tuy nhiên có thể dựng đồ thị đạt gần cận này, nên trên đồ thị dày nó không cho cải thiện tiệm cận.

Với đồ thị có hướng, nếu thao tác chỉ quan tâm tới lân cận theo cạnh vào hoặc cạnh ra, ta vẫn có thể coi cạnh là vô hướng trong bước định hướng phụ trợ, rồi khi duyệt một cạnh thì kiểm tra nó có đúng chiều mà thao tác yêu cầu hay không.

4. Một hướng mở rộng khác

4.1. Ví dụ

Ví dụ 3. Vũ Bái Phong Hoàng. Cho đồ thị vô hướng G có n đỉnh, m cạnh; mỗi đỉnh x có trọng số a_x . Hỗ trợ:

- với l, r, v , cộng v vào mọi a_x với $x \in R(i)$ cho một $i \in [l, r]$;
- với l, r , hỏi

$$\sum_{i \in [l, r]} \sum_{x \in R(i)} a_x.$$

Lời giải căn bậc hai truyền thống không còn áp dụng trực tiếp; cần chuyển góc nhìn.

4.1.1. Chuyển thành bài toán dãy

Định nghĩa dãy lân cận $\text{seq}(G)$ như sau: duyệt $i = 1$ tới n , và với mọi $j \in R(i)$, thêm j vào cuối dãy. Mỗi cạnh đóng góp hai lần nên $|\text{seq}(G)| = 2m$. Vì ta duyệt i theo thứ tự, lân cận của các đỉnh trong một đoạn $[l, r]$ tương ứng với một đoạn $[l', r']$ trên $\text{seq}(G)$. Do đó bài toán chuyển thành:

Cho một dãy s độ dài $2m$, giá trị trong $[1, n]$. Mỗi giá trị i tương ứng với trọng số a_i . Hỗ trợ:

- với l, r, v , cộng v vào a_{s_i} cho mọi $i \in [l, r]$;
- với l, r , hỏi $\sum_{i \in [l, r]} a_{s_i}$.

4.1.2. Phân khối trên dãy

Chọn độ dài khối B , chia dãy lân cận thành C khối. Với một đoạn $[l, r]$, gọi các khối nằm trọn trong đoạn là khối nguyên, còn các phần giao ở hai đầu là khối lẻ. Ta tách đóng góp của thao tác sửa tới thao tác hỏi:

- khối lẻ sửa với khối lẻ hỏi: xử lý trực tiếp $O(B)$;
- khối nguyên sửa với truy vấn: tiền xử lý $f_{i,j}$, tổng trên tiền tố j nếu mọi vị trí trong khối i được cộng 1. Khi sửa, duy trì tag_i . Khi hỏi $[l, r]$, đóng góp là

$$\sum_{i=1}^C \text{tag}_i (f_{i,r} - f_{i,l-1});$$

- khối lẻ sửa với khối nguyên hỏi: tương tự tiền xử lý $g_{i,j}$, tổng trên khối j nếu tiền tố i được cộng 1, rồi cộng dồn đóng góp của hai đầu lẻ.

Lấy $B = \sqrt{2m}$ cho thời gian $O((m+q)\sqrt{m})$ và không gian $O(m\sqrt{m})$; nếu xử lý offline theo từng khối, không gian có thể giảm về tuyến tính.

4.1.3. Quy giảm từ nhân ma trận

Xét biến thể có hai đồ thị có hướng G_1, G_2 . Với mỗi đỉnh i , ký hiệu $R_1(i), R_2(i)$ là tập đỉnh đi tới từ i trong hai đồ thị. Thao tác sửa dùng R_1 , thao tác hỏi dùng R_2 . Cách phân khối ở trên vẫn hoạt động với cách hiểu lại các mảng f, g .

Để thấy bài toán đủ mạnh, lấy $n = \sqrt{m}$ và hai ma trận 01 kích thước $n \times n$, A, B . Nếu $A_{i,j} = 1$, thêm cạnh $i \rightarrow j$ vào G_1 ; nếu $B_{i,j} = 1$, thêm cạnh $j \rightarrow i$ vào G_2 . Khi đó phần tử

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}$$

của tích $C = AB$ nhận được bằng cách sửa đoạn $[i, i]$, hỏi đoạn $[j, j]$, rồi hoàn tác. Vì vậy bài toán này không dễ hơn nhân ma trận tổng quát; độ phức tạp của lời giải phân khối là hợp lý trong khung này.

4.2. Một ví dụ khác

Ví dụ 6. Du lịch. Cho đồ thị vô hướng $G = (V, E)$ và q truy vấn. Mỗi truy vấn cho đoạn $[l, r]$, cần đếm số dãy $x_1, \dots, x_k$ thỏa:

- $x_1, x_k \in [l, r]$;
- với mọi $i < k$, $(x_i, x_{i+1}) \in E$.

Ở đây $k \in [2, 4]$ là hằng số.

4.2.1. Trường hợp $k = 2$

Đây là bài hỏi số cạnh có hai đầu trong $[l, r]$, tương đương bài đếm điểm hai chiều. Có thể xử lý offline bằng quét tuyến và cây Fenwick.

4.2.2. Trường hợp $k = 3$

Một cách trực tiếp là dùng ví dụ trước: cộng 1 vào lân cận của các đỉnh trong đoạn, rồi hỏi tổng lân cận trên đoạn, đạt $O((m + q)\sqrt{m})$. Để chuẩn bị cho mở rộng, bài viết xét cách khác bằng Mo offline. Khi dịch đầu mút, thêm đóng góp của lân cận đỉnh mới vào đáp án, rồi cộng 1 vào các đỉnh đó. Nếu đỉnh có bậc lớn, chi phí lớn; vì vậy thay vì chạy Mo trên đỉnh, chạy trên dãy lân cận $\text{seq}(G)$, đưa bài toán về cộng điểm và hỏi điểm. Phân tích Mo chuẩn cho độ phức tạp $O(m\sqrt{q})$.

4.3. Trường hợp $k = 4$

Dùng Mo hai lần offline. Trong cách Mo ở trên, chi phí dịch một điểm thực chất là đóng góp của một điểm vào một đoạn; có thể sai phân thành hai tiền tố. Bài toán trở thành: trên $\text{seq}(G)$, có $O(m)$ lần cộng 1 vào lân cận của một điểm, và $O(m\sqrt{q})$ lần hỏi trọng số của một điểm.

Tiếp tục dùng chia bậc theo ngưỡng B . Nếu đỉnh được hỏi có bậc không quá B , duyệt trực tiếp các cạnh; nếu bậc lớn, tính đóng góp ngay ở bước sửa. Khi m, q cùng bậc, lấy $B = m^{1/4}$ cho độ phức tạp $O(m^{7/4})$. Cách định hướng trên đồ thị thưa cũng đạt cùng bậc trong khung này.

5. Tổng kết

Bài viết bắt đầu từ bài toán lân cận cổ điển, khảo sát lời giải tốt hơn trên đồ thị thưa và chứng minh nó không tệ hơn cách căn bậc hai truyền thống. Sau đó bài viết mở rộng vấn đề theo hướng thao tác đoạn, chuyển lân cận trên đồ thị thành bài toán dãy có thể duy trì bằng phân khối hoặc Mo offline. Điểm chung là không cố gắng khai thác mọi cấu trúc đồ thị, mà biến quan hệ lân cận thành đối tượng dễ bảo trì hơn.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng trao đổi và học tập. Xin cảm ơn cha mẹ đã nuôi dưỡng và giáo dục. Xin cảm ơn thầy Liu Xiaogang và các bạn ở Trường Trung học số 1 Tiêu Tác đã giúp đỡ. Xin cảm ơn Wang Xiangqian, Zhi Yangfan, Zhou Zekun và Wu Yanru đã đọc và góp ý cho bài viết.

Tài liệu tham khảo

1. Wu Zuotong. *Báo cáo ra đề Công viên*. Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc IOI 2019.

Ghi chú trích xuất

Bài này được dịch từ trang PDF 88–93, tương ứng các trang in 169–178. Trang PDF 88 chứa nửa trái là kết bài trước, và trang PDF 93 chứa nửa phải là đầu bài kế tiếp; bản dịch chỉ lấy phần thuộc bài này. Bài không có hình minh họa cần nhập lại.

Bài 12. Bàn về một lớp thuật toán duy trì thông tin dựa trên chia để trị bằng cây đoạn

Huang Jiaxu, Trường Trung học trực thuộc Đại học Sư phạm Phúc Kiến

Tóm tắt

Bài viết giới thiệu một lớp thuật toán offline kết hợp chia để trị bằng cây đoạn với cấu trúc dữ liệu có thể hoàn tác. Trọng tâm là cách dùng DSU hoàn tác để duy trì thông tin của các thành phần liên thông khi cạnh chỉ xuất hiện trên một khoảng thời gian, rồi mở rộng sang các phép sửa, hỏi và phiên bản bền vững. Các ví dụ cho thấy cùng một khuôn mẫu có thể xử lý thông tin có phép nghịch đảo, thông tin chỉ có phép gộp đơn điệu, thông tin theo đoạn trên dãy, và dữ liệu trên cây phiên bản.

1. Giới thiệu chia để trị bằng cây đoạn

Chia để trị bằng cây đoạn thường được dùng khi ta biết trước toàn bộ dãy thao tác. Mỗi thông tin chỉ có hiệu lực trên một hoặc vài khoảng thời gian; ta đưa khoảng đó vào các nút của một cây đoạn theo trục thời gian. Khi DFS trên cây, ta thêm toàn bộ thông tin gắn với nút hiện tại, xử lý các con, rồi hoàn tác những thay đổi vừa thêm. Vì vậy kỹ thuật này đặc biệt phù hợp với các bài toán mà thêm thông tin dễ hơn xóa thông tin.

Ví dụ 1.1. Liên thông động offline. Cho một đồ thị vô hướng đơn, hỗ trợ thêm cạnh, xóa cạnh và hỏi hai đỉnh có liên thông hay không. Với mỗi cạnh, gom các khoảng thời gian mà cạnh tồn tại rồi đưa các khoảng đó vào cây đoạn. Trong DFS, dùng DSU hoàn tác để thêm cạnh; tại lá, trả lời truy vấn ở thời điểm tương ứng; khi quay lui, khôi phục DSU về trạng thái trước khi vào nút. Mỗi cạnh được đưa vào $O(\log m)$ nút, mỗi lần gộp DSU tốn $O(\log n)$ nếu dùng tìm gốc không nén đường đi, nên tổng thời gian là $O(m \log m \log n)$ và không gian là $O(m \log m)$.

2. Duy trì thông tin toàn cục của thành phần liên thông

Ví dụ 2.1. Tháp truyền tin. Cho đồ thị có n đỉnh và m cạnh. Đỉnh i chỉ tồn tại trong khoảng $[L_i, R_i]$. Cần xác định với mỗi đỉnh xem có thời điểm nào nó tồn tại và liên thông với đỉnh 1 hay không. Có thể nhìn bài toán như sau: tại mỗi thời điểm, cộng 1 vào toàn bộ đỉnh thuộc thành phần liên thông chứa đỉnh 1; sau cùng, đỉnh nào có tổng trọng số dương là một đáp án hợp lệ.

2.1. Thuật toán 1

Nếu thông tin trên một thành phần có phép nghịch đảo, ta có thể đặt nhãn lười trên đại diện DSU. Khi gộp tập y vào tập x , ghi lại nhãn hiện tại tag_1 của x . Khi hoàn tác, nếu nhãn mới của x là tag_2 , phần chênh lệch $\text{tag}_2 - \text{tag}_1$ chính là lượng tác động cần chuyển cho tập y . Cách này giải được ví dụ trên, nhưng phụ thuộc vào việc thông tin có thể trừ ngược lại; nhiều phép cập nhật đơn điệu không có tính chất đó.

Ví dụ 2.2. Cho đồ thị vô hướng có n đỉnh, m cạnh và trọng số đỉnh a_i . Cạnh $i = (x_i, y_i)$ chỉ tồn tại trong khoảng $[l_i, r_i]$. Cần hỗ trợ hai thao tác: hỏi trọng số lớn nhất trong thành phần liên thông hiện tại của x , và thay mọi trọng số trong thành phần đó bằng $\max(\cdot, w)$. Phép cập nhật max không có phép nghịch đảo, nên thuật toán nhãn lười ở trên không đủ.

2.2. Thuật toán 2

Xét một đại diện x . Trong quá trình DFS trên cây đoạn, các tập được gộp vào tập của x tạo thành một dãy theo thứ tự ngăn xếp: chỉ thêm ở cuối và khi quay lui cũng chỉ xóa ở cuối. Do đó mỗi đại diện duy trì một stack; phần tử trên stack lưu thông tin của đoạn đã gộp, giá trị lớn nhất tiền tố và nhãn lười. Các thao tác cần có là:

- thêm một khối thông tin vào cuối;
- xóa khối cuối khi hoàn tác;
- áp dụng cập nhật max lên toàn bộ dãy;
- hỏi giá trị lớn nhất hiện tại của dãy.

Khi xóa phần tử cuối, nhãn lười của nó được đẩy xuống phần tử mới ở đỉnh stack và sang stack của đại diện vừa tách ra. Nhờ cấu trúc LIFO của DSU hoàn tác, các phép thêm, xóa, sửa toàn cục và hỏi toàn cục đều là $O(1)$ ngoài chi phí tìm đại diện. Tổng số lần gộp do cây đoạn sinh ra là $O(m \log q)$, nên thời gian là $O(n + (m \log q + q) \log n)$ và không gian trực tiếp là $O(n + m \log q + q)$.

Một cách hiểu khác là mỗi thành phần đang được biểu diễn bởi một cây nhị phân kiểu cây tái cấu trúc Kruskal. Gộp hai tập tạo một nút cha, còn hoàn tác thì đẩy nhãn từ nút cha xuống hai con và tách cây. Các phép sửa và hỏi trên thành phần chỉ thao tác ở gốc của cây đó.

2.3. Tối ưu không gian

Nếu lưu sẵn mọi cạnh vào các nút cây đoạn, không gian là $O(m \log q)$. Ta có thể giảm không gian bằng cách truyền danh sách cạnh xuống khi đệ quy. Ở một nút, mô phỏng thao tác đưa một khoảng vào cây đoạn: những cạnh cần đi vào con trái được gom thành một đoạn đầu, những cạnh cần đi vào con phải được gom thành một đoạn khác, tương tự sắp xếp một dãy 0/1 trong thời gian tuyến tính. Nhờ vậy không cần lưu danh sách tại tất cả các nút.

Dù tổng số lần gộp vẫn là $O(m \log q)$, tại một thời điểm không dùng rollback qua nhiều nhánh đồng thời; trong DSU sống, n đỉnh chỉ có thể gộp nhiều nhất $n - 1$ lần. Kết hợp với việc tái sử dụng bộ nhớ, không gian của thuật toán 2 có thể giảm xuống $O(n + m + q)$.

2.4. Duy trì thông tin đoạn trên dãy

Bài viết còn nêu ví dụ “cây đoạn mẫu 2”: trên một dãy, hỗ trợ nhân đoạn, cộng đoạn và hỏi tổng đoạn. Xem dãy như một đường đi, mỗi thao tác trên đoạn tương ứng với việc cắt hai cạnh biên, sửa thành phần ở giữa, rồi nối lại. Áp dụng trực tiếp thuật toán 2 cho ta $O((n + q) \log q \log n)$.

Do các thành phần luôn là các đoạn liên tiếp, cạnh chỉ nối ở hai đầu. Nếu lưu đại diện DSU của hai đầu mỗi đoạn, ta xác định thành phần cần sửa trong $O(1)$. Khi đó thời gian giảm còn $O((n + q) \log q)$ và không gian là $O(n + q)$.

3. Sửa và hỏi điểm

Ví dụ 3.1. Vẫn trong mô hình đồ thị có cạnh xuất hiện theo khoảng thời gian, ngoài hai thao tác ở ví dụ 2.2, thêm hai thao tác mới: hỏi trọng số hiện tại của một đỉnh x , và đặt trọng số riêng của x thành w . Ta cần vừa duy trì thông tin toàn cục của thành phần, vừa truy cập được từng điểm bên trong thành phần đó.

3.1. Duy trì thông tin của một tập

Với một tập được biểu diễn như dãy theo thứ tự stack, các thao tác trở thành: thêm cuối, xóa cuối, sửa toàn dãy bằng nhãn max, hỏi nhãn của một phần tử bất kỳ, sửa một phần tử, và hỏi giá trị lớn nhất toàn dãy. Một stack đơn không còn đủ. Có thể dùng cây cân bằng, nhưng vì chỉ thêm và xóa ở cuối, bài viết dùng phân nhóm nhị phân kết hợp cây đoạn. Mỗi tập duy trì $O(\log n)$ cây đoạn nhỏ. Thêm hoặc xóa cuối chỉ cần gộp hoặc tách $O(\log n)$ nút, mỗi nút được xử lý $O(1)$. Sửa toàn dãy, hỏi toàn dãy, sửa điểm và hỏi điểm đều chạy trong $O(\log n)$.

3.2. Duy trì thông tin của thành phần liên thông

Khi gộp hai thành phần, trước hết tìm đại diện rồi thêm thông tin của một tập vào cuối tập kia; khi hoàn tác, hỏi nhãn cần truyền xuống tập con, xóa phần tử cuối, rồi áp nhãn đó lên tập con. Các thao tác sửa và hỏi toàn thành phần mất $O(\log n)$ sau khi đã tìm đại diện.

Với thao tác điểm, cần đi theo đường từ đỉnh được hỏi lên gốc của cây DSU, đẩy các nhãn lười trên đường đó xuống đúng điểm. Đường này có $O(\log n)$ mức, mỗi mức cần thao tác trên cấu trúc của một tập, nên sửa hoặc hỏi điểm tốn $O(\log^2 n)$. Vì vậy toàn bộ thuật toán chạy trong

$$O(n + m \log q \log n + q \log^2 n),$$

và với cách tái sử dụng bộ nhớ ở trên, không gian sống có thể giữ ở $O(n + m + q)$.

4. Duy trì thông tin có tính bền vững

Ví dụ 4.1. Cho đồ thị có n đỉnh, m cạnh và trọng số đỉnh a_i . Phiên bản 0 có toàn bộ cạnh. Mỗi thời điểm i tạo phiên bản mới dựa trên một phiên bản trước đó u , với một trong ba thao tác:

- hỏi trọng số lớn nhất trong thành phần của x ;
- cập nhật toàn bộ thành phần của x bằng $\max(\cdot, w)$;
- đảo trạng thái tồn tại của cạnh x .

Mỗi thao tác đều tạo ra phiên bản i từ phiên bản u . Các phiên bản tạo thành một cây phiên bản.

4.1. Phân tích ban đầu

Nếu chỉ xét một trục thời gian tuyến tính, mỗi lần thêm cạnh có một khoảng tác dụng và có thể phân rã thành $O(\log q)$ nút cây đoạn; các miền này lồng nhau hoặc rời nhau, nên DSU hoàn tác hoạt động tự nhiên. Trên cây phiên bản, miền tác dụng của một thay đổi là một cây con. Vì thao tác ở phiên bản mới dựa trên đúng trạng thái của phiên bản cha đã chọn, các miền tác dụng không thể bị phân rã tùy tiện thành những miền giao nhau. Đây là phần khó của bài toán bền vững.

4.2. Cấu trúc tổng thể

Thực hiện phân rã nặng–nhẹ trên cây phiên bản. Với mỗi chuỗi nặng, dựng một cây đoạn theo thứ tự DFS của các đỉnh trên chuỗi; với mỗi nhóm con nhẹ của một đỉnh, dựng một cây đoạn theo thứ tự DFS của các cây con nhẹ. Các thao tác thêm cạnh sau khi phân rã được chia thành ba loại:

- *cạnh khoảng nặng*: tại đỉnh x , cạnh có hiệu lực trên cây con của x nhưng loại bỏ cây con của con nặng;

- *cạnh khoảng nhẹ*: tại đỉnh x , cạnh có hiệu lực trên toàn bộ cây con của x ;
- *cạnh điểm*: cạnh chỉ có hiệu lực đúng tại phiên bản x .

4.3. Phân rã thao tác thêm cạnh

Với mỗi cạnh gốc của đồ thị, lấy các đỉnh phiên bản nơi trạng thái của cạnh đó thay đổi và dựng cây ảo trên tập đỉnh này. Tổng số đỉnh cây ảo trên mọi cạnh là theo bậc $O(m + q)$, còn tổng số cạnh cây ảo là $O(q)$; thời gian dựng là $O((m + q) \log q)$.

Với một đỉnh x của cây ảo, nếu thao tác tại x là xóa cạnh thì bỏ qua; nếu là thêm cạnh thì thêm một cạnh điểm ở x . Sau đó xét các cây con thật của x không chứa đỉnh cây ảo nào kế tiếp. Những cây con này là nơi trạng thái cạnh không đổi. Nếu đó là con nhẹ, đưa một cạnh khoảng nhẹ vào cây đoạn của nhóm con nhẹ; nếu đó là con nặng, đưa một cạnh khoảng nặng dọc theo chuỗi nặng đến đáy chuỗi. Mỗi cạnh của cây ảo chỉ làm tăng số đoạn phủ một lượng hằng số hoặc $O(\log q)$.

Với một cạnh cây ảo (x, y) , trong đó x là tổ tiên của y , nếu x là thao tác thêm cạnh thì phạm vi đúng phải là cây con của x trừ đi cây con của đứa con chứa y . Nếu y và đứa con đó nằm trên cùng một chuỗi nặng, ta thêm một đoạn nặng từ cha của y lên tới đứa con ấy. Nếu không, tách tại đỉnh đầu chuỗi chứa y , xử lý đoạn nặng tương ứng rồi tiếp tục trên phần còn lại. Nếu y là đầu chuỗi, chuyển về xử lý điểm ở cha của y . Mỗi lần đi qua nhiều nhất $O(\log q)$ chuỗi, nên tổng số đoạn phủ sinh ra là $O(m + q \log q)$.

4.4. Xử lý truy vấn

Sau khi các cạnh đã được phân rã, xử lý cây phiên bản bằng cùng DSU hoàn tác và cấu trúc thông tin thành phần của thuật toán 2. Khi giải một cây con có gốc là đầu chuỗi nặng x , ta:

1. duyệt cây đoạn của chuỗi nặng, thêm các cạnh khoảng nặng khi vào nút;
2. tại một lá y , thêm các cạnh điểm của y , thực hiện thao tác tại y , rồi hoàn tác các cạnh điểm;
3. duyệt cây đoạn của các con nhẹ của y , thêm các cạnh khoảng nhẹ;
4. tại lá tương ứng với một con nhẹ z , đệ quy xử lý cây con của z , sau đó hoàn tác những thao tác thuộc cây con đó;
5. khi ra khỏi một cây đoạn hoặc một chuỗi, hoàn tác toàn bộ cạnh đã thêm trong phạm vi đó.

Các phép sửa thành phần cần rollback cả nhãn lười và giá trị phụ trợ, vì vậy mỗi thay đổi được ghi vào một stack hoàn tác riêng. Cách duyệt trên bảo đảm khi trả lời một phiên bản, DSU đúng với trạng thái đồ thị của phiên bản đó.

4.5. Độ phức tạp

Số đoạn phủ sau phân rã là $O(m + q \log q)$. Đưa các đoạn này vào cây đoạn lại sinh thêm một hệ số $O(\log q)$, nên số lần thêm cạnh thực sự là $O((m + q \log q) \log q)$. Mỗi lần thêm cạnh cần tìm đại diện DSU và cập nhật thông tin thành phần, do đó tổng thời gian là

$$O(n + (m + q \log q) \log q \log n).$$

Không gian là $O(n + (m + q \log q) \log q)$, chủ yếu nằm ở các đoạn phủ và stack hoàn tác.

5. Tổng kết

Bài viết trình bày cách kết hợp chia để trị bằng cây đoạn với DSU hoàn tác để duy trì thông tin của thành phần liên thông, rồi mở rộng ý tưởng đó sang cây phiên bản bằng phân rã nặng–nhẹ. Điểm cốt lõi là giữ thứ tự LIFO của những thay đổi cần hoàn tác; khi thứ tự này còn đúng, ta có thể đặt nhiều cấu trúc dữ liệu phức tạp hơn lên trên DSU mà vẫn rollback được.

Chia để trị bằng cây đoạn và các cấu trúc có thể hoàn tác còn nhiều ứng dụng khác trong bài toán offline. Bài viết hy vọng giúp người đọc hiểu rõ hơn năng lực của kỹ thuật này và tiếp tục khai thác nó trong các mô hình duy trì thông tin khác.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu. Xin cảm ơn thầy Wang Keke, thầy Zhou Cheng và cô Huang Lihua đã quan tâm, chỉ dẫn. Xin cảm ơn cha mẹ đã quan tâm và ủng hộ. Xin cảm ơn tất cả bạn học và thầy cô đã giúp đỡ.

Tài liệu tham khảo

1. Luo Zhezheng. *Từ Unknown bàn về một lớp phương pháp duy trì thông tin đoạn hỗ trợ thêm và xóa ở cuối*. Tập luận văn đội tuyển quốc gia Olympic Tin học Trung Quốc, 2016.
2. Xu Haoran. *Bàn về vài lời giải phi cổ điển cho bài toán cấu trúc dữ liệu*. Tập luận văn đội tuyển quốc gia Olympic Tin học Trung Quốc, 2013.

Ghi chú trích xuất

Bài này được dịch từ trang PDF 93–97, tương ứng các trang in 179–186. Trang PDF 93 chứa phần cuối bài trước ở nửa trái, và trang PDF 97 chứa phần đầu bài kế tiếp ở nửa phải; bản dịch chỉ lấy phần thuộc bài này. Bài không có hình minh họa cần nhập lại.